

Minimum Edit Distance

Definition of Minimum Edit
Distance

How similar are two strings?

Spell correction

- The user typed “graffe”

Which is closest?

- graf
- graft
- grail
- giraffe

- Computational Biology

- Align two sequences of nucleotides

```
AGGCTATCACCTGACCTCCAGGCCGATGCCC  
TAGCTATCACGACCGCGGTCGATTTGCCCGAC
```

- Resulting alignment:

```
-AGGCTATCACCTGACCTCCAGGCCGA--TGCCC---  
TAG-CTATCAC--GACCGC--GGTCGATTTGCCCGAC
```

- Also for Machine Translation, Information Extraction, Speech Recognition

Edit Distance

The minimum edit distance between two strings

Is the minimum number of editing operations

- Insertion
- Deletion
- Substitution

Needed to transform one into the other

Minimum Edit Distance

Two strings and their **alignment**:

I	N	T	E	*	N	T	I	O	N
*	E	X	E	C	U	T	I	O	N

Minimum Edit Distance

I N T E * N T I O N

| | | | | | | | | |

* E X E C U T I O N

d s s i s

If each operation has cost of 1

- Distance between these is 5

If substitutions cost 2 (Levenshtein)

- Distance between them is 8

Other uses of Edit Distance in NLP

Evaluating Machine Translation and speech recognition

R Spokesman confirms senior government adviser was appointed
H Spokesman said the senior adviser was appointed
 S I D I

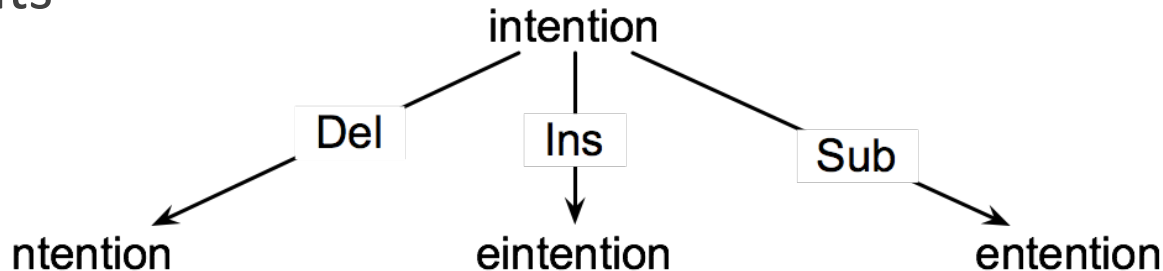
Named Entity Extraction and Entity Coreference

- **IBM Inc.** announced today
- **IBM** profits
- **Stanford Professor Jennifer Eberhardt** announced yesterday
- for **Professor Eberhardt**...

How to find the Min Edit Distance?

Searching for a path (sequence of edits) from the start string to the final string:

- **Initial state:** the word we're transforming
- **Operators:** insert, delete, substitute
- **Goal state:** the word we're trying to get to
- **Path cost:** what we want to minimize: the number of edits



Minimum Edit as Search

But the space of all edit sequences is huge!

- We can't afford to navigate naïvely
- Lots of distinct paths wind up at the same state.
 - We don't have to keep track of all of them
 - Just the shortest path to each of those revisited states.

Defining Min Edit Distance

For two strings

- X of length n
- Y of length m

We define $D(i,j)$

- the edit distance between $X[1..i]$ and $Y[1..j]$
 - i.e., the first i characters of X and the first j characters of Y
- The edit distance between X and Y is thus $D(n,m)$

Minimum Edit Distance

Definition of Minimum Edit Distance

Minimum Edit Distance

Computing Minimum Edit
Distance

Dynamic Programming for Minimum Edit Distance

Dynamic programming: A tabular computation of $D(n,m)$

Solving problems by combining solutions to subproblems.

Bottom-up

- We compute $D(i,j)$ for small i,j
- And compute larger $D(i,j)$ based on previously computed smaller values
- i.e., compute $D(i,j)$ for all i ($0 < i < n$) and j ($0 < j < m$)

Defining Min Edit Distance (Levenshtein)

Initialization

$$D(i, 0) = i$$

$$D(0, j) = j$$

Recurrence Relation:

For each $i = 1 \dots M$

For each $j = 1 \dots N$

$$D(i, j) = \min \begin{cases} D(i-1, j) + 1 \\ D(i, j-1) + 1 \\ D(i-1, j-1) + \begin{cases} 2; & \text{if } X(i) \neq Y(j) \\ 0; & \text{if } X(i) = Y(j) \end{cases} \end{cases}$$

Termination:

$D(N, M)$ is distance

The Edit Distance Table

N	9									
O	8									
I	7									
T	6									
N	5									
E	4									
T	3									
N	2									
I	1									
#	0	1	2	3	4	5	6	7	8	9
	#	E	X	E	C	U	T	I	O	N

The Edit Distance Table

N	9									
O	8									
I	7									
T	6									
N	5									
E	4									
T	3									
N	2									
I	1									
#	0	1	2	3	4	5	6	7	8	9
	#	E	X	E	C	U	T	I	O	N

$$D(i,j) = \min \begin{cases} D(i-1,j) + 1 \\ D(i,j-1) + 1 \\ D(i-1,j-1) + \begin{cases} 2; & \text{if } S_1(i) \neq S_2(j) \\ 0; & \text{if } S_1(i) = S_2(j) \end{cases} \end{cases}$$



Edit Distance

$$D(i,j) = \min \begin{cases} D(i-1,j) + 1 \\ D(i,j-1) + 1 \\ D(i-1,j-1) + \begin{cases} 2; & \text{if } S_1(i) \neq S_2(j) \\ 0; & \text{if } S_1(i) = S_2(j) \end{cases} \end{cases}$$

N	9									
O	8									
I	7									
T	6									
N	5									
E	4									
T	3									
N	2									
I	1									
#	0	1	2	3	4	5	6	7	8	9
	#	E	X	E	C	U	T	I	O	N

The Edit Distance Table

N	9	8	9	10	11	12	11	10	9	8
O	8	7	8	9	10	11	10	9	8	9
I	7	6	7	8	9	10	9	8	9	10
T	6	5	6	7	8	9	8	9	10	11
N	5	4	5	6	7	8	9	10	11	10
E	4	3	4	5	6	7	8	9	10	9
T	3	4	5	6	7	8	7	8	9	8
N	2	3	4	5	6	7	8	7	8	7
I	1	2	3	4	5	6	7	6	7	8
#	0	1	2	3	4	5	6	7	8	9
	#	E	X	E	C	U	T	I	O	N

Minimum Edit Distance

Computing Minimum Edit
Distance

Minimum Edit Distance

Backtrace for Computing
Alignments

Computing alignments

Edit distance isn't sufficient

- We often need to **align** each character of the two strings to each other

We do this by keeping a “backtrace”

Every time we enter a cell, remember where we came from

When we reach the end,

- Trace back the path from the upper right corner to read off the alignment

Edit Distance

$$D(i,j) = \min \begin{cases} D(i-1,j) + 1 \\ D(i,j-1) + 1 \\ D(i-1,j-1) + \begin{cases} 2; & \text{if } S_1(i) \neq S_2(j) \\ 0; & \text{if } S_1(i) = S_2(j) \end{cases} \end{cases}$$

N	9									
O	8									
I	7									
T	6									
N	5									
E	4									
T	3									
N	2									
I	1									
#	0	1	2	3	4	5	6	7	8	9
	#	E	X	E	C	U	T	I	O	N

MinEdit with Backtrace

n	9	↓ 8	↙←↓ 9	↙←↓ 10	↙←↓ 11	↙←↓ 12	↓ 11	↓ 10	↓ 9	↙ 8	
o	8	↓ 7	↙←↓ 8	↙←↓ 9	↙←↓ 10	↙←↓ 11	↓ 10	↓ 9	↙ 8	← 9	
i	7	↓ 6	↙←↓ 7	↙←↓ 8	↙←↓ 9	↙←↓ 10	↓ 9	↙ 8	← 9	← 10	
t	6	↓ 5	↙←↓ 6	↙←↓ 7	↙←↓ 8	↙←↓ 9	↙ 8	← 9	← 10	←↓ 11	
n	5	↓ 4	↙←↓ 5	↙←↓ 6	↙←↓ 7	↙←↓ 8	↙←↓ 9	↙←↓ 10	↙←↓ 11	↙↓ 10	
e	4	↙ 3	← 4	↙← 5	← 6	← 7	←↓ 8	↙←↓ 9	↙←↓ 10	↓ 9	
t	3	↙←↓ 4	↙←↓ 5	↙←↓ 6	↙←↓ 7	↙←↓ 8	↙ 7	←↓ 8	↙←↓ 9	↓ 8	
n	2	↙←↓ 3	↙←↓ 4	↙←↓ 5	↙←↓ 6	↙←↓ 7	↙←↓ 8	↓ 7	↙←↓ 8	↙ 7	
i	1	↙←↓ 2	↙←↓ 3	↙←↓ 4	↙←↓ 5	↙←↓ 6	↙←↓ 7	↙ 6	← 7	← 8	
#	0	1	2	3	4	5	6	7	8	9	
	#	e	x	e	c	u	t	i	o	n	

Adding Backtrace to Minimum Edit Distance

Base conditions:

$$D(i, 0) = i$$

$$D(0, j) = j$$

Termination:

$$D(N, M) \text{ is distance}$$

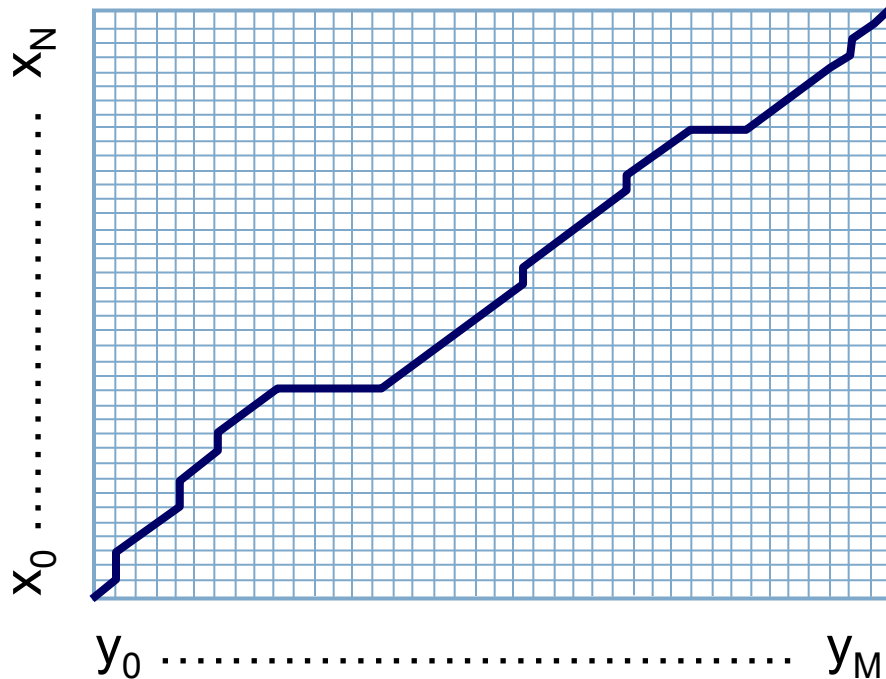
Recurrence Relation:

For each $i = 1 \dots M$

For each $j = 1 \dots N$

$$D(i, j) = \min \begin{cases} D(i-1, j) + 1 & \text{deletion} \\ D(i, j-1) + 1 & \text{insertion} \\ D(i-1, j-1) + \begin{cases} 2; & \text{if } X(i) \neq Y(j) \\ 0; & \text{if } X(i) = Y(j) \end{cases} & \text{substitution} \end{cases}$$
$$\text{ptr}(i, j) = \begin{cases} \text{LEFT} & \text{insertion} \\ \text{DOWN} & \text{deletion} \\ \text{DIAG} & \text{substitution} \end{cases}$$

The Distance Matrix



Every non-decreasing path

from $(0,0)$ to (M, N)

corresponds to
an alignment
of the two sequences

An optimal alignment is composed
of optimal subalignments

Result of Backtrace

Two strings and their **alignment**:

I	N	T	E	*	N	T	I	O	N
*	E	X	E	C	U	T	I	O	N

Performance

Time:

$O(nm)$

Space:

$O(nm)$

Backtrace

$O(n+m)$

Edit Distance

$$D(i,j) = \min \begin{cases} D(i-1,j) + 1 \\ D(i,j-1) + 1 \\ D(i-1,j-1) + \begin{cases} 2; & \text{if } S_1(i) \neq S_2(j) \\ 0; & \text{if } S_1(i) = S_2(j) \end{cases} \end{cases}$$

G	3			
O	2			
D	1			
#	0	1	2	3
	#	C	O	W

Edit Distance

$$D(i,j) = \min \begin{cases} D(i-1,j) + 1 \\ D(i,j-1) + 1 \\ D(i-1,j-1) + \begin{cases} 2; & \text{if } S_1(i) \neq S_2(j) \\ 0; & \text{if } S_1(i) = S_2(j) \end{cases} \end{cases}$$

G	3	4	3	4
O	2	3	2	3
D	1	2	3	4
#	0	1	2	3
	#	C	O	W

Edit Distance

$$D(i,j) = \min \begin{cases} D(i-1,j) + 1 \\ D(i,j-1) + 1 \\ D(i-1,j-1) + \begin{cases} 2; & \text{if } S_1(i) \neq S_2(j) \\ 0; & \text{if } S_1(i) = S_2(j) \end{cases} \end{cases}$$

G	3 ↓	4 ↙	3 ↙	4 ← ↙ ↓
O	2 ↓	3 ↙	2 ↙	3 ↙
D	1 ↓	2 ↙	3 ↓	4 ↓
#	0	1 ←	2 ←	3 ←
	#	C	O	W

Minimum Edit Distance

Backtrace for Computing
Alignments

Minimum Edit Distance

Weighted Minimum Edit Distance

Weighted Edit Distance

Why would we add weights to the computation?

- Spell Correction: some letters are more likely to be mistyped than others
- Biology: certain kinds of deletions or insertions are more likely than others

Confusion matrix for spelling errors

sub[X, Y] = Substitution of X (incorrect) for Y (correct)

X	Y (correct)																									
	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o	p	q	r	s	t	u	v	w	x	y	z
a	0	0	7	1	342	0	0	2	118	0	1	0	0	3	76	0	0	1	35	9	9	0	1	0	5	0
b	0	0	9	9	2	2	3	1	0	0	0	5	11	5	0	10	0	0	2	1	0	0	8	0	0	0
c	6	5	0	16	0	9	5	0	0	0	1	0	7	9	1	10	2	5	39	40	1	3	7	1	1	0
d	1	10	13	0	12	0	5	5	0	0	2	3	7	3	0	1	0	43	30	22	0	0	4	0	2	0
e	388	0	3	11	0	2	2	0	89	0	0	3	0	5	93	0	0	14	12	6	15	0	1	0	18	0
f	0	15	0	3	1	0	5	2	0	0	0	3	4	1	0	0	0	6	4	12	0	0	2	0	0	0
g	4	1	11	11	9	2	0	0	0	1	1	3	0	0	2	1	3	5	13	21	0	0	1	0	3	0
h	1	8	0	3	0	0	0	0	0	0	2	0	12	14	2	3	0	3	1	11	0	0	2	0	0	0
i	103	0	0	0	146	0	1	0	0	0	0	6	0	0	49	0	0	0	2	1	47	0	2	1	15	0
j	0	1	1	9	0	0	1	0	0	0	0	2	1	0	0	0	0	0	5	0	0	0	0	0	0	0
k	1	2	8	4	1	1	2	5	0	0	0	0	5	0	2	0	0	0	6	0	0	0	4	0	0	3
l	2	10	1	4	0	4	5	6	13	0	1	0	0	14	2	5	0	11	10	2	0	0	0	0	0	0
m	1	3	7	8	0	2	0	6	0	0	4	4	0	180	0	6	0	0	9	15	13	3	2	2	3	0
n	2	7	6	5	3	0	1	19	1	0	4	35	78	0	0	7	0	28	5	7	0	0	1	2	0	2
o	91	1	1	3	116	0	0	0	25	0	2	0	0	0	0	14	0	2	4	14	39	0	0	0	18	0
p	0	11	1	2	0	6	5	0	2	9	0	2	7	6	15	0	0	1	3	6	0	4	1	0	0	0
q	0	0	1	0	0	0	27	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
r	0	14	0	30	12	2	2	8	2	0	5	8	4	20	1	14	0	0	12	22	4	0	0	1	0	0
s	11	8	27	33	35	4	0	1	0	1	0	27	0	6	1	7	0	14	0	15	0	0	5	3	20	1
t	3	4	9	42	7	5	19	5	0	1	0	14	9	5	5	6	0	11	37	0	0	2	19	0	7	6
u	20	0	0	0	44	0	0	0	64	0	0	0	0	0	2	43	0	0	4	0	0	0	2	0	8	0
v	0	0	7	0	0	3	0	0	0	0	0	1	0	0	1	0	0	0	8	3	0	0	0	0	0	0
w	2	2	1	0	1	0	0	2	0	0	1	0	0	0	0	7	0	6	3	3	1	0	0	0	0	0
x	0	0	0	2	0	0	0	0	0	0	0	0	0	0	0	0	0	0	9	0	0	0	0	0	0	0
y	0	0	2	0	15	0	1	7	15	0	0	0	2	0	6	1	0	7	36	8	5	0	0	1	0	0
z	0	0	0	7	0	0	0	0	0	0	0	7	5	0	0	0	0	2	21	3	0	0	0	0	3	0

The main culprit



Weighted Min Edit Distance

Initialization:

$$D(0,0) = 0$$

$$D(i,0) = D(i-1,0) + \text{del}[x(i)]; \quad 1 < i \leq N$$

$$D(0,j) = D(0,j-1) + \text{ins}[y(j)]; \quad 1 < j \leq M$$

Recurrence Relation:

$$D(i,j) = \min \left\{ \begin{array}{l} D(i-1,j) + \text{del}[x(i)] \\ D(i,j-1) + \text{ins}[y(j)] \\ D(i-1,j-1) + \text{sub}[x(i),y(j)] \end{array} \right.$$

Termination:

$D(N,M)$ is distance

Where did the name, dynamic programming, come from?

...The 1950s were not good years for mathematical research. [the] Secretary of Defense ...had a pathological fear and hatred of the word, research...

I decided therefore to use the word, “**programming**”.

I wanted to get across the idea that this was dynamic, this was multistage... I thought, let's ... take a word that has an absolutely precise meaning, namely **dynamic**... it's impossible to use the word, **dynamic**, in a pejorative sense. Try thinking of some combination that will possibly give it a pejorative meaning. It's impossible.

Thus, I thought dynamic programming was a good name. It was something not even a Congressman could object to.”

Richard Bellman, “Eye of the Hurricane: an autobiography” 1984.

Minimum Edit Distance

Weighted Minimum Edit Distance

Chapter 2

Regular Expressions, Text
Normalization, and Edit
Distance

Basic Text Processing

Regular Expressions

Regular expressions

A formal language for specifying text strings

How can we search for any of these?

- woodchuck
- woodchucks
- Woodchuck
- Woodchucks



Regular Expressions: Disjunctions

Letters inside square brackets []

Pattern	Matches
[wW]oodchuck	Woodchuck, woodchuck
[1234567890]	Any digit

Ranges [A-Z]

Pattern	Matches	
[A-Z]	An upper case letter	<u>D</u> renched Blossoms
[a-z]	A lower case letter	<u>m</u> y beans were impatient
[0-9]	A single digit	Chapter <u>1</u> : Down the Rabbit Hole

Regular Expressions: Negation in Disjunction

Negations [^Ss]

- Carat means negation only when first in []

Pattern	Matches	
[^A-Z]	Not an upper case letter	O <u>y</u> fn pripetchik
[^Ss]	Neither 'S' nor 's'	<u>I</u> have no exquisite reason"
[^e^]	Neither e nor ^	Look h <u>e</u> re
a\^b	The pattern a carat b	Look up <u>a^b</u> now

Regular Expressions: More Disjunction

Woodchuck is another name for groundhog!

The pipe | for disjunction

Pattern	Matches
<code>groundhog woodchuck</code>	<code>woodchuck</code>
<code>yours mine</code>	<code>yours</code>
<code>a b c</code>	<code>= [abc]</code>
<code>[gG]roundhog [Ww]oodchuck</code>	<code>Woodchuck</code>



Regular Expressions: ? * + .

Pattern	Matches	
colou?r	Optional previous char	<u>color</u> <u>colour</u>
oo*h!	0 or more of previous char	<u>oh!</u> <u>ooh!</u> <u>oooh!</u> <u>ooooh!</u>
o+h!	1 or more of previous char	<u>oh!</u> <u>ooh!</u> <u>oooh!</u> <u>ooooh!</u>
baa+		<u>baa</u> <u>baaa</u> <u>baaaa</u> <u>baaaaa</u>
beg.n		<u>begin</u> <u>begun</u> <u>begun</u> <u>beg3n</u>



Stephen C Kleene

Kleene *, Kleene +

Regular Expressions: Anchors [^] ^{\$}

Pattern	Matches
[^] [A-Z]	<u>P</u> alo Alto
[^] [[^] A-Za-z]	<u>1</u> <u>"</u> Hello"
\. ^{\$}	The end <u>.</u>
. ^{\$}	The end <u>?</u> The end <u>!</u>

Example

Find me all instances of the word “the” in a text.

- `the`
 - Misses capitalized examples
- `[tT]he`
 - Incorrectly returns other or theology
- `[tT]he\b`

Errors

The process we just went through was based on fixing two kinds of errors:

1. Matching strings that we should not have matched
(there, then, other)

False positives (Type I errors)

2. Not matching things that we should have matched
(The)

False negatives (Type II errors)

Errors cont.

In NLP we are always dealing with these kinds of errors.

Reducing the error rate for an application often involves two antagonistic efforts:

- Increasing accuracy or precision (minimizing false positives)
- Increasing coverage or recall (minimizing false negatives).

Summary

Regular expressions play a surprisingly large role

- Sophisticated sequences of regular expressions are often the first model for any text processing text

For hard tasks, we use machine learning classifiers

- But regular expressions are still used for pre-processing, or as features in the classifiers
- Can be very useful in capturing generalizations

Basic Text Processing

More Regular Expressions: Substitutions and ELIZA

Substitutions

Substitution in Python and UNIX commands:

sed is the UNIX String Editor

```
sed 's/regex1/pattern/'
```

e.g.:

```
sed 's/colour/color/'
```

Capture Groups

- Say we want to put angles around all numbers:
the 35 boxes → *the <35> boxes*
 - Use parens () to "capture" a pattern into a numbered register (1, 2, 3...)
 - Use \1 to refer to the contents of the register
 - -E is the switch to indicate the pattern is regexsed
- ```
sed -E 's/([0-9]+)/<\1>/g'
```



# Capture groups: multiple registers

/the (.\*?)er they (.\*), the \1er we \2/

Matches

*the faster they ran, the faster we ran*

But not

*the faster they ran, the faster we ate*

# But suppose we don't want to capture?

Parentheses have a double function: grouping terms, and capturing

Non-capturing groups: add a `?:` after paren:

```
/(?:some|a few) (people|cats) like some \1/
matches
```

- some cats like some cats

but not

- some cats like some some

# Lookahead assertions

`(?= pattern)` is true if pattern matches, but is **zero-width; doesn't advance character pointer**

`(?! pattern)` true if a pattern does not match

How to match, at the beginning of a line, any single word that doesn't start with "Volcano":

```
/^(?!Volcano)[A-Za-z]+/
```

# Words and Corpora

## Basic Text Processing

# How many words in a sentence?

"I do uh main- mainly business data processing"

- Fragments, filled pauses

"Seuss's **cat** in the hat is different from other **cats**!"

- **Lemma**: same stem, part of speech, rough word sense
  - **cat** and **cats** = same lemma
- **Wordform**: the full inflected surface form
  - **cat** and **cats** = different wordforms

# How many words in a sentence?

they lay back on the San Francisco grass and looked at the stars  
and their

**Type:** an element of the vocabulary.

**Token:** an instance of that type in running text.

How many?

- 15 tokens (or 14)
- 13 types (or 12) (or 11?)

# How many words in a corpus?

$N$  = number of tokens

$V$  = vocabulary = set of types,  $|V|$  is size of vocabulary

Heaps' Law = Herdan's Law =  $|V| = kN^\beta$  = where often  $.67 < \beta < .75$

i.e., vocabulary size grows with  $>$  square root of the number of word tokens

|                                 | Tokens = $N$ | Types = $ V $ |
|---------------------------------|--------------|---------------|
| Switchboard phone conversations | 2.4 million  | 20 thousand   |
| Shakespeare                     | 884,000      | 31 thousand   |
| COCA                            | 440 million  | 2 million     |
| Google N-grams                  | 1 trillion   | 13+ million   |

# Corpora

Words don't appear out of nowhere!

A text is produced by

- a specific writer(s),
- at a specific time,
- in a specific variety,
- of a specific language,
- for a specific function.



# Corpora vary along dimension like

- **Language:** 7097 languages in the world
- **Variety**, like African American Language varieties.
  - AAE Twitter posts might include forms like "*iont*" (*I don't*)
- **Code switching**, e.g., Spanish/English, Hindi/English:
  - S/E: Por primera vez veo a @username actually being hateful! It was beautiful:)  
*[For the first time I get to see @username actually being hateful! it was beautiful:]*
  - H/E: dost tha or ra- hega ... dont worry ... but dherya rakhe  
*["he was and will remain a friend ... don't worry ... but have faith"]*
- **Genre:** newswire, fiction, scientific articles, Wikipedia
- **Author Demographics:** writer's age, gender, ethnicity, SES

# Corpus datasheets

Gebru et al (2020), Bender and Friedman (2018)

## **Motivation:**

- Why was the corpus collected?
- By whom?
- Who funded it?

**Situation:** In what situation was the text written?

**Collection process:** If it is a subsample how was it sampled? Was there consent? Pre-processing?

**+Annotation process, language variety, demographics, etc.**

# Words and Corpora

## Basic Text Processing

# Basic Text Processing

## Word tokenization

# Text Normalization

Every NLP task requires text normalization:

1. Tokenizing (segmenting) words
2. Normalizing word formats
3. Segmenting sentences

# Space-based tokenization

## A very simple way to tokenize

- For languages that use space characters between words
  - Arabic, Cyrillic, Greek, Latin, etc., based writing systems
- Segment off a token between instances of spaces

## Unix tools for space-based tokenization

- The "tr" command
- Inspired by Ken Church's UNIX for Poets
- Given a text file, output the word tokens and their frequencies

# Simple Tokenization in UNIX

(Inspired by Ken Church's UNIX for Poets.)

Given a text file, output the word tokens and their frequencies

```
tr -sc 'A-Za-z' '\n' < shakes.txt
```

Change all non-alpha to newlines

```
| sort
```

Sort in alphabetical order

```
| uniq -c
```

Merge and count each type

```
1945 A
 72 AARON
 19 ABBESS
 5 ABBOT
... ..
```

```
25 Aaron
 6 Abate
 1 Abates
 5 Abbess
 6 Abbey
 3 Abbot
```

```
.... ...
```

# The first step: tokenizing

```
tr -sc 'A-Za-z' '\n' < shakes.txt | head
```

THE

SONNETS

by

William

Shakespeare

From

fairest

creatures

We

...



# The second step: sorting

```
tr -sc 'A-Za-z' '\n' < shakes.txt | sort | head
```

A

A

A

A

A

A

A

A

A

...

# More counting

## Merging upper and lower case

```
tr '[:upper:]' '[:lower:]' < shakes.txt | tr -sc 'A-Za-z' '\n' | sort |
uniq -c
```

## Sorting the counts

```
tr '[:upper:]' '[:lower:]' < shakes.txt | tr -sc 'A-Za-z' '\n' | sort | uniq -c |
sort -n -r
```

```
23243 the
22225 i
18618 and
16339 to
15687 of
12780 a
12163 you
10839 my
10005 in
8954 d
```

What happened here?

# Issues in Tokenization

Can't just blindly remove punctuation:

- m.p.h., Ph.D., AT&T, cap'n
- prices (\$45.55)
- dates (01/02/06)
- URLs (<http://www.stanford.edu>)
- hashtags ([#nlproc](#))
- email addresses ([someone@cs.colorado.edu](mailto:someone@cs.colorado.edu))

Clitic: a word that doesn't stand on its own

- "are" in [we're](#), French "je" in [j'ai](#), "le" in [l'honneur](#)

When should multiword expressions (MWE) be words?

- [New York](#), [rock 'n' roll](#)

# Tokenization in NLTK

Bird, Loper and Klein (2009), *Natural Language Processing with Python*. O'Reilly

```
['That', 'U.S.A.', 'poster-print', 'costs', '$12.40', '...']
[>>> text = 'That U.S.A. poster-print costs $12.40...'
[>>> pattern = r'''(?x) # set flag to allow verbose regexps
... (?:[A-Z]\.)+ # abbreviations, e.g. U.S.A.
... | \w+(?:-\w+)* # words with optional internal hyphens
... | \$?\d+(?:\.\d+)?%?# # currency and percentages, e.g. $12.40, 82%
... | \.\.\. # ellipsis
... | [][.,;"'?:_`-] # these are separate tokens; includes], [
... '''
[>>> nltk.regexp_tokenize(text, pattern)
['That', 'U.S.A.', 'poster-print', 'costs', '$12.40', '...']
```

# Tokenization in languages without spaces

Many languages (like Chinese, Japanese, Thai) don't use spaces to separate words!

How do we decide where the token boundaries should be?

# Word tokenization in Chinese

Chinese words are composed of characters called "**hanzi**" (or sometimes just "**zi**")

Each one represents a meaning unit called a morpheme.

Each word has on average 2.4 of them.

But deciding what counts as a word is complex and not agreed upon.

# How to do word tokenization in Chinese?

姚明进入总决赛 “Yao Ming reaches the finals”

# How to do word tokenization in Chinese?

姚明进入总决赛 “Yao Ming reaches the finals”

(Yáomíng jìnrù zǒng juésài)

3 words?

姚明      进入      总决赛

YaoMing reaches finals



# How to do word tokenization in Chinese?

姚明进入总决赛 “Yao Ming reaches the finals”

3 words?

姚明      进入      总决赛

YaoMing reaches finals

5 words?

姚      明      进入      总      决赛

Yao Ming reaches overall finals

# How to do word tokenization in Chinese?

姚明进入总决赛 “Yao Ming reaches the finals”

3 words?

姚明 进入 总决赛

YaoMing reaches finals

5 words?

姚 明 进入 总 决赛

Yao Ming reaches overall finals

7 characters? (don't use words at all):

姚 明 进 入 总 决 赛

Yao Ming enter enter overall decision game

# Word tokenization / segmentation

So in Chinese it's common to just treat each character (zi) as a token.

- So the **segmentation** step is very simple

In other languages (like Thai and Japanese), more complex word segmentation is required.

- The standard algorithms are neural sequence models trained by supervised machine learning.

# Word tokenization / segmentation

ヤオ・ミン が 決勝 進出。

(Yao Min ga kesshou shinshutsu)

Foreign Entity particle verb object

- 4 separate "alphabets"
  - Katakana - loan words, emphasis, stylization
  - Hiragana - connectors, tense, particles
  - Kanji - nouns, verbs, adjectives, adverbs
  - Romaji - Western alphabet

# Basic Text Processing

## Word tokenization

# Basic Text Processing

## Byte Pair Encoding

# Another option for text tokenization

Instead of

- white-space segmentation
- single-character segmentation

**Use the data** to tell us how to tokenize.

**Subword tokenization** (because tokens can be parts of words as well as whole words)

# Subword tokenization

Three common algorithms:

- **Byte-Pair Encoding (BPE)** (Sennrich et al., 2016)
- **Unigram language modeling tokenization** (Kudo, 2018)
- **WordPiece** (Schuster and Nakajima, 2012)

All have 2 parts:

- A token **learner** that takes a raw training corpus and induces a vocabulary (a set of tokens).
- A token **segmenter** that takes a raw test sentence and tokenizes it according to that vocabulary



# Byte Pair Encoding (BPE) token learner

Let vocabulary be the set of all individual characters

= {A, B, C, D,..., a, b, c, d....}

Repeat:

- Choose the two symbols that are most frequently adjacent in the training corpus (say 'A', 'B')
- Add a new merged symbol 'AB' to the vocabulary
- Replace every adjacent 'A' 'B' in the corpus with 'AB'.

Until  $k$  merges have been done.

# BPE token learner algorithm

**function** BYTE-PAIR ENCODING(strings  $C$ , number of merges  $k$ ) **returns** vocab  $V$

$V \leftarrow$  all unique characters in  $C$                       # initial set of tokens is characters

**for**  $i = 1$  **to**  $k$  **do**                                      # merge tokens til  $k$  times

$t_L, t_R \leftarrow$  Most frequent pair of adjacent tokens in  $C$

$t_{NEW} \leftarrow t_L + t_R$                               # make new token by concatenating

$V \leftarrow V + t_{NEW}$                                   # update the vocabulary

    Replace each occurrence of  $t_L, t_R$  in  $C$  with  $t_{NEW}$               # and update the corpus

**return**  $V$

# Byte Pair Encoding (BPE) Addendum

Most subword algorithms are run inside space-separated tokens.

So we commonly first add a special end-of-word symbol '\_\_\_' before space in training corpus

Next, separate into letters.

# BPE token learner

Original (very fascinating🙄) corpus:

low low low low low lowest lowest newer newer newer  
newer newer newer wider wider wider new new

Add end-of-word tokens, resulting in this vocabulary:

**vocabulary**

\_, d, e, i, l, n, o, r, s, t, w

# BPE token learner

**corpus**

5 l o w \_  
2 l o w e s t \_  
6 n e w e r \_  
3 w i d e r \_  
2 n e w \_

**vocabulary**

\_, d, e, i, l, n, o, r, s, t, w

Merge **e r** to **er**

**corpus**

5 l o w \_  
2 l o w e s t \_  
6 n e w er \_  
3 w i d er \_  
2 n e w \_

**vocabulary**

\_, d, e, i, l, n, o, r, s, t, w, er

# BPE

## corpus

5 l o w \_  
2 l o w e s t \_  
6 n e w e r \_  
3 w i d e r \_  
2 n e w \_

## vocabulary

\_, d, e, i, l, n, o, r, s, t, w, e r

Merge **er \_** to **er\_**

## corpus

5 l o w \_  
2 l o w e s t \_  
6 n e w e r\_  
3 w i d e r\_  
2 n e w \_

## vocabulary

\_, d, e, i, l, n, o, r, s, t, w, e r, e r\_

# BPE

## corpus

5 l o w \_  
2 l o w e s t \_  
6 n e w er\_  
3 w i d er\_  
2 n e w \_

## vocabulary

\_, d, e, i, l, n, o, r, s, t, w, er, er\_

Merge **n** **e** to **ne**

## corpus

5 l o w \_  
2 l o w e s t \_  
6 ne w er\_  
3 w i d er\_  
2 ne w \_

## vocabulary

\_, d, e, i, l, n, o, r, s, t, w, er, er\_, ne

# BPE

The next merges are:

| Merge      | Current Vocabulary                                                       |
|------------|--------------------------------------------------------------------------|
| (ne, w)    | _, d, e, i, l, n, o, r, s, t, w, er, er_, ne, new                        |
| (l, o)     | _, d, e, i, l, n, o, r, s, t, w, er, er_, ne, new, lo                    |
| (lo, w)    | _, d, e, i, l, n, o, r, s, t, w, er, er_, ne, new, lo, low               |
| (new, er_) | _, d, e, i, l, n, o, r, s, t, w, er, er_, ne, new, lo, low, newer_       |
| (low, _)   | _, d, e, i, l, n, o, r, s, t, w, er, er_, ne, new, lo, low, newer_, low_ |



# BPE token segmenter algorithm

On the test data, run each merge learned from the training data:

- Greedily
- In the order we learned them
- (test frequencies don't play a role)

So: merge every **e r** to **er**, then merge **er \_** to **er\_**, etc.

Result:

- Test set "n e w e r \_" would be tokenized as a full word
- Test set "l o w e r \_" would be two tokens: "low er\_"

# Properties of BPE tokens

Usually include frequent words

And frequent subwords

- Which are often morphemes like *-est* or *-er*

A **morpheme** is the smallest meaning-bearing unit of a language

- *unlikeliest* has 3 morphemes *un-*, *likely*, and *-est*

# Byte Pair Encoding

## Basic Text Processing

# Basic Text Processing

Word Normalization and  
other issues

# Word Normalization

Putting words/tokens in a standard format

- U.S.A. or USA
- uhhuh or uh-huh
- Fed or fed
- am, is, be, are

# Case folding

Applications like IR: reduce all letters to lower case

- Since users tend to use lower case
- Possible exception: upper case in mid-sentence?
  - e.g., *General Motors*
  - *Fed* vs. *fed*
  - *SAIL* vs. *sail*

For sentiment analysis, MT, Information extraction

- Case is helpful (***US*** versus ***us*** is important)

# Lemmatization

Represent all words as their lemma, their shared root  
= dictionary headword form:

- *am, are, is* → *be*
- *car, cars, car's, cars'* → *car*
- Spanish **quiero** ('I want'), **quieres** ('you want')  
→ **querer** 'want'
- *He is reading detective stories*  
→ *He be read detective story*

# Lemmatization is done by Morphological Parsing

## Morphemes:

- The small meaningful units that make up words
- **Stems**: The core meaning-bearing units
- **Affixes**: Parts that adhere to stems, often with grammatical functions

## Morphological Parsers:

- Parse *cats* into two morphemes *cat* and *s*
- Parse Spanish *amaren* ('if in the future they would love') into morpheme *amar* 'to love', and the morphological features *3PL* and *future subjunctive*.



# Stemming

Reduce terms to stems, chopping off affixes crudely

This was not the map we found in Billy Bones's chest, but an accurate copy, complete in all things-names and heights and soundings-with the single exception of the red crosses and the written notes.



Thi wa not the map we found in Billi Bone s chest but an accur copi complet in all thing name and height and sound with the singl except of the red cross and the written note

.

# Porter Stemmer

Based on a series of rewrite rules run in series

- A cascade, in which output of each pass fed to next pass

Some sample rules:

ATIONAL → ATE (e.g., relational → relate)

ING → ε if stem contains vowel (e.g., motoring → motor)

SSES → SS (e.g., grasses → grass)

# Dealing with complex morphology is necessary for many languages

- e.g., the Turkish word:
- **Uygarlastiramadiklarimizdanmissinizcasina**
- `(behaving) as if you are among those whom we could not civilize'
- **Uygar** `civilized' + **las** `become'
  - + **tir** `cause' + **ama** `not able'
  - + **dik** `past' + **lar** `plural'
  - + **imiz** `p1pl' + **dan** `abl'
  - + **mis** `past' + **siniz** `2pl' + **casina** `as if'

# Dealing with complex morphology is necessary for many languages

- e.g., the German word:
- **Backpfeifengesicht**
- 'A face that is badly in need of a fist' or 'a punchable face'
- **Backe** 'cheek'
  - + **pfeife** 'whistle' - 'to whistle past'
  - + **gesicht** 'face'

# Sentence Segmentation

!, ? mostly unambiguous but **period** “.” is very ambiguous

- Sentence boundary
- Abbreviations like Inc. or Dr.
- Numbers like .02% or 4.3

Common algorithm: Tokenize first: use rules or ML to classify a period as either (a) part of the word or (b) a sentence-boundary.

- An abbreviation dictionary can help

Sentence segmentation can then often be done by rules based on this tokenization.

# Basic Text Processing

## Word Normalization and other issues

# Language Modeling

## Introduction to N-grams

# Probabilistic Language Models

Today's goal: assign a probability to a sentence

- Machine Translation:
  - $P(\text{high winds tonite}) > P(\text{large winds tonite})$
- Spell Correction
  - The office is about fifteen minuets from my house
    - $P(\text{about fifteen minutes from}) > P(\text{about fifteen minuets from})$
- Speech Recognition
  - $P(\text{I saw a van}) \gg P(\text{eyes awe of an})$
  - + Summarization, question-answering, etc., etc.!!

Why?



# Probabilistic Language Modeling

Goal: compute the probability of a sentence or sequence of words:

$$P(W) = P(w_1, w_2, w_3, w_4, w_5 \dots w_n)$$

Related task: probability of an upcoming word:

$$P(w_5 | w_1, w_2, w_3, w_4)$$

A model that computes either of these:

$P(W)$  or  $P(w_n | w_1, w_2 \dots w_{n-1})$  is called a **language model**.

Better: **the grammar** But **language model** or **LM** is standard

# How to compute $P(W)$

How to compute this joint probability:

- $P(\text{its, water, is, so, transparent, that})$

Intuition: let's rely on the Chain Rule of Probability

# Reminder: The Chain Rule

Recall the definition of conditional probabilities

$$p(B | A) = P(A,B)/P(A) \quad \text{Rewriting: } P(A,B) = P(A)P(B | A)$$

More variables:

$$P(A,B,C,D) = P(A)P(B | A)P(C | A,B)P(D | A,B,C)$$

The Chain Rule in General

$$P(x_1, x_2, x_3, \dots, x_n) = P(x_1)P(x_2 | x_1)P(x_3 | x_1, x_2) \dots P(x_n | x_1, \dots, x_{n-1})$$

The Chain Rule applied to compute joint probability of words in sentence

$$P(w_1 w_2 \dots w_n) = \prod_i P(w_i | W_1 w_2 \dots w_{i-1})$$

$P(\text{"its water is so transparent"}) =$

$P(\text{its}) \times P(\text{water} | \text{its}) \times P(\text{is} | \text{its water})$

$\times P(\text{so} | \text{its water is}) \times P(\text{transparent} | \text{its water is so})$

# How to estimate these probabilities

Could we just count and divide?

$$P(\text{the} \mid \text{its water is so transparent that}) = \frac{\textit{Count}(\text{its water is so transparent that the})}{\textit{Count}(\text{its water is so transparent that})}$$

No! Too many possible sentences!

We'll never see enough data for estimating these

# Markov Assumption



Andrei Markov

Simplifying assumption:

$$P(\text{the} \mid \text{its water is so transparent that}) \approx P(\text{the} \mid \text{that})$$

Or maybe

$$P(\text{the} \mid \text{its water is so transparent that}) \approx P(\text{the} \mid \text{transparent that})$$

# Markov Assumption

$$P(w_1 w_2 \dots w_n) \approx \prod_i P(w_i | w_{i-k} \dots w_{i-1})$$

In other words, we approximate each component in the product

$$P(w_i | w_1 w_2 \dots w_{i-1}) \approx P(w_i | w_{i-k} \dots w_{i-1})$$

# Simplest case: Unigram model

$$P(w_1 w_2 \dots w_n) \approx \prod_i P(w_i)$$

Some automatically generated sentences from a unigram model

fifth, an, of, futures, the, an, incorporated, a,  
a, the, inflation, most, dollars, quarter, in, is,  
mass

thrift, did, eighty, said, hard, 'm, july, bullish

that, or, limited, the



# Bigram model

- Condition on the previous word:

$$P(w_i | w_1 w_2 \dots w_{i-1}) \approx P(w_i | w_{i-1})$$

texaco, rose, one, in, this, issue, is, pursuing, growth, in,  
a, boiler, house, said, mr., gurria, mexico, 's, motion,  
control, proposal, without, permission, from, five, hundred,  
fifty, five, yen

outside, new, car, parking, lot, of, the, agreement, reached

this, would, be, a, record, november

# N-gram models

We can extend to trigrams, 4-grams, 5-grams

In general this is an insufficient model of language

- because language has **long-distance dependencies**:

“The computer which I had just put into the machine room on the fifth floor crashed.”

But we can often get away with N-gram models

# Language Modeling

## Introduction to N-grams

# Language Modeling

## Estimating N-gram Probabilities

# Estimating bigram probabilities

The Maximum Likelihood Estimate (MLE)

$$P(w_n | w_{n-1}) = \frac{C(w_{n-1}w_n)}{\sum_w C(w_{n-1}w)}$$



$$P(w_n w_{n-1}) = \frac{C(w_{n-1}w_n)}{C(w_{n-1})}$$

# An example

$$P(w_n w_{n-1}) = \frac{C(w_{n-1} w_n)}{C(w_{n-1})}$$

<s> I am Sam </s>

<s> Sam I am </s>

<s> I do not like green eggs and ham </s>

$$P(\text{I} \mid \text{<s>}) = \frac{2}{3} = .67$$

$$P(\text{Sam} \mid \text{<s>}) = \frac{1}{3} = .33$$

$$P(\text{am} \mid \text{I}) = \frac{2}{3} = .67$$

$$P(\text{</s>} \mid \text{Sam}) = \frac{1}{2} = 0.5$$

$$P(\text{Sam} \mid \text{am}) = \frac{1}{2} = .5$$

$$P(\text{do} \mid \text{I}) = \frac{1}{3} = .33$$

# MLE for N-grams

$$P(w_n | w_{n-N+1:n-1}) = \frac{C(w_{n-N+1:n-1}w_n)}{\sum_w C(w_{n-N+1:n-1})}$$

More examples:  
Berkeley Restaurant Project sentences

can you tell me about any good cantonese restaurants close by  
mid priced thai food is what i'm looking for  
tell me about chez panisse  
can you give me a listing of the kinds of food that are available  
i'm looking for a good place to eat breakfast  
when is caffe venezia open during the day



# Raw bigram counts

Out of 9222 sentences

|         | i  | want | to  | eat | chinese | food | lunch | spend |
|---------|----|------|-----|-----|---------|------|-------|-------|
| i       | 5  | 827  | 0   | 9   | 0       | 0    | 0     | 2     |
| want    | 2  | 0    | 608 | 1   | 6       | 6    | 5     | 1     |
| to      | 2  | 0    | 4   | 686 | 2       | 0    | 6     | 211   |
| eat     | 0  | 0    | 2   | 0   | 16      | 2    | 42    | 0     |
| chinese | 1  | 0    | 0   | 0   | 0       | 82   | 1     | 0     |
| food    | 15 | 0    | 15  | 0   | 1       | 4    | 0     | 0     |
| lunch   | 2  | 0    | 0   | 0   | 0       | 1    | 0     | 0     |
| spend   | 1  | 0    | 1   | 0   | 0       | 0    | 0     | 0     |

# Handling Sparse Matrices

- Store in a CSR or CSC
- Good for quick recall and memory allocation

|    |    |    |    |    |    |
|----|----|----|----|----|----|
| 10 | 20 | 0  | 0  | 0  | 0  |
| 0  | 30 | 0  | 40 | 0  | 0  |
| 0  | 0  | 50 | 60 | 70 | 0  |
| 0  | 0  | 0  | 0  | 0  | 80 |

CSR:

V = [ 10 20 30 40 50 60 70 80 ]  
COL\_INDEX = [ 0 1 1 3 2 3 4 5 ]  
ROW\_INDEX = [ 0 2 4 7 ]

# Handling Sparse Matrices

- Store in a CSR or CSC
- Good for quick recall and memory allocation

|    |    |    |    |    |    |
|----|----|----|----|----|----|
| 10 | 20 | 0  | 0  | 0  | 0  |
| 0  | 30 | 0  | 40 | 0  | 0  |
| 0  | 0  | 50 | 60 | 70 | 0  |
| 0  | 0  | 0  | 0  | 0  | 80 |

CSC:

V = [ 10 20 30 50 40 60 70 80 ]  
ROW\_INDEX = [ 0 1 1 2 3 3 4 5 ]  
COL\_INDEX = [ 0 0 2 1 2 3 ]

# Raw bigram probabilities

Normalize by unigrams:

Result:

| i    | want | to   | eat | chinese | food | lunch | spend |
|------|------|------|-----|---------|------|-------|-------|
| 2533 | 927  | 2417 | 746 | 158     | 1093 | 341   | 278   |

|         | i       | want | to     | eat    | chinese | food   | lunch  | spend   |
|---------|---------|------|--------|--------|---------|--------|--------|---------|
| i       | 0.002   | 0.33 | 0      | 0.0036 | 0       | 0      | 0      | 0.00079 |
| want    | 0.0022  | 0    | 0.66   | 0.0011 | 0.0065  | 0.0065 | 0.0054 | 0.0011  |
| to      | 0.00083 | 0    | 0.0017 | 0.28   | 0.00083 | 0      | 0.0025 | 0.087   |
| eat     | 0       | 0    | 0.0027 | 0      | 0.021   | 0.0027 | 0.056  | 0       |
| chinese | 0.0063  | 0    | 0      | 0      | 0       | 0.52   | 0.0063 | 0       |
| food    | 0.014   | 0    | 0.014  | 0      | 0.00092 | 0.0037 | 0      | 0       |
| lunch   | 0.0059  | 0    | 0      | 0      | 0       | 0.0029 | 0      | 0       |
| spend   | 0.0036  | 0    | 0.0036 | 0      | 0       | 0      | 0      | 0       |

# Bigram estimates of sentence probabilities

$$P(i | \langle s \rangle) = 0.25$$

$$P(\text{english} | \text{want}) = 0.0011$$

$$P(\text{food} | \text{english}) = 0.5$$

$$P(\langle /s \rangle | \text{food}) = 0.68$$

$$P(\langle s \rangle \text{ I want english food } \langle /s \rangle) =$$

$$P(I | \langle s \rangle)$$

$$\times P(\text{want} | I)$$

$$\times P(\text{english} | \text{want})$$

$$\times P(\text{food} | \text{english})$$

$$\times P(\langle /s \rangle | \text{food})$$

$$= .000031$$

What kinds of language knowledge can we derive from bigrams?

$$P(\text{english} | \text{want}) = .0011$$

$$P(\text{chinese} | \text{want}) = .0065$$

$$P(\text{to} | \text{want}) = .66$$

$$P(\text{eat} | \text{to}) = .28$$

$$P(\text{food} | \text{to}) = 0$$

$$P(\text{want} | \text{spend}) = 0$$

$$P(i | \langle s \rangle) = .25$$

# Practical Issues

- These probabilities will get *very* small
  - So we do everything in log space
    - Avoid underflow
    - (also adding is faster than multiplying)
- $$\log(p_1 p_2 p_3 p_4) = \log p_1 + \log p_2 + \log p_3 + \log p_4$$

# Google N-Gram Release, August 2006

AUG

3

## All Our N-gram are Belong to You

Posted by Alex Franz and Thorsten Brants, Google Machine Translation Team

Here at Google Research we have been using word [n-gram models](#) for a variety of R&D projects,

...

That's why we decided to share this enormous dataset with everyone. We processed 1,024,908,267,229 words of running text and are publishing the counts for all 1,176,470,663 five-word sequences that appear at least 40 times. There are 13,588,391 unique words, after discarding words that appear less than 200 times.



# Google N-Gram Release

serve as the incoming 92  
serve as the incubator 99  
serve as the independent 794  
serve as the index 223  
serve as the indication 72  
serve as the indicator 120  
serve as the indicators 45  
serve as the indispensable 111  
serve as the indispensable 40  
serve as the individual 234

# Google Book N-grams

<https://books.google.com/ngrams>

# Language Modeling

## Estimating N-gram Probabilities

# Language Modeling

## Evaluation and Perplexity

# Evaluation: How good is our model?

Does our language model prefer good sentences to bad ones?

- Assign higher probability to “real” or “frequently observed” sentences than “ungrammatical” or “rarely observed” sentences?

We train parameters of our model on a **training set**.

We test the model's performance on data we haven't seen.

- A **test set** is an unseen dataset that is different from our training set, totally unused.
- An **evaluation metric** tells us how well our model does on the test set.

# Extrinsic evaluation of N-gram models

## Best evaluation for comparing models A and B

- Put each model in a task
  - spelling corrector, speech recognizer, MT system
- Run the task, get an accuracy for A and for B
  - How many misspelled words corrected properly
  - How many words translated correctly
- Compare accuracy for A and B

# Difficulty of extrinsic (in-vivo) evaluation of N-gram models

## Extrinsic evaluation

- Time-consuming; can take days or weeks

So

- Sometimes use **intrinsic** evaluation: **perplexity**
- Bad approximation
  - unless the test data looks **just** like the training data
  - So **generally only useful in pilot experiments**
- But is helpful to think about.

# Intuition of Perplexity

## The Shannon Game:

- How well can we predict the next word?

I always order pizza with cheese and \_\_\_\_\_

The 33<sup>rd</sup> President of the US was \_\_\_\_\_

I saw a \_\_\_\_\_

- Unigrams are terrible at this game. (Why?)

mushrooms 0.1

pepperoni 0.1

anchovies 0.01

....

fried rice 0.0001

....

and 1e-100

## A better model of a text

- is one which assigns a higher probability to the word that actually occurs



# Perplexity

The best language model is one that best predicts an unseen test set

- Gives the highest  $P(\text{sentence})$

Perplexity is the inverse probability of the test set, normalized by the number of words:

$$PP(W) = P(w_1 w_2 \dots w_N)^{-\frac{1}{N}}$$

$$= \sqrt[N]{\frac{1}{P(w_1 w_2 \dots w_N)}}$$

Chain rule:

$$PP(W) = \sqrt[N]{\prod_{i=1}^N \frac{1}{P(w_i | w_1 \dots w_{i-1})}}$$

For bigrams:

$$PP(W) = \sqrt[N]{\prod_{i=1}^N \frac{1}{P(w_i | w_{i-1})}}$$

**Minimizing perplexity is the same as maximizing probability**

# The Shannon Game intuition for perplexity



Claude Shannon

How hard is the task of recognizing digits '0,1,2,3,4,5,6,7,8,9'

- Perplexity 10

How hard is recognizing (30,000) names at Microsoft.

- Perplexity = 30,000

Let's imagine a call-routing phone system gets 120K calls and has to recognize

- "Operator" (let's say this occurs 1 in 4 calls)
- "Sales" (1 in 4)
- "Technical Support" (1 in 4)
- 30,000 different names (each name occurring 1 time in the 120K calls)
- What is the perplexity? Next slide

# The Shannon Game intuition for perplexity

Imagine a call-routing phone system gets 120K calls and has to recognize

- "Operator" (let's say this occurs 1 in 4 calls)
- "Sales" (1 in 4)
- "Technical Support" (1 in 4)
- 30,000 different names (each name occurring 1 time in the 120K calls)

We get the perplexity of this sequence of length 120K by first multiplying 120K probabilities (90K of which are  $1/4$  and 30K of which are  $1/120K$ ), and then taking the inverse 120,000th root:

$$\text{Perp} = (\frac{1}{4} * \frac{1}{4} * \frac{1}{4} * \frac{1}{4} * \frac{1}{4} * \dots * \frac{1}{120K} * \frac{1}{120K} * \dots)^{(-1/120K)}$$

But this can be arithmetically simplified to just  $N = 4$ : the operator ( $1/4$ ), the sales ( $1/4$ ), the tech support ( $1/4$ ), and the 30,000 names ( $1/120,000$ ):

$$\text{Perplexity} = ((\frac{1}{4} * \frac{1}{4} * \frac{1}{4} * \frac{1}{120K})^{(-1/4)}) = 52.6$$

# Perplexity as branching factor

Let's suppose a sentence consisting of random digits

What is the perplexity of this sentence according to a model that assign  $P=1/10$  to each digit?

$$\begin{aligned}\text{PP}(W) &= P(w_1 w_2 \dots w_N)^{-\frac{1}{N}} \\ &= \left(\frac{1}{10}\right)^{-\frac{1}{N}} \\ &= \frac{1}{10}^{-1} \\ &= 10\end{aligned}$$

# Lower perplexity = better model

Training 38 million words, test 1.5 million words, WSJ

| N-gram<br>Order | Unigram | Bigram | Trigram |
|-----------------|---------|--------|---------|
| Perplexity      | 962     | 170    | 109     |

# Testing Using Perplexity

1. Train your model on the training data
  1. Most of the data you have collected (~90%)
2. Use the held out test data set (~10%) to calculate perplexity:

$$P = \sqrt[N]{-\sum_{i=1}^n \log_{10} P(\text{sentence}_i)}$$

where N=# sentences in the test set,

# Testing Using Perplexity

$$P = \sqrt[n]{\sum_{i=1}^n \log_{10} P(\text{sentence}_i)}$$

1. For each sentence in the test set
  1. calculate the probability of the sentence
  2. take the  $\log_{10}$  of the probability
  3. Add result to a running total of all the probabilities
2. Take the inverse Nth root of the total

# Example Test

|         | i       | want | to     | eat    | chinese | food   | lunch  | spend   |
|---------|---------|------|--------|--------|---------|--------|--------|---------|
| i       | 0.002   | 0.33 | 0      | 0.0036 | 0       | 0      | 0      | 0.00079 |
| want    | 0.0022  | 0    | 0.66   | 0.0011 | 0.0065  | 0.0065 | 0.0054 | 0.0011  |
| to      | 0.00083 | 0    | 0.0017 | 0.28   | 0.00083 | 0      | 0.0025 | 0.087   |
| eat     | 0       | 0    | 0.0027 | 0      | 0.021   | 0.0027 | 0.056  | 0       |
| chinese | 0.0063  | 0    | 0      | 0      | 0       | 0.52   | 0.0063 | 0       |
| food    | 0.014   | 0    | 0.014  | 0      | 0.00092 | 0.0037 | 0      | 0       |
| lunch   | 0.0059  | 0    | 0      | 0      | 0       | 0.0029 | 0      | 0       |
| spend   | 0.0036  | 0    | 0.0036 | 0      | 0       | 0      | 0      | 0       |

$P(\text{want} | i) \times P(\text{to} | \text{want}) \times P(\text{eat} | \text{to}) \times P(\text{chinese} | \text{eat}) \times P(\text{food} | \text{chinese}) =$

$$0.33 \times 0.66 \times 0.28 \times 0.021 \times 0.52 = 0.00066594528$$



# Example Test

|         | i       | want | to     | eat    | chinese | food   | lunch  | spend   |
|---------|---------|------|--------|--------|---------|--------|--------|---------|
| i       | 0.002   | 0.33 | 0      | 0.0036 | 0       | 0      | 0      | 0.00079 |
| want    | 0.0022  | 0    | 0.66   | 0.0011 | 0.0065  | 0.0065 | 0.0054 | 0.0011  |
| to      | 0.00083 | 0    | 0.0017 | 0.28   | 0.00083 | 0      | 0.0025 | 0.087   |
| eat     | 0       | 0    | 0.0027 | 0      | 0.021   | 0.0027 | 0.056  | 0       |
| chinese | 0.0063  | 0    | 0      | 0      | 0       | 0.52   | 0.0063 | 0       |
| food    | 0.014   | 0    | 0.014  | 0      | 0.00092 | 0.0037 | 0      | 0       |
| lunch   | 0.0059  | 0    | 0      | 0      | 0       | 0.0029 | 0      | 0       |
| spend   | 0.0036  | 0    | 0.0036 | 0      | 0       | 0      | 0      | 0       |

$\log_{10}(0.00066594528) = -3.1766$

total += -3.1766

# Language Modeling

## Evaluation and Perplexity

# Language Modeling

Generalization and zeros

# The Shannon Visualization Method

Choose a random bigram

( $\langle s \rangle$ ,  $w$ ) according to its probability

Now choose a random bigram

( $w$ ,  $x$ ) according to its probability

And so on until we choose  $\langle /s \rangle$

Then string the words together

$\langle s \rangle$  I  
I want  
want to  
to eat  
eat Chinese  
Chinese food  
food  $\langle /s \rangle$   
I want to eat Chinese food

# Approximating Shakespeare

1

gram

–To him swallowed confess hear both. Which. Of save on trail for are ay device and rote life have

–Hill he late speaks; or! a more to leg less first you enter

2

gram

–Why dost stand forth thy canopy, forsooth; he is this palpable hit the King Henry. Live king. Follow.

–What means, sir. I confess she? then all sorts, he is trim, captain.

3

gram

–Fly, and will rid me these news of price. Therefore the sadness of parting, as they say, 'tis done.

–This shall forbid it should be branded, if renown made it empty.

4

gram

–King Henry. What! I will go seek the traitor Gloucester. Exeunt some of the watch. A great banquet serv'd in;

–It cannot be but so.

# Shakespeare as corpus

$N=884,647$  tokens,  $V=29,066$

Shakespeare produced 300,000 bigram types out of  $V^2= 844$  million possible bigrams.

- So 99.96% of the possible bigrams were never seen (have zero entries in the table)

Quadrigrams worse: What's coming out looks like Shakespeare because it *is* Shakespeare

# The Wall Street Journal is not Shakespeare (no offense)

1  
gram

Months the my and issue of year foreign new exchange's september were recession exchange new endorsed a acquire to six executives

2  
gram

Last December through the way to preserve the Hudson corporation N. B. E. C. Taylor would seem to complete the major central planners one point five percent of U. S. E. has already old M. X. corporation of living on information such as more frequently fishing to keep her

3  
gram

They also point to ninety nine point six billion dollars from two hundred four oh six three percent of the rates of interest stores as Mexico and Brazil on market conditions

# Can you guess the training set author of the LM that generated these random 3-gram sentences?

They also point to ninety nine point six billion dollars from two hundred four oh six three percent of the rates of interest stores as Mexico and gram Brazil on market conditions

This shall forbid it should be branded, if renown made it empty.

“You are uniformly charming!” cried he, with a smile of associating and now and then I bowed and they perceived a chaise and four to wish for.



# The perils of overfitting

N-grams only work well for word prediction if the test corpus looks like the training corpus

- In real life, it often doesn't
- We need to train robust models that generalize!
- One kind of generalization: Zeros!
  - Things that don't ever occur in the training set
  - But occur in the test set

# Zeros

Training set:

- ... denied the allegations
- ... denied the reports
- ... denied the claims
- ... denied the request

$$P(\text{"offer"} \mid \text{denied the}) = 0$$

- Test set

- ... denied the offer
- ... denied the loan

# Zero probability bigrams

## Bigrams with zero probability

- mean that we will assign 0 probability to the test set!

And hence we cannot compute perplexity (can't divide by 0)!

# Language Modeling

Generalization and zeros

# Language Modeling

Smoothing: Add-one  
(Laplace) smoothing

# The intuition of smoothing

When we have sparse statistics:

$P(w \mid \text{denied the})$

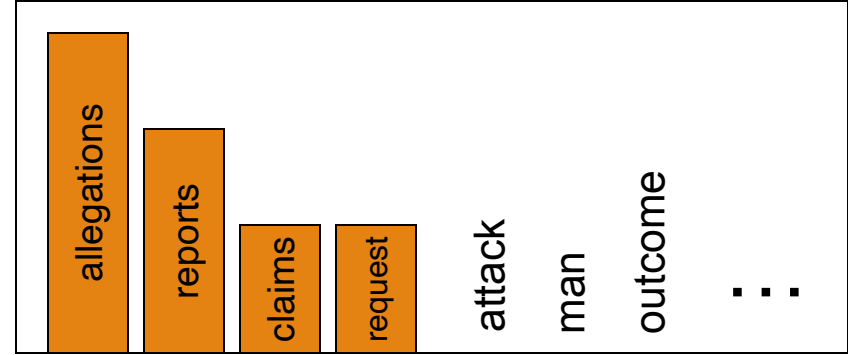
3 allegations

2 reports

1 claims

1 request

7 total



Steal probability mass to generalize better

$P(w \mid \text{denied the})$

2.5 allegations

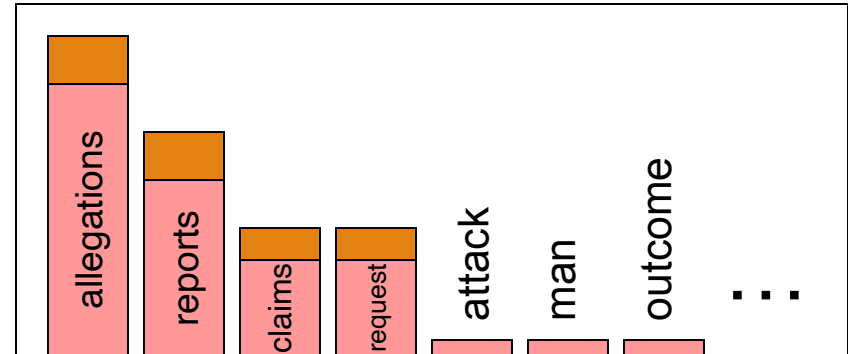
1.5 reports

0.5 claims

0.5 request

2 other

7 total



# Add-one estimation

Also called Laplace smoothing

Pretend we saw each word one more time than we did

Just add one to all the counts!



Simon-Pierre  
marquis de Laplace

MLE estimate:

$$P_{MLE}(w_i | w_{i-1}) = \frac{\alpha(w_{i-1}, w_i)}{\alpha(w_{i-1})}$$

Add-1 estimate:

$$P_{Add-1}(w_i | w_{i-1}) = \frac{\alpha(w_{i-1}, w_i) + 1}{\alpha(w_{i-1}) + V}$$

## Berkeley Restaurant Corpus: Laplace smoothed bigram counts

|         | i  | want | to  | eat | chinese | food | lunch | spend |
|---------|----|------|-----|-----|---------|------|-------|-------|
| i       | 6  | 828  | 1   | 10  | 1       | 1    | 1     | 3     |
| want    | 3  | 1    | 609 | 2   | 7       | 7    | 6     | 2     |
| to      | 3  | 1    | 5   | 687 | 3       | 1    | 7     | 212   |
| eat     | 1  | 1    | 3   | 1   | 17      | 3    | 43    | 1     |
| chinese | 2  | 1    | 1   | 1   | 1       | 83   | 2     | 1     |
| food    | 16 | 1    | 16  | 1   | 2       | 5    | 1     | 1     |
| lunch   | 3  | 1    | 1   | 1   | 1       | 2    | 1     | 1     |
| spend   | 2  | 1    | 2   | 1   | 1       | 1    | 1     | 1     |



# Laplace-smoothed bigrams

$$P^*(w_n|w_{n-1}) = \frac{C(w_{n-1}w_n) + 1}{C(w_{n-1}) + V}$$

|         | i       | want    | to      | eat     | chinese | food    | lunch   | spend   |
|---------|---------|---------|---------|---------|---------|---------|---------|---------|
| i       | 0.0015  | 0.21    | 0.00025 | 0.0025  | 0.00025 | 0.00025 | 0.00025 | 0.00075 |
| want    | 0.0013  | 0.00042 | 0.26    | 0.00084 | 0.0029  | 0.0029  | 0.0025  | 0.00084 |
| to      | 0.00078 | 0.00026 | 0.0013  | 0.18    | 0.00078 | 0.00026 | 0.0018  | 0.055   |
| eat     | 0.00046 | 0.00046 | 0.0014  | 0.00046 | 0.0078  | 0.0014  | 0.02    | 0.00046 |
| chinese | 0.0012  | 0.00062 | 0.00062 | 0.00062 | 0.00062 | 0.052   | 0.0012  | 0.00062 |
| food    | 0.0063  | 0.00039 | 0.0063  | 0.00039 | 0.00079 | 0.002   | 0.00039 | 0.00039 |
| lunch   | 0.0017  | 0.00056 | 0.00056 | 0.00056 | 0.00056 | 0.0011  | 0.00056 | 0.00056 |
| spend   | 0.0012  | 0.00058 | 0.0012  | 0.00058 | 0.00058 | 0.00058 | 0.00058 | 0.00058 |

# Reconstituted counts

$$c^*(w_{n-1}w_n) = \frac{[C(w_{n-1}w_n) + 1] \times C(w_{n-1})}{C(w_{n-1}) + V}$$

|         | i    | want  | to    | eat   | chinese | food | lunch | spend |
|---------|------|-------|-------|-------|---------|------|-------|-------|
| i       | 3.8  | 527   | 0.64  | 6.4   | 0.64    | 0.64 | 0.64  | 1.9   |
| want    | 1.2  | 0.39  | 238   | 0.78  | 2.7     | 2.7  | 2.3   | 0.78  |
| to      | 1.9  | 0.63  | 3.1   | 430   | 1.9     | 0.63 | 4.4   | 133   |
| eat     | 0.34 | 0.34  | 1     | 0.34  | 5.8     | 1    | 15    | 0.34  |
| chinese | 0.2  | 0.098 | 0.098 | 0.098 | 0.098   | 8.2  | 0.2   | 0.098 |
| food    | 6.9  | 0.43  | 6.9   | 0.43  | 0.86    | 2.2  | 0.43  | 0.43  |
| lunch   | 0.57 | 0.19  | 0.19  | 0.19  | 0.19    | 0.38 | 0.19  | 0.19  |
| spend   | 0.32 | 0.16  | 0.32  | 0.16  | 0.16    | 0.16 | 0.16  | 0.16  |

# Compare with raw bigram counts

|         | i  | want | to  | eat | chinese | food | lunch | spend |
|---------|----|------|-----|-----|---------|------|-------|-------|
| i       | 5  | 827  | 0   | 9   | 0       | 0    | 0     | 2     |
| want    | 2  | 0    | 608 | 1   | 6       | 6    | 5     | 1     |
| to      | 2  | 0    | 4   | 686 | 2       | 0    | 6     | 211   |
| eat     | 0  | 0    | 2   | 0   | 16      | 2    | 42    | 0     |
| chinese | 1  | 0    | 0   | 0   | 0       | 82   | 1     | 0     |
| food    | 15 | 0    | 15  | 0   | 1       | 4    | 0     | 0     |
| lunch   | 2  | 0    | 0   | 0   | 0       | 1    | 0     | 0     |
| spend   | 1  | 0    | 1   | 0   | 0       | 0    | 0     | 0     |

|         | i    | want  | to    | eat   | chinese | food | lunch | spend |
|---------|------|-------|-------|-------|---------|------|-------|-------|
| i       | 3.8  | 527   | 0.64  | 6.4   | 0.64    | 0.64 | 0.64  | 1.9   |
| want    | 1.2  | 0.39  | 238   | 0.78  | 2.7     | 2.7  | 2.3   | 0.78  |
| to      | 1.9  | 0.63  | 3.1   | 430   | 1.9     | 0.63 | 4.4   | 133   |
| eat     | 0.34 | 0.34  | 1     | 0.34  | 5.8     | 1    | 15    | 0.34  |
| chinese | 0.2  | 0.098 | 0.098 | 0.098 | 0.098   | 8.2  | 0.2   | 0.098 |
| food    | 6.9  | 0.43  | 6.9   | 0.43  | 0.86    | 2.2  | 0.43  | 0.43  |
| lunch   | 0.57 | 0.19  | 0.19  | 0.19  | 0.19    | 0.38 | 0.19  | 0.19  |
| spend   | 0.32 | 0.16  | 0.32  | 0.16  | 0.16    | 0.16 | 0.16  | 0.16  |

# Add-1 estimation is a blunt instrument

So add-1 isn't used for N-grams:

- We'll see better methods

But add-1 is used to smooth other NLP models

- For text classification
- In domains where the number of zeros isn't so huge.

# Language Modeling

Smoothing: Add-one  
(Laplace) smoothing

# Language Modeling

Interpolation, Backoff, and  
Web-Scale LMs

# Backoff and Interpolation

Sometimes it helps to use **less** context

- Condition on less context for contexts you haven't learned much about

## **Backoff:**

- use trigram if you have good evidence,
- otherwise bigram, otherwise unigram

## **Interpolation:**

- mix unigram, bigram, trigram

Interpolation works better

# Linear Interpolation

## Simple interpolation

$$\begin{aligned}\hat{P}(w_n|w_{n-2}w_{n-1}) = & \lambda_1 P(w_n|w_{n-2}w_{n-1}) \\ & + \lambda_2 P(w_n|w_{n-1}) \\ & + \lambda_3 P(w_n)\end{aligned}\quad \sum_i \lambda_i = 1$$

Lambdas conditional on context:

$$\begin{aligned}\hat{P}(w_n|w_{n-2}w_{n-1}) = & \lambda_1(w_{n-2}^{n-1})P(w_n|w_{n-2}w_{n-1}) \\ & + \lambda_2(w_{n-1}^{n-1})P(w_n|w_{n-1}) \\ & + \lambda_3(w_{n-2}^{n-1})P(w_n)\end{aligned}$$



# How to set the lambdas?

Use a **held-out** corpus

Training Data

Held-Out  
Data

Test  
Data

Choose  $\lambda$ s to maximize the probability of held-out data:

- Fix the N-gram probabilities (on the training data)
- Then search for  $\lambda$ s that give largest probability to held-out set:

$$\log P(w_1 \dots w_n \mid M(\lambda_1 \dots \lambda_k)) = \sum_i \log P_{M(\lambda_1 \dots \lambda_k)}(w_i \mid w_{i-1})$$

# Unknown words: Open versus closed vocabulary tasks

If we know all the words in advanced

- Vocabulary  $V$  is fixed
- Closed vocabulary task

Often we don't know this

- **Out Of Vocabulary** = OOV words
- Open vocabulary task

Instead: create an unknown word token <UNK>

- Training of <UNK> probabilities
  - Create a fixed lexicon  $L$  of size  $V$
  - At text normalization phase, any training word not in  $L$  changed to <UNK>
  - Now we train its probabilities like a normal word
- At decoding time
  - If text input: Use UNK probabilities for any word not in training

# Huge web-scale n-grams

How to deal with, e.g., Google N-gram corpus

## Pruning

- Only store N-grams with count  $>$  threshold.
- Remove singletons of higher-order n-grams
- Entropy-based pruning

## Efficiency

- Efficient data structures like tries
- Bloom filters: approximate language models
- Store words as indexes, not strings
  - Use Huffman coding to fit large numbers of words into two bytes
- Quantize probabilities (4-8 bits instead of 8-byte float)

# Building a Model

## Gathering Data

# Data Sets

Every model is only as good as the data it uses

There are many existing data sets that you can use to train your language model

- [WebText2](#) - All of Reddit up to April 2020
- [Bookcorpus](#) - 1.1GB of eBook text
- [Library Genesis](#) - 35GB of digitized book text
- [HuggingFace](#) - Open source AI site with links to many collections
- [CommonCrawl](#) - Free open repository of web crawl data

# Scrape Your Own Data!

Existing datasets come with a lot of issues

- Incorrect facts
  - Outdated, unverified, purposeful poisoning
- Specific word usages
  - Slang, regional terms
- Low-resource languages
  - If not English, Japanese, Spanish, Chinese...
- Innate Biases
- Noise

# Roll Your Own Language Model!

## Web Scraping 101

### 1. Identify the source web sites

- What are you trying to do with your data?
- What are you looking for?
- What **aren't** you looking for?
- What medium(s)?
- Does age of data matter?
- Dynamic vs static data?

# Roll Your Own Language Model!

## 2. Choose a scraping tool

- [BeautifulSoup \(python\)](#)
- [ScraPy \(python\)](#)
- [Selenium \(various\)](#)
- [JSoup \(java\)](#)
- [HTMLUnit \(java\)](#)
- [Tons of JavaScript options](#)
- [CPR \(C++\)](#)
- [Scalpel \(Haskell\)](#)

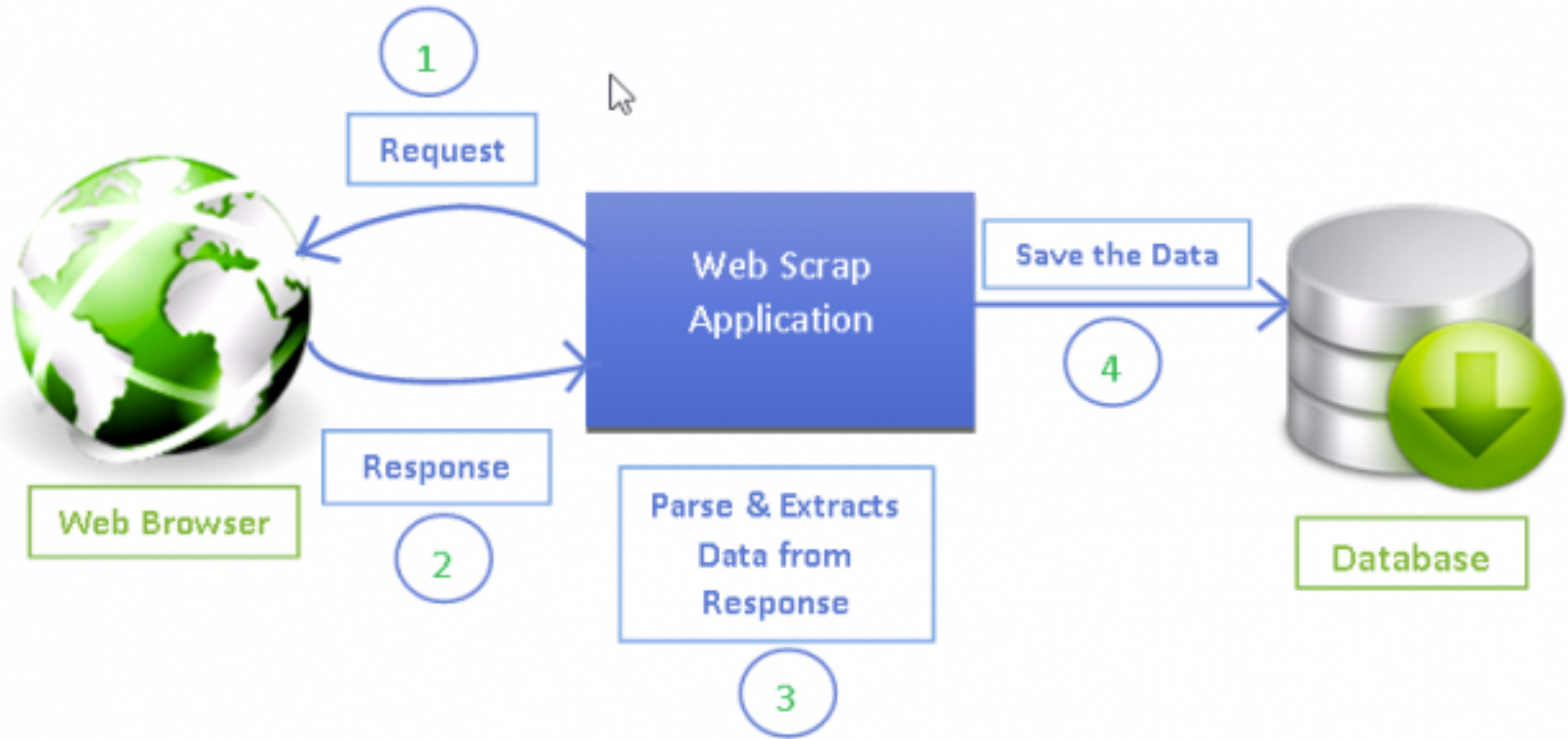


# Roll Your Own Language Model!

## 3. Write a script

- Target the type of data you want
- URLs, HTML tags, keywords, file types, etc
  - Most web text content is contained within `<p>` and `<h#>` tags
  - Modern responsive pages bury the content
  - Many sites store text in PDF files
- Dynamic Crawling?

# Roll Your Own Language Model!



# Roll Your Own Language Model!

4. Run the script - This will take time (i.e. days)

- Do not let it get too greedy (timeouts, cooldowns)
- Respect [robots.txt](#) files
- Only scrape public data

# Roll Your Own Language Model!

## 5. Clean and preprocess the data

- normalizing
- tokenizing
- de-duping
- remove HTML tags, etc
- **SAVE YOUR RAW DATA**

## 6. Train your LM

# 1. Identify Sources

- The most important part of building a LM
- Identify the task you are trying to implement
- Focus on sources that are directly related first
  - Indirect relationships are important for breadth, and will come naturally
- Make liberal use of keywords, tags, links, and other identifying features when discovering sources

## 2. Scraping Tools

- Different tools offer different features
- BeautifulSoup, Scrapy, JSoup are simple parsers
  - Downloads pages and extracts data
- Selenium is a collection of tools that automates web browsers
  - For more complex tasks
  - Much better for non-text scraping



- Common Crawl is slightly different
- It is both a scraping tool and a repository
- Archives of 15 years of web scraping
  - Literal Petabytes of TEXT
- Archives themselves can be searched and scraped
- [Tons of good tutorials](#)

### 3. Write Your Script

- Script should do a few basic things
  - Identify a starting URL or list of starting URLs
  - Set the search parameters
    - What is it looking for
    - What should it explicitly ignore
  - Check for a robots.txt file
    - Use the contents to limit the links to follow
  - Parse the page and save the data



# Example script in BeautifulSoup

```
import requests
from bs4 import BeautifulSoup

Send a request to the website
url = 'https://www.example.com'
response = requests.get(url)

Parse the HTML content using
BeautifulSoup
soup =
BeautifulSoup(response.content,
'html.parser')
```

```
Extract the data you need
title = soup.find('title').text
paragraphs = soup.find_all('p')
links = soup.find_all('a')

Print the data
print('Title:', title)
print('Paragraphs:')
for p in paragraphs:
 print(p.text)
print('Links:')
for link in links:
 print(link.get('href'))
```

## 4. Running Your Script

- Scripts are very fast compared to people
  - Often too fast
  - Include "cooldowns"
  - Queue unresponsive links
- BE CAREFUL WHAT YOU SCRAPE!
  - Pay attention to the Robots.txt file
  - Do not follow any links not whitelisted
  - [And don't tell Missouri what you find](#)

## 4a. Legal/Ethical

- Most authorities do not understand what you're doing
- Cannot stop you from doing it
  - Therefore they are afraid of it
- Very few laws have been passed
  - Because it's hard to figure out what is illegal about it
- Lawsuits are starting



## 4a. Legal/Ethical

- What should you worry about?
  - Are you monetizing this?
  - Who "owns" the text?
  - What is the purpose of the text?
  - Has the owner communicated their wishes?
  - Where will this be used?
  - How will this be used?



## 5. Clean and Preprocess Your Data

- Scraped data is a mess
  - HTML tags, spacing, page numbers, headers, footers, image captions, duplicate pages, etc, etc
- More scripts are needed to put data into a usable form
- Tokenizing happens here, along with N-Gram generation
- Filtering out unwanted terms or documents

## 5. Clean and Preprocess Your Data

- Be sure to anonymize your data before processing
  - Any identifying content that is not a public figure should be removed if not needed
    - SSN, ID#, dates, account names



## 6. Train the Language Model

- Once the data is normalized, you can run the math
- Token and/or N-Gram counting
- POS Tagging
- Smoothing/Interpolation
- Joint Probabilities
- Embeddings
- Much, much more...

# Building a Model

## Gathering Data



# Text Classification and Naive Bayes

## The Task of Text Classification

# Is this spam?

**Subject:** Important notice!

**From:** Stanford University <newsforum@stanford.edu>

**Date:** October 28, 2011 12:34:16 PM PDT

**To:** undisclosed-recipients;;

---

Greats News!

You can now access the latest news by using the link below to login to Stanford University News Forum.

<http://www.123contactform.com/contact-form-StanfordNew1-236335.html>

Click on the above link to login for more information about this new exciting forum. You can also copy the above link to your browser bar and login for more information about the new services.

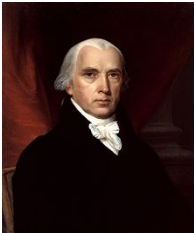
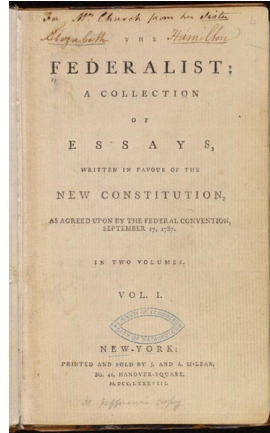
© Stanford University. All Rights Reserved.

# Who wrote which Federalist papers?

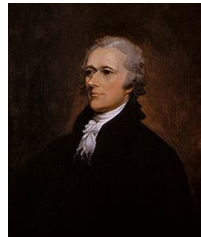
1787-8: anonymous essays try to convince New York to ratify U.S Constitution: Jay, Madison, Hamilton.

Authorship of 12 of the letters in dispute

1963: solved by Mosteller and Wallace using Bayesian methods



James Madison



Alexander Hamilton

# What is the subject of this medical article?

## MEDLINE Article



# MeSH Subject Category Hierarchy

# Antagonists and Inhibitors

# Blood Supply

# Chemistry

# Drug Therapy

# Embryology

# Epidemiology

...

# Positive or negative review?

- + *...zany characters and richly applied satire, and some great plot twists*
- *It was pathetic. The worst part about it was the boxing scenes...*
- + *...awesome caramel sauce and sweet toasty almonds. I love this place!*
- *...awful pizza and ridiculously overpriced...*

# Positive or negative movie review?

+ ...zany characters and **richly** applied satire, and some **great** plot twists

— It was **pathetic**. The **worst** part about it was the boxing scenes...

+ ...**awesome** caramel sauce and sweet toasty almonds. I **love** this place!

— ...**awful** pizza and **ridiculously** overpriced...

# Basic Sentiment Classification

Sentiment analysis is the detection of **attitudes**

Simple task we focus on in this chapter

- Is the attitude of this text positive or negative?

We return to affect classification in later chapters

# Summary: Text Classification

Sentiment analysis

Spam detection

Authorship identification

Language Identification

Assigning subject categories, topics, or genres

...



# Text Classification: definition

*Input:*

- a document  $d$
- a fixed set of classes  $C = \{c_1, c_2, \dots, c_J\}$

*Output:* a predicted class  $c \in C$

# Classification Methods: Hand-coded rules

Rules based on combinations of words or other features

- spam: black-list-address OR (“dollars” AND “you have been selected”)

Accuracy can be high

- If rules carefully refined by expert

But building and maintaining these rules is expensive

# Classification Methods:

## Supervised Machine Learning

### *Input:*

- a document  $d$
- a fixed set of classes  $C = \{c_1, c_2, \dots, c_J\}$
- A training set of  $m$  hand-labeled documents  $(d_1, c_1), \dots, (d_m, c_m)$

### *Output:*

- a learned classifier  $\gamma: d \rightarrow c$

# Classification Methods: Supervised Machine Learning

## Any kind of classifier

- Naïve Bayes
- Logistic regression
- Neural networks
- k-Nearest Neighbors
- ...

# Generative Classifier

**Generative classifiers** (like Chat-GPT) look at patterns in data and generates a probability distribution that describes those patterns

- Given a large number of cat pictures, it will identify patterns that appear in common, such as ear shape, eye size, etc
- It then builds a probability model for what features are likely to indicate the picture is a cat
- When given an unclassified picture, it compares the features of the picture to its model of a cat and returns the likelihood that the picture is, indeed a cat.

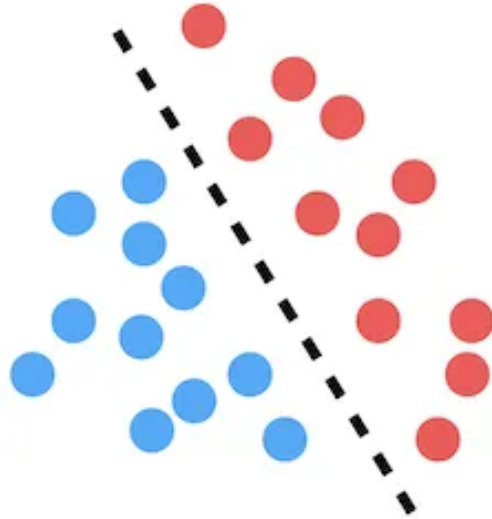
# Generative Classifier

- Thus, a model is *generated* by comparing the documents of the training data and finding patterns that appear in that data
- In NLP, these patterns are usually words, N-gram, Named Entities, and other tokens

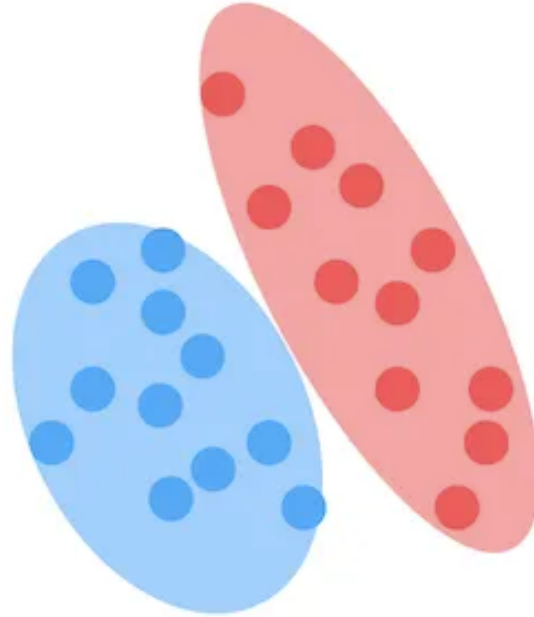
# Discriminative Classifier

- A discriminative classifier doesn't try and generate the full model of the class, but instead tries to learn the decision boundary that separates the classes
- It identifies the features that are most useful in distinguishing between one class and another

**Discriminative**



**Generative**



Naive Bayes is a **Generative Classifier**  
Logistic Regression is a **Discriminative Classifier**



# Generative vs Discriminative

| <b>Generative</b>                                  |                                            | <b>Discriminative</b>                          |                                          |
|----------------------------------------------------|--------------------------------------------|------------------------------------------------|------------------------------------------|
| Pros                                               | Cons                                       | Pros                                           | Cons                                     |
| Can handle complex patterns                        | May overfit on large datasets              | High Accurac                                   | May generalize poorly on new data        |
| Useful for small datasets                          | Computationally expensive                  | Flexible, can be trained on many ML algorithms | Requires a lot of data                   |
| Can learn from unlabeled data                      | Sensitive to outliers and severe imbalance | Works well with imbalanced data                | Does not handle missing data efficiently |
| Can capture joint distribution of input and output |                                            |                                                |                                          |

# Text Classification and Naive Bayes

## The Task of Text Classification

# Text Classification and Naive Bayes

## The Naive Bayes Classifier

# Naive Bayes Intuition

Simple ("naive") classification method based on Bayes rule

Relies on very simple representation of document

- **Bag of words**

# The Bag of Words Representation

I love this movie! It's sweet, but with satirical humor. The dialogue is great and the adventure scenes are fun... It manages to be whimsical and romantic while laughing at the conventions of the fairy tale genre. I would recommend it to just about anyone. I've seen it several times, and I'm always happy to see it again whenever I have a friend who hasn't seen it yet!



|           |     |
|-----------|-----|
| it        | 6   |
| I         | 5   |
| the       | 4   |
| to        | 3   |
| and       | 3   |
| seen      | 2   |
| yet       | 1   |
| would     | 1   |
| whimsical | 1   |
| times     | 1   |
| sweet     | 1   |
| satirical | 1   |
| adventure | 1   |
| genre     | 1   |
| fairy     | 1   |
| humor     | 1   |
| have      | 1   |
| great     | 1   |
| ...       | ... |

# The bag of words representation

$Y($

|           |     |
|-----------|-----|
| seen      | 2   |
| sweet     | 1   |
| whimsical | 1   |
| recommend | 1   |
| happy     | 1   |
| ...       | ... |

$) = C$


# Bayes Rule of Inference

$$P(H | E) = \frac{P(E | H) \cdot P(H)}{P(E)}$$

H: Hypothesis

P(H): Prior Probability

E: Evidence

P(H|E): Posterier Probability

P(E | H): Likelihood

# Bayes' Rule Applied to Documents and Classes

- For a document  $d$  and a class  $c$

$$P(c|d) = \frac{P(d|c)P(c)}{P(d)}$$



# Naive Bayes Classifier (I)

$$C_{MAP} = \operatorname{argmax}_{c \in C} P(c | d)$$

MAP is “maximum a posteriori” = most likely class

$$= \operatorname{argmax}_{c \in C} \frac{P(d | c) P(c)}{P(d)}$$

Bayes Rule

$$= \operatorname{argmax}_{c \in C} P(d | c) P(c)$$

Dropping the denominator

# Naive Bayes Classifier (II)

"Likelihood"

"Prior"

$$C_{MAP} = \operatorname{argmax}_{c \in C} P(d | c) P(c)$$

$$= \operatorname{argmax}_{c \in C} P(x_1, x_2, \dots, x_n | c) P(c)$$

Document d  
represented as  
features x1..xn

# Multinomial Naive Bayes Independence Assumptions

$$P(x_1, x_2, \dots, x_n \mid c)$$

**Bag of Words assumption:** Assume position doesn't matter

**Conditional Independence:** Assume the feature probabilities  $P(x_i \mid c_j)$  are independent given the class  $c$ .

$$P(x_1, \dots, x_n \mid c) = P(x_1 \mid c) \cdot P(x_2 \mid c) \cdot P(x_3 \mid c) \cdot \dots \cdot P(x_n \mid c)$$

# Multinomial Naive Bayes Classifier

$$C_{MAP} = \operatorname{argmax}_{c \in C} P(x_1, x_2, \dots, x_n | c) P(c)$$

$$C_{NB} = \operatorname{argmax}_{c \in C} P(c_j) \prod_{x \in X} P(x | c)$$

## Applying Multinomial Naive Bayes Classifiers to Text Classification

positions  $\leftarrow$  all word positions in test document

$$c_{NB} = \operatorname{argmax}_{c_j \in C} P(c_j) \prod_{i \in \text{positions}} P(x_i | c_j)$$

# Problems with multiplying lots of probs

There's a problem with this:

$$c_{NB} = \operatorname{argmax}_{c_j \in C} P(c_j) \prod_{i \in \text{positions}} P(x_i | c_j)$$

Multiplying lots of probabilities can result in floating-point underflow!

.0006 \* .0007 \* .0009 \* .01 \* .5 \* .000008....

Idea: Use logs, because  $\log(ab) = \log(a) + \log(b)$

We'll sum logs of probabilities instead of multiplying probabilities!

# We actually do everything in log space

Instead of this:

$$c_{NB} = \operatorname{argmax}_{c_j \in C} P(c_j) \prod_{i \in \text{positions}} P(x_i | c_j)$$

This:

$$c_{NB} = \operatorname{argmax}_{c_j \in C} \left[ \log P(c_j) + \sum_{i \in \text{positions}} \log P(x_i | c_j) \right]$$

Notes:

1) Taking log doesn't change the ranking of classes!

The class with highest probability also has highest log probability!

2) It's a linear model:

Just a max of a sum of weights: a **linear** function of the inputs

So Naïve Bayes is a **linear classifier**

# Text Classification and Naive Bayes

## The Naive Bayes Classifier



# Text Classification and Naïve Bayes

## Naive Bayes: Learning

# Learning the Multinomial Naive Bayes Model

First attempt: maximum likelihood estimates

- simply use the frequencies in the data

$$\hat{P}(c_j) = \frac{N_{c_j}}{N_{total}}$$

$$\hat{P}(w_i | c_j) = \frac{\text{count}(w_i, c_j)}{\sum_{w \in V} \text{count}(w, c_j)}$$

# Parameter estimation

$$\hat{P}(w_i | c_j) = \frac{\text{count}(w_i, c_j)}{\sum_{w \in V} \text{count}(w, c_j)}$$

fraction of times word  $w_i$  appears  
among all words in documents of topic  $c_j$

Create mega-document for topic  $j$  by concatenating all docs in this topic

- Use frequency of  $w$  in mega-document

# Problem with Maximum Likelihood

What if we have seen no training documents with the word *fantastic* and classified in the topic **positive** (*thumbs-up*)?

$$\hat{P}(\text{"fantastic"} \mid \text{positive}) = \frac{\text{count}(\text{"fantastic"}, \text{positive})}{\sum_{w \in V} \text{count}(w, \text{positive})} = 0$$

Zero probabilities cannot be conditioned away, no matter the other evidence!

$$c_{MAP} = \operatorname{argmax}_c \hat{P}(c) \prod_i \hat{P}(x_i \mid c)$$

## Laplace (add-1) smoothing for Naïve Bayes

$$\begin{aligned}\hat{P}(w_i | c) &= \frac{\text{count}(w_i, c) + 1}{\sum_{w \in V} (\text{count}(w, c) + 1)} \\ &= \frac{\text{count}(w_i, c) + 1}{\left( \sum_{w \in V} \text{count}(w, c) \right) + |V|}\end{aligned}$$

# Multinomial Naïve Bayes: Learning

- From training corpus, extract *Vocabulary*

Calculate  $P(c_j)$  terms

- For each  $c_j$  in  $C$  do

$docs_j \leftarrow$  all docs with class  $= c_j$

$$P(c_j) \leftarrow \frac{|docs_j|}{|\text{total \# documents}|}$$

- Calculate  $P(w_k | c_j)$  terms

- $Text_j \leftarrow$  single doc containing all  $docs_j$

- For each word  $w_k$  in *Vocabulary*

$n_k \leftarrow$  # of occurrences of  $w_k$  in  $Text_j$

$$P(w_k | c_j) \leftarrow \frac{n_k + \alpha}{n + \alpha | \text{Vocabulary} |}$$

# Unknown words

What about unknown words

- that appear in our test data
- but not in our training data or vocabulary?

We **ignore** them

- Remove them from the test document!
- Pretend they weren't there!
- Don't include any probability for them at all!

Why don't we build an unknown word model?

- It doesn't help: knowing which class has more unknown words is not generally helpful!

# Stop words

## Some systems ignore stop words

- **Stop words:** very frequent words like *the* and *a*.
  - Sort the vocabulary by word frequency in training set
  - Call the top 10 or 50 words the **stopword list**.
  - Remove all stop words from both training and test sets
    - As if they were never there!

## But removing stop words doesn't usually help

- So in practice most NB algorithms use **all** words and **don't** use stopwords lists



**function** TRAIN NAIVE BAYES(D, C) **returns**  $\log P(c)$  and  $\log P(w|c)$

**for each** class  $c \in C$                       # Calculate  $P(c)$  terms

$N_{doc}$  = number of documents in D

$N_c$  = number of documents from D in class c

$\logprior[c] \leftarrow \log \frac{N_c}{N_{doc}}$

$V \leftarrow$  vocabulary of D

$bigdoc[c] \leftarrow$  **append**(d) **for** d  $\in$  D **with** class c

**for each** word  $w$  in V                      # Calculate  $P(w|c)$  terms

$count(w, c) \leftarrow$  # of occurrences of  $w$  in  $bigdoc[c]$

$\loglikelihood[w, c] \leftarrow \log \frac{count(w, c) + 1}{\sum_{w' \text{ in } V} (count(w', c) + 1)}$

**return**  $\logprior, \loglikelihood, V$

**function** TEST NAIVE BAYES( $testdoc, \logprior, \loglikelihood, C, V$ ) **returns** best c

**for each** class  $c \in C$

$sum[c] \leftarrow \logprior[c]$

**for each** position  $i$  in  $testdoc$

$word \leftarrow testdoc[i]$

**if**  $word \in V$

$sum[c] \leftarrow sum[c] + \loglikelihood[word, c]$

**return**  $\operatorname{argmax}_c sum[c]$

# Text Classification and Naive Bayes

## Naive Bayes: Learning

# Text Classification and Naive Bayes

## Sentiment and Binary Naive Bayes

Let's do a worked sentiment example!

|          | Cat | Documents                             |
|----------|-----|---------------------------------------|
| Training | -   | just plain boring                     |
|          | -   | entirely predictable and lacks energy |
|          | -   | no surprises and very few laughs      |
|          | +   | very powerful                         |
|          | +   | the most fun film of the summer       |
| Test     | ?   | predictable with no fun               |

# Build Vocabulary

|          | Cat | Documents                             |
|----------|-----|---------------------------------------|
| Training | -   | just plain boring                     |
|          | -   | entirely predictable and lacks energy |
|          | -   | no surprises and very few laughs      |
|          | +   | very powerful                         |
|          | +   | the most fun film of the summer       |
| Test     | ?   | predictable with no fun               |

$V = \{\text{and, boring, energy, entirely, few, film, fun, just, lacks, laughs, most, no, of, plain, powerful, predictable, summer, surprises, the, very}\} \rightarrow |V|=20$

# Prior Probability

|          | Cat | Documents                             |
|----------|-----|---------------------------------------|
| Training | -   | just plain boring                     |
|          | -   | entirely predictable and lacks energy |
|          | -   | no surprises and very few laughs      |
|          | +   | very powerful                         |
|          | +   | the most fun film of the summer       |
| Test     | ?   | predictable with no fun               |

$$P(-) = 3 \rightarrow 3/5 = 0.6$$

$$P(+) = 2 \rightarrow 2/5 = 0.4$$

# Define classes

|          | Cat | Documents                             |
|----------|-----|---------------------------------------|
| Training | -   | just plain boring                     |
|          | -   | entirely predictable and lacks energy |
|          | -   | no surprises and very few laughs      |
|          | +   | very powerful                         |
|          | +   | the most fun film of the summer       |
| Test     | ?   | predictable with no fun               |

$N(-) = \{\text{and, and, boring, energy, entirely, few, just, lacks, laughs, no, plain, predictable, surprises, very}\} \rightarrow |N-| = 14$

$N(+) = \{\text{film, fun, most, of, powerful, summer, the, the, very}\} \rightarrow |N+| = 9$

# Train models with Laplace Smoothing

|          | Cat | Documents                             |
|----------|-----|---------------------------------------|
| Training | -   | just plain boring                     |
|          | -   | entirely predictable and lacks energy |
|          | -   | no surprises and very few laughs      |
|          | +   | very powerful                         |
|          | +   | the most fun film of the summer       |
| Test     | ?   | predictable with no fun               |

$$P_{-}(\text{and}) = (C_{-}+1) / (|N_{-}| + |V|) =$$

$$(2 + 1) / (14 + 20) = 3/34 = 0.0882,$$

$$P_{-}(\text{boring}) = (1 + 1) / (14 + 20) = 2/34 = 0.0588$$

...



# Run Test through the classifier

|          | Cat | Documents                             |
|----------|-----|---------------------------------------|
| Training | -   | just plain boring                     |
|          | -   | entirely predictable and lacks energy |
|          | -   | no surprises and very few laughs      |
|          | +   | very powerful                         |
|          | +   | the most fun film of the summer       |
| Test     | ?   | predictable with no fun               |

Test={predicable, with, no, fun}

"with" does not appear in V, so  
drop it ->

Test={predictable, no, fun}

# Likelihood based on trained models

|          | Cat | Documents                             |
|----------|-----|---------------------------------------|
| Training | -   | just plain boring                     |
|          | -   | entirely predictable and lacks energy |
|          | -   | no surprises and very few laughs      |
|          | +   | very powerful                         |
|          | +   | the most fun film of the summer       |
| Test     | ?   | predictable with no fun               |

$P(\text{"predictable"}|-) \rightarrow 2/34$

$P(\text{"no"}|-) \rightarrow 2/34$

$P(\text{"fun"}|-) \rightarrow 1/34$

$$P(-)P(\text{Test}|-) = 3/5 * 2/34 * 2/34 * 1/34$$
$$= 6.1 \times 10^{-5}$$

# Likelihood based on trained models

|          | Cat | Documents                             |
|----------|-----|---------------------------------------|
| Training | -   | just plain boring                     |
|          | -   | entirely predictable and lacks energy |
|          | -   | no surprises and very few laughs      |
|          | +   | very powerful                         |
|          | +   | the most fun film of the summer       |
| Test     | ?   | predictable with no fun               |

$P(\text{"predictable"}|+) \rightarrow 1/34$

$P(\text{"no"}|+) \rightarrow 1/34$

$P(\text{"fun"}|+) \rightarrow 2/34$

$$P(+|Test) = \frac{2}{5} * \frac{1}{34} * \frac{1}{34} * \frac{2}{34}$$
$$= 3.2 \times 10^{-5}$$

# A worked sentiment example with add-1 smoothing

|          | Cat | Documents                             |
|----------|-----|---------------------------------------|
| Training | -   | just plain boring                     |
|          | -   | entirely predictable and lacks energy |
|          | -   | no surprises and very few laughs      |
|          | +   | very powerful                         |
|          | +   | the most fun film of the summer       |
| Test     | ?   | predictable <del>with</del> no fun    |

1. Prior from training:

$$\hat{P}(c_j) = \frac{N_{c_j}}{N_{total}} \quad \begin{array}{l} P(-) = 3/5 \\ P(+) = 2/5 \end{array}$$

2. Drop "with"

3. Likelihoods from training:

$$p(w_i | c) = \frac{\text{count}(w_i, c) + 1}{\left( \sum_{w \in V} \text{count}(w, c) + 1 \right) + |V|}$$

$$\begin{array}{ll} P(\text{"predictable"}|-) = \frac{1+1}{14+20} & P(\text{"predictable"}|+) = \frac{0+1}{9+20} \\ P(\text{"no"}|-) = \frac{1+1}{14+20} & P(\text{"no"}|+) = \frac{0+1}{9+20} \\ P(\text{"fun"}|-) = \frac{0+1}{14+20} & P(\text{"fun"}|+) = \frac{1+1}{9+20} \end{array}$$

4. Scoring the test set:

$$P(-)P(S|-) = \frac{3}{5} \times \frac{2 \times 2 \times 1}{34^3} = 6.1 \times 10^{-5}$$

$$P(+ )P(S|+) = \frac{2}{5} \times \frac{1 \times 1 \times 2}{29^3} = 3.2 \times 10^{-5}$$

# Optimizing for sentiment analysis

For tasks like sentiment, word **occurrence** seems to be more important than word **frequency**.

- The occurrence of the word *fantastic* tells us a lot
- The fact that it occurs 5 times may not tell us much more.

## **Binary multinominal naive bayes, or binary NB**

- Clip our word counts at 1
- Note: this is different than Bernoulli naive bayes; see the textbook at the end of the chapter.

# Binary multinominal naive Bayes

## **Four original documents:**

- it was pathetic the worst part was the boxing scenes
- no plot twists or great scenes
- + and satire and great plot twists
- + great scenes great film

# Binary multinominal naive Bayes

## Four original documents:

- it was pathetic the worst part was the boxing scenes
- no plot twists or great scenes
- + and satire and great plot twists
- + great scenes great film

|          | NB<br>Counts |   |
|----------|--------------|---|
|          | +            | – |
| and      | 2            | 0 |
| boxing   | 0            | 1 |
| film     | 1            | 0 |
| great    | 3            | 1 |
| it       | 0            | 1 |
| no       | 0            | 1 |
| or       | 0            | 1 |
| part     | 0            | 1 |
| pathetic | 0            | 1 |
| plot     | 1            | 1 |
| satire   | 1            | 0 |
| scenes   | 1            | 2 |
| the      | 0            | 2 |
| twists   | 1            | 1 |
| was      | 0            | 2 |
| worst    | 0            | 1 |

# Binary multinominal naive Bayes

## Four original documents:

- it was pathetic the worst part was the boxing scenes
- no plot twists or great scenes
- + and satire and great plot twists
- + great scenes great film

## After per-document binarization:

- it was pathetic the worst part boxing scenes
- no plot twists or great scenes
- + and satire great plot twists
- + great scenes film

|          | NB Counts |   |
|----------|-----------|---|
|          | +         | – |
| and      | 2         | 0 |
| boxing   | 0         | 1 |
| film     | 1         | 0 |
| great    | 3         | 1 |
| it       | 0         | 1 |
| no       | 0         | 1 |
| or       | 0         | 1 |
| part     | 0         | 1 |
| pathetic | 0         | 1 |
| plot     | 1         | 1 |
| satire   | 1         | 0 |
| scenes   | 1         | 2 |
| the      | 0         | 2 |
| twists   | 1         | 1 |
| was      | 0         | 2 |
| worst    | 0         | 1 |



# Binary multinominal naive Bayes

## Four original documents:

- it was pathetic the worst part was the boxing scenes
- no plot twists or great scenes
- + and satire and great plot twists
- + great scenes great film

## After per-document binarization:

- it was pathetic the worst part boxing scenes
- no plot twists or great scenes
- + and satire great plot twists
- + great scenes film

|          | NB Counts |   | Binary Counts |   |
|----------|-----------|---|---------------|---|
|          | +         | - | +             | - |
| and      | 2         | 0 | 1             | 0 |
| boxing   | 0         | 1 | 0             | 1 |
| film     | 1         | 0 | 1             | 0 |
| great    | 3         | 1 | 2             | 1 |
| it       | 0         | 1 | 0             | 1 |
| no       | 0         | 1 | 0             | 1 |
| or       | 0         | 1 | 0             | 1 |
| part     | 0         | 1 | 0             | 1 |
| pathetic | 0         | 1 | 0             | 1 |
| plot     | 1         | 1 | 1             | 1 |
| satire   | 1         | 0 | 1             | 0 |
| scenes   | 1         | 2 | 1             | 2 |
| the      | 0         | 2 | 0             | 1 |
| twists   | 1         | 1 | 1             | 1 |
| was      | 0         | 2 | 0             | 1 |
| worst    | 0         | 1 | 0             | 1 |

Counts can still be 2! Binarization is within-doc!

# Text Classification and Naive Bayes

## Sentiment and Binary Naive Bayes

# Text Classification and Naive Bayes

## More on Sentiment Classification

# Sentiment Classification: Dealing with Negation

I really like this movie

I really **don't** like this movie

Negation changes the meaning of "like" to negative.

Negation can also change negative to positive-ish

- **Don't** dismiss this film
- **Doesn't** let us get bored

# Sentiment Classification: Dealing with Negation

Das, Sanjiv and Mike Chen. 2001. Yahoo! for Amazon: Extracting market sentiment from stock message boards. In Proceedings of the Asia Pacific Finance Association Annual Conference (APFA).

Bo Pang, Lillian Lee, and Shivakumar Vaithyanathan. 2002. Thumbs up? Sentiment Classification using Machine Learning Techniques. EMNLP-2002, 79—86.

Simple baseline method:

Add NOT\_ to every word between negation and following punctuation:

didn't like this movie , but I



didn't NOT\_like NOT\_this NOT\_movie but I

# Sentiment Classification: Lexicons

Sometimes we don't have enough labeled training data

In that case, we can make use of pre-built word lists

Called **lexicons**

There are various publicly available lexicons

# MPQA Subjectivity Cues Lexicon

Theresa Wilson, Janyce Wiebe, and Paul Hoffmann (2005). Recognizing Contextual Polarity in Phrase-Level Sentiment Analysis. Proc. of HLT-EMNLP-2005.

Riloff and Wiebe (2003). Learning extraction patterns for subjective expressions. EMNLP-2003.

Home page: [https://mpqa.cs.pitt.edu/lexicons/subj\\_lexicon/](https://mpqa.cs.pitt.edu/lexicons/subj_lexicon/)

6885 words from 8221 lemmas, annotated for intensity (strong/weak)

- 2718 positive
- 4912 negative

+ : *admirable, beautiful, confident, dazzling, ecstatic, favor, glee, great*

– : *awful, bad, bias, catastrophe, cheat, deny, envious, foul, harsh, hate*

# The General Inquirer

Philip J. Stone, Dexter C Dunphy, Marshall S. Smith, Daniel M. Ogilvie. 1966. The General Inquirer: A Computer Approach to Content Analysis. MIT Press

- Home page: <http://www.wjh.harvard.edu/~inquirer>
- List of Categories: <http://www.wjh.harvard.edu/~inquirer/homecat.htm>
- Spreadsheet: <http://www.wjh.harvard.edu/~inquirer/inquirerbasic.xls>

## Categories:

- Positive (1915 words) and Negative (2291 words)
- Strong vs Weak, Active vs Passive, Overstated versus Understated
- Pleasure, Pain, Virtue, Vice, Motivation, Cognitive Orientation, etc

Free for Research Use



# Using Lexicons in Sentiment Classification

**Add a feature** that gets a count whenever a word from the lexicon occurs

- E.g., a feature called "**this word occurs in the positive lexicon**" or "**this word occurs in the negative lexicon**"

Now all positive words (*good, great, beautiful, wonderful*) or negative words count for that feature.

Using 1-2 features isn't as good as using all the words.

- But when training data is sparse or not representative of the test set, dense lexicon features can help

# Summary: Naive Bayes is Not So Naive

Very Fast, low storage requirements

Work well with very small amounts of training data

Robust to Irrelevant Features

Irrelevant Features cancel each other without affecting results

Very good in domains with many equally important features

Decision Trees suffer from *fragmentation* in such cases – especially if little data

Optimal if the independence assumptions hold: If assumed independence is correct, then it is the Bayes Optimal Classifier for problem

A good dependable baseline for text classification

- **But we will see other classifiers that give better accuracy**

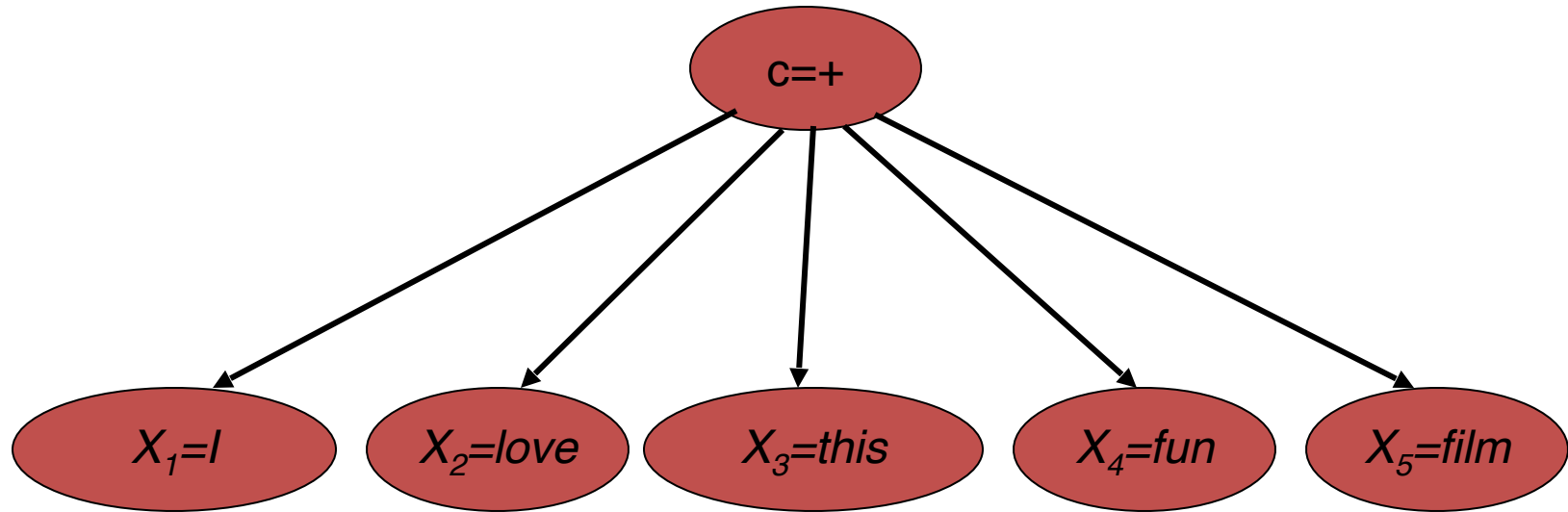
# Text Classification and Naive Bayes

## More on Sentiment Classification

# Text Classification and Naïve Bayes

## Naïve Bayes: Relationship to Language Modeling

# Generative Model for Multinomial Naïve Bayes



# Naïve Bayes and Language Modeling

Naïve Bayes classifiers can use any sort of feature

- URL, email address, dictionaries, network features

But if, as in the previous slides

- We use **only** word features
- we use **all** of the words in the text (not a subset)

Then

- Naïve Bayes has an important similarity to language modeling.

# Each class = a unigram language model

Assigning each word:  $P(\text{word} \mid c)$

Assigning each sentence:  $P(s \mid c) = \prod P(\text{word} \mid c)$

| Class <i>pos</i> |      |          |             |             |            |             |
|------------------|------|----------|-------------|-------------|------------|-------------|
| 0.1              | I    |          |             |             |            |             |
| 0.1              | love | <u>I</u> | <u>love</u> | <u>this</u> | <u>fun</u> | <u>film</u> |
| 0.01             | this | 0.1      | 0.1         | .05         | 0.01       | 0.1         |
| 0.05             | fun  |          |             |             |            |             |
| 0.1              | film |          |             |             |            |             |

$$P(s \mid \text{pos}) = 0.00000005$$

...

# Naïve Bayes as a Language Model

Which class assigns the higher probability to s?

| Model pos |      |
|-----------|------|
| 0.1       | I    |
| 0.1       | love |
| 0.01      | this |
| 0.05      | fun  |
| 0.1       | film |

| Model neg |      |
|-----------|------|
| 0.2       | I    |
| 0.001     | love |
| 0.01      | this |
| 0.005     | fun  |
| 0.1       | film |

| I     | love  | this  | fun   | film  |
|-------|-------|-------|-------|-------|
| <hr/> | <hr/> | <hr/> | <hr/> | <hr/> |
| 0.1   | 0.1   | 0.01  | 0.05  | 0.1   |
| 0.2   | 0.001 | 0.01  | 0.005 | 0.1   |

$$P(s|\text{pos}) > P(s|\text{neg})$$



# Text Classification and Naïve Bayes

## Naïve Bayes: Relationship to Language Modeling

# Text Classification and Naïve Bayes

Precision, Recall, and F  
measure

# Evaluation

Let's consider just binary text classification tasks

Imagine you're the CEO of Delicious Pie Company

You want to know what people are saying about your pies

So you build a "Delicious Pie" tweet detector

- Positive class: tweets about Delicious Pie Co
- Negative class: all other tweets

# The 2-by-2 confusion matrix

|                             |                 | <i>gold standard labels</i>        |                       |                                               |
|-----------------------------|-----------------|------------------------------------|-----------------------|-----------------------------------------------|
|                             |                 | gold positive                      | gold negative         |                                               |
| <i>system output labels</i> | system positive | <b>true positive</b>               | <b>false positive</b> | <b>precision</b> = $\frac{tp}{tp+fp}$         |
|                             | system negative | <b>false negative</b>              | <b>true negative</b>  |                                               |
|                             |                 | <b>recall</b> = $\frac{tp}{tp+fn}$ |                       | <b>accuracy</b> = $\frac{tp+tn}{tp+fp+tn+fn}$ |

# Evaluation: Accuracy

Why don't we use **accuracy** as our metric?

Imagine we saw 1 million tweets

- 100 of them talked about Delicious Pie Co.
- 999,900 talked about something else

We could build a dumb classifier that just labels every tweet "not about pie"

- It would get 99.99% accuracy!!! Wow!!!!
- But useless! Doesn't return the comments we are looking for!
- That's why we use **precision** and **recall** instead

# Evaluation: Precision

% of items the system detected (i.e., items the system labeled as positive) that are in fact positive (according to the human gold labels)

$$\textit{Precision} = \frac{\textit{TruePositives}}{\textit{TruePositives} + \textit{FalsePositives}}$$

# Evaluation: Recall

% of items actually present in the input that were correctly identified by the system.

$$\text{Recall} = \frac{\text{TruePositives}}{\text{TruePositives} + \text{FalseNegatives}}$$

# Why Precision and recall

## Our dumb pie-classifier

- Just label nothing as "about pie"

Accuracy=99.99%

but

Recall = 0

- (it doesn't get any of the 100 Pie tweets)

Precision and recall, unlike accuracy, emphasize true positives:

- finding the things that we are supposed to be looking for.



## A combined measure: F

F measure: a single number that combines P and R:

$$F_{\beta} = \frac{(\beta^2 + 1)PR}{\beta^2 P + R}$$

We almost always use balanced  $F_1$  (i.e.,  $\beta = 1$ )

$$F_1 = \frac{2PR}{P + R}$$

# Text Classification and Naive Bayes

Evaluation with more than  
two classes

# Confusion Matrix for 3-class classification

|                      |        | <i>gold labels</i>                            |                                                   |                                                    |                                                       |
|----------------------|--------|-----------------------------------------------|---------------------------------------------------|----------------------------------------------------|-------------------------------------------------------|
|                      |        | urgent                                        | normal                                            | spam                                               |                                                       |
| <i>system output</i> | urgent | 8                                             | 10                                                | 1                                                  | <b>precision<sub>u</sub></b> = $\frac{8}{8+10+1}$     |
|                      | normal | 5                                             | 60                                                | 50                                                 | <b>precision<sub>n</sub></b> = $\frac{60}{5+60+50}$   |
|                      | spam   | 3                                             | 30                                                | 200                                                | <b>precision<sub>s</sub></b> = $\frac{200}{3+30+200}$ |
|                      |        | <b>recall<sub>u</sub></b> = $\frac{8}{8+5+3}$ | <b>recall<sub>n</sub></b> = $\frac{60}{10+60+30}$ | <b>recall<sub>s</sub></b> = $\frac{200}{1+50+200}$ |                                                       |

# How to combine P/R from 3 classes to get one metric

## Macroaveraging:

- compute the performance for each class, and then average over classes

## Microaveraging:

- collect decisions for all classes into one confusion matrix
- compute precision and recall from that table.

# Macroaveraging and Microaveraging

**Class 1: Urgent**

|                  | true<br>urgent | true<br>not |
|------------------|----------------|-------------|
| system<br>urgent | 8              | 11          |
| system<br>not    | 8              | 340         |

$$\text{precision} = \frac{8}{8+11} = .42$$

**Class 2: Normal**

|                  | true<br>normal | true<br>not |
|------------------|----------------|-------------|
| system<br>normal | 60             | 55          |
| system<br>not    | 40             | 212         |

$$\text{precision} = \frac{60}{60+55} = .52$$

**Class 3: Spam**

|                | true<br>spam | true<br>not |
|----------------|--------------|-------------|
| system<br>spam | 200          | 33          |
| system<br>not  | 51           | 83          |

$$\text{precision} = \frac{200}{200+33} = .86$$

**Pooled**

|               | true<br>yes | true<br>no |
|---------------|-------------|------------|
| system<br>yes | 268         | 99         |
| system<br>no  | 99          | 635        |

$$\text{microaverage precision} = \frac{268}{268+99} = .73$$

$$\text{macroaverage precision} = \frac{.42 + .52 + .86}{3} = .60$$

# Text Classification and Naive Bayes

## Test Sets and Cross-Validation

# Development Test Sets ("Devsets") and Cross-validation

Training set

Development Test Set

Test Set

Train on training set, tune on devset, report on testset

- This avoids overfitting ('tuning to the test set')
- More conservative estimate of performance
- But paradox: want as much data as possible for training, and as much for dev; how to split?

# Cross-validation: multiple splits

Pool results over splits, Compute pooled dev performance





Text  
Classification  
and Naive  
Bayes

Avoiding Harms in  
Classification

# Harms in sentiment classifiers

Kiritchenko and Mohammad (2018) found that most sentiment classifiers assign lower sentiment and more negative emotion to sentences with African American names in them.

This perpetuates negative stereotypes that associate African Americans with negative emotions

<https://aclanthology.org/S18-2005.pdf>

# Harms in toxicity classification

Toxicity detection is the task of detecting hate speech, abuse, harassment, or other kinds of toxic language

But some toxicity classifiers incorrectly flag as being toxic sentences that are non-toxic but simply mention identities like blind people, women, or gay people.

This could lead to censorship of discussion about these groups.

# What causes these harms?

Can be caused by:

- Problems in the training data; machine learning systems are known to amplify the biases in their training data.
- Problems in the human labels
- Problems in the resources used (like lexicons)
- Problems in model architecture (like what the model is trained to optimize)

Mitigation of these harms is an open research area

Meanwhile: **model cards**

# Model Cards

(Mitchell et al., 2019)

For each algorithm you release, document:

- training algorithms and parameters
- training data sources, motivation, and preprocessing
- evaluation data sources, motivation, and preprocessing
- intended use and users
- model performance across different demographic or other groups and environmental situations

<https://huggingface.co/blog/model-cards>

# Evaluating Language Model Bias

(Mitchell et al., 2019)

For each algorithm you release, document:

- training algorithms and parameters
- training data sources, motivation, and preprocessing
- evaluation data sources, motivation, and preprocessing
- intended use and users
- model performance across different demographic or other groups and environmental situations

<https://huggingface.co/blog/evaluating-llm-bias>

Text  
Classification  
and Naive  
Bayes

Avoiding Harms in  
Classification

# Logistic Regression

Background: Generative and  
Discriminative Classifiers



# Logistic Regression

Important analytic tool in natural and social sciences

Baseline supervised machine learning tool for classification

Is also the foundation of neural networks

# Generative and Discriminative Classifiers

Naive Bayes is a **generative** classifier

by contrast:

Logistic regression is a **discriminative** classifier

# Generative and Discriminative Classifiers

Suppose we're distinguishing cat from dog images



imagenet



imagenet

# Generative Classifier:

- Build a model of what's in a cat image
  - Knows about whiskers, ears, eyes
  - Assigns a probability to any image:
    - how cat-y is this image?



Also build a model for dog images

Now given a new image:

**Run both models and see which one fits better**

# Discriminative Classifier

Just try to distinguish dogs from cats



Oh look, dogs have collars!  
Let's ignore everything else

Finding the correct class  $c$  from a document  $d$  in  
Generative vs Discriminative Classifiers

Naive Bayes

$$\hat{c} = \operatorname{argmax}_{c \in \mathcal{C}} \overbrace{P(d|c)}^{\text{likelihood}} \overbrace{P(c)}^{\text{prior}}$$

Logistic Regression

$$\hat{c} = \operatorname{argmax}_{c \in \mathcal{C}} \overbrace{P(c|d)}^{\text{posterior}}$$

# Components of a probabilistic machine learning classifier

Given  $m$  input/output pairs  $(x^{(i)}, y^{(i)})$ :

1. A **feature representation** of the input. For each input observation  $x^{(i)}$ , a vector of features  $[x_1, x_2, \dots, x_n]$ . Feature  $j$  for input  $x^{(i)}$  is  $x_j$ , more completely  $x_j^{(i)}$ , or sometimes  $f_j(x)$ .
2. A **classification function** that computes  $\hat{y}$ , the estimated class, via  $p(y|x)$ , like the **sigmoid** or **softmax** functions.
3. An objective function for learning, like **cross-entropy loss**.
4. An algorithm for optimizing the objective function: **stochastic gradient descent**.

# The two phases of logistic regression

**Training:** we learn weights  $w$  and  $b$  using **stochastic gradient descent** and **cross-entropy loss**.

**Test:** Given a test example  $x$  we compute  $p(y|x)$  using learned weights  $w$  and  $b$ , and return whichever label ( $y = 1$  or  $y = 0$ ) is higher probability



# Logistic Regression

Background: Generative and  
Discriminative Classifiers

# Classification in Logistic Regression

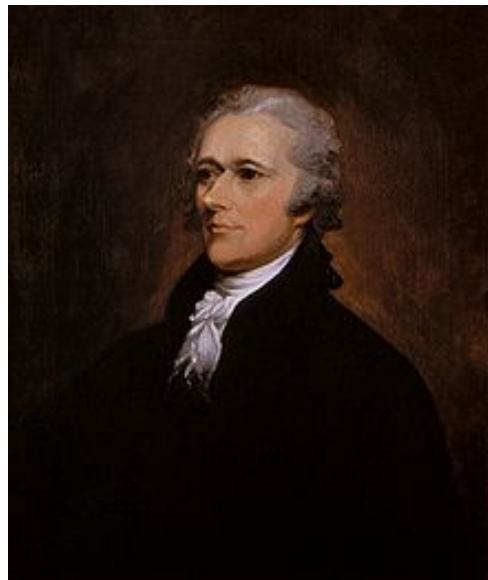
## Logistic Regression

# Classification Reminder

Positive/negative sentiment

Spam/not spam

Authorship attribution  
(Hamilton or Madison?)



Alexander Hamilton

# Text Classification: definition

*Input:*

- a document  $x$
- a fixed set of classes  $C = \{c_1, c_2, \dots, c_J\}$

*Output:* a predicted class  $\hat{y} \in C$

# Binary Classification in Logistic Regression

Given a series of input/output pairs:

- $(x^{(i)}, y^{(i)})$

For each observation  $x^{(i)}$

- We represent  $x^{(i)}$  by a **feature vector**  $[x_1, x_2, \dots, x_n]$
- We compute an output: a predicted class  $\hat{y}^{(i)} \in \{0, 1\}$

# Features in logistic regression

- For feature  $x_i$ , weight  $w_i$  tells is how important is  $x_i$ 
  - $x_i$  = "review contains 'awesome'":  $w_i = +10$
  - $x_j$  = "review contains 'abysmal'":  $w_j = -10$
  - $x_k$  = "review contains 'mediocre'":  $w_k = -2$

# Logistic Regression for one observation $x$

Input observation: vector  $x = [x_1, x_2, \dots, x_n]$

Weights: one per feature:  $W = [w_1, w_2, \dots, w_n]$

- Sometimes we call the weights  $\theta = [\theta_1, \theta_2, \dots, \theta_n]$

Output: a predicted class  $\hat{y} \in \{0, 1\}$

(multinomial logistic regression:  $\hat{y} \in \{0, 1, 2, 3, 4\}$ )

# How to do classification

For each feature  $x_i$ , weight  $w_i$  tells us importance of  $x_i$

- (Plus we'll have a bias  $b$ )

We'll sum up all the weighted features and the bias

$$z = \left( \sum_{i=1}^n w_i x_i \right) + b$$

$$z = a \cdot b + b$$

If this sum is high, we say  $y=1$ ; if low, then  $y=0$



But we want a probabilistic classifier

We need to formalize “sum is high”.

We’d like a principled classifier that gives us a probability, just like Naive Bayes did

We want a model that can tell us:

$$p(y=1 \mid x; \theta)$$

$$p(y=0 \mid x; \theta)$$

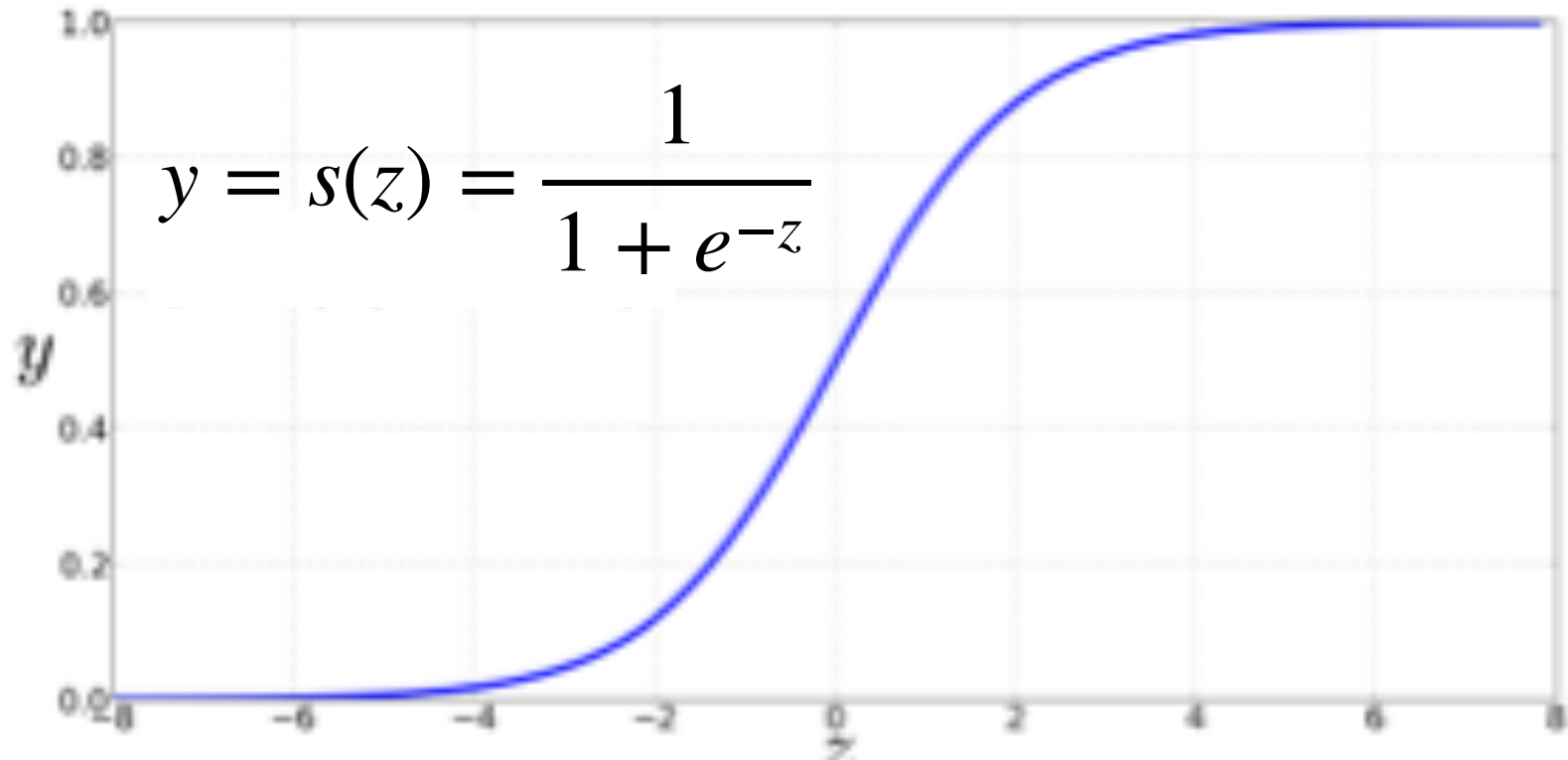
The problem:  $z$  isn't a probability, it's just a number!

$$z = a \cdot b + b$$

Solution: use a function of  $z$  that goes from 0 to 1

$$y = s(z) = \frac{1}{1 + e^{-z}} = \frac{1}{1 + \exp(-z)}$$

# The very useful sigmoid or logistic function



# Idea of logistic regression

We'll compute  $w \cdot x + b$

And then we'll pass it through the sigmoid function:

$$\sigma(w \cdot x + b)$$

And we'll just treat it as a probability

Making probabilities with sigmoids

$$\begin{aligned} P(y = 1) &= \sigma(w \cdot x + b) \\ &= \frac{1}{1 + \exp(-(w \cdot x + b))} \end{aligned}$$

$$\begin{aligned} P(y = 0) &= 1 - \sigma(w \cdot x + b) \\ &= 1 - \frac{1}{1 + \exp(-(w \cdot x + b))} \\ &= \frac{\exp(-(w \cdot x + b))}{1 + \exp(-(w \cdot x + b))} \end{aligned}$$

By the way:

$$\begin{aligned} P(y=0) &= 1 - \sigma(w \cdot x + b) &= \sigma(-(w \cdot x + b)) \\ &= 1 - \frac{1}{1 + \exp(-(w \cdot x + b))} \\ &= \frac{\exp(-(w \cdot x + b))}{1 + \exp(-(w \cdot x + b))} \end{aligned}$$

Because

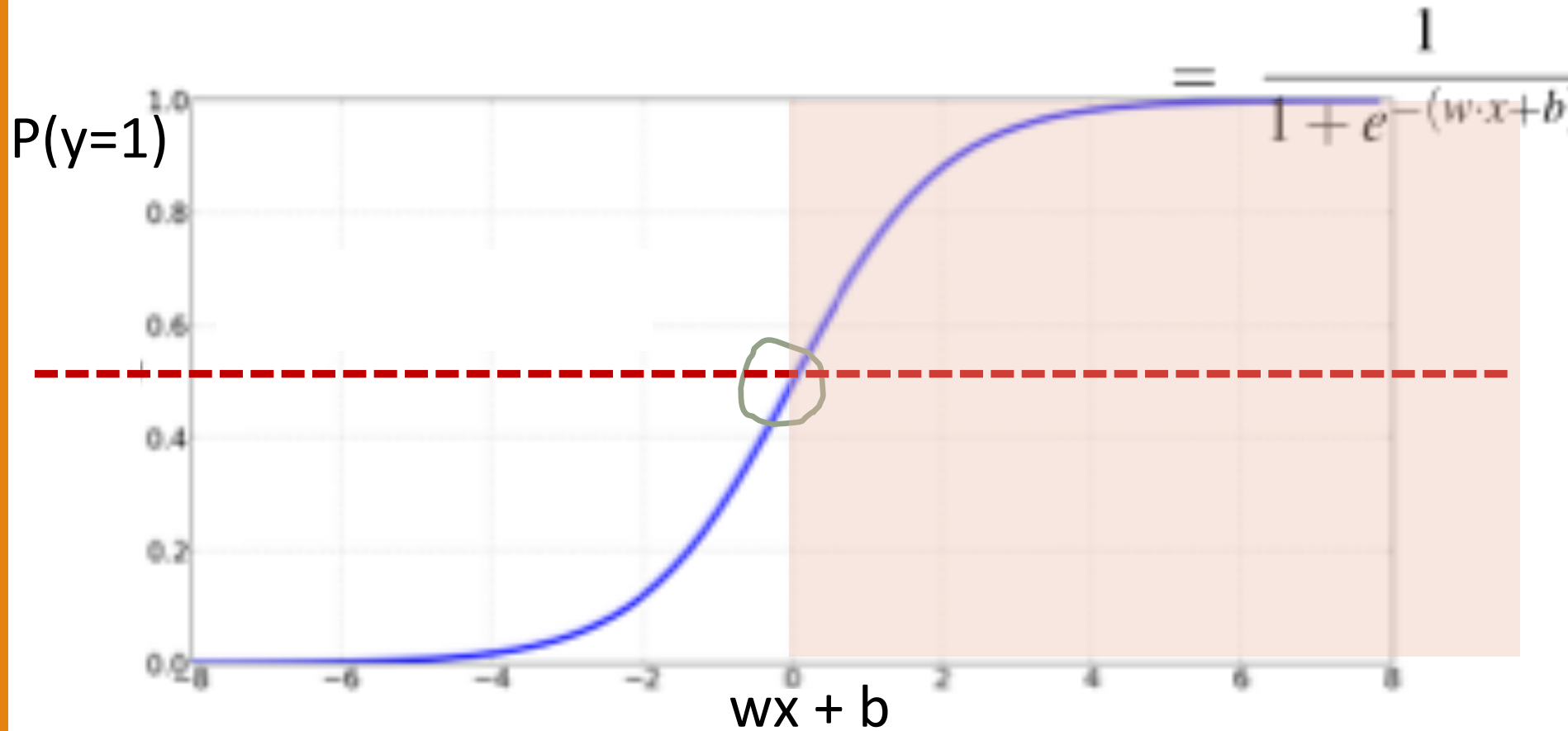
$$1 - \sigma(x) = \sigma(-x)$$

# Turning a probability into a classifier

$$\hat{y} = \begin{cases} 1 & \text{if } P(y = 1|x) > 0.5 \\ 0 & \text{otherwise} \end{cases}$$

0.5 here is called the **decision boundary**

The probabilistic classifier  $P(y = 1) = \sigma(w \cdot x + b)$





## Turning a probability into a classifier

$$\hat{y} = \begin{cases} 1 & \text{if } P(y = 1|x) > 0.5 \\ 0 & \text{otherwise} \end{cases} \quad \begin{array}{l} \text{if } w \cdot x + b > 0 \\ \text{if } w \cdot x + b \leq 0 \end{array}$$

# Classification in Logistic Regression

## Logistic Regression

# Logistic Regression

Logistic Regression: a text example  
on sentiment classification

# Sentiment example: does $y=1$ or $y=0$ ?

It's hokey . There are virtually no surprises , and the writing is second-rate .  
So why was it so enjoyable ? For one thing , the cast is  
great . Another nice touch is the music . I was overcome with the urge to get off  
the couch and start dancing . It sucked me in , and it'll do the same to you .

It's hokey. There are virtually no surprises, and the writing is second-rate. So why was it so enjoyable? For one thing, the cast is great. Another nice touch is the music I was overcome with the urge to get off the couch and start dancing. It sucked me in, and it'll do the same to you.

Diagram illustrating feature extraction from the text:

- $x_1 = 3$  (count of positive lexicon)
- $x_2 = 2$  (count of negative lexicon)
- $x_3 = 1$  (presence of "no")
- $x_4 = 3$  (count of 1st and 2nd pronouns)
- $x_5 = 0$  (presence of "!")
- $x_6 = 4.19$  (log(word count of doc))

| Var   | Definition                                                                              | Value in Fig. 5.2 |
|-------|-----------------------------------------------------------------------------------------|-------------------|
| $x_1$ | count(positive lexicon) $\in$ doc)                                                      | 3                 |
| $x_2$ | count(negative lexicon) $\in$ doc)                                                      | 2                 |
| $x_3$ | $\begin{cases} 1 & \text{if "no" } \in \text{ doc} \\ 0 & \text{otherwise} \end{cases}$ | 1                 |
| $x_4$ | count(1st and 2nd pronouns $\in$ doc)                                                   | 3                 |
| $x_5$ | $\begin{cases} 1 & \text{if "!" } \in \text{ doc} \\ 0 & \text{otherwise} \end{cases}$  | 0                 |
| $x_6$ | log(word count of doc)                                                                  | $\ln(66) = 4.19$  |

# Classifying sentiment for input x

| Var   | Definition                                                                              | Value            |
|-------|-----------------------------------------------------------------------------------------|------------------|
| $x_1$ | count(positive lexicon $\in$ doc)                                                       | 3                |
| $x_2$ | count(negative lexicon $\in$ doc)                                                       | 2                |
| $x_3$ | $\begin{cases} 1 & \text{if "no" } \in \text{ doc} \\ 0 & \text{otherwise} \end{cases}$ | 1                |
| $x_4$ | count(1st and 2nd pronouns $\in$ doc)                                                   | 3                |
| $x_5$ | $\begin{cases} 1 & \text{if "!" } \in \text{ doc} \\ 0 & \text{otherwise} \end{cases}$  | 0                |
| $x_6$ | log(word count of doc)                                                                  | $\ln(66) = 4.19$ |

Suppose  $w = [2.5, -5.0, -1.2, 0.5, 2.0, 0.7]$

$$b = 0.1$$

# Classifying sentiment for input x

$$\begin{aligned} p(+|x) &= P(Y=1|x) = s(w \cdot x + b) \\ &= s([2.5, -5.0, -1.2, 0.5, 2.0, 0.7] \cdot [3, 2, 1, 3, 0, 4.19] + 0.1) \\ &= s(.833) \\ &= 0.70 \end{aligned}$$

$$\begin{aligned} p(-|x) &= P(Y=0|x) = 1 - s(w \cdot x + b) \\ &= 0.30 \end{aligned}$$

We can build features for logistic regression for any classification task: period disambiguation

End of sentence

This ends in a period.

The house at 465 Main St. is new.

Not end

$x_1 = \begin{cases} 1 & \text{if "Case}(w_i) = \text{Lower"} \\ 0 & \text{otherwise} \end{cases}$

$x_2 = \begin{cases} 1 & \text{if " } w_i \in \text{AcronymDict"} \\ 0 & \text{otherwise} \end{cases}$

$x_3 = \begin{cases} 1 & \text{if " } w_i = \text{St. \& Case}(w_{i-1}) = \text{Cap"} \\ 0 & \text{otherwise} \end{cases}$



## Classification in (binary) logistic regression: summary

Given:

- a set of classes: (+ sentiment, - sentiment)
- a vector  $\mathbf{x}$  of features  $[x_1, x_2, \dots, x_n]$ 
  - $x_1 = \text{count}(\text{"awesome"})$
  - $x_2 = \log(\text{number of words in review})$
- A vector  $\mathbf{w}$  of weights  $[w_1, w_2, \dots, w_n]$ 
  - $w_i$  for each feature  $f_i$

$$\begin{aligned} P(y = 1) &= \sigma(w \cdot x + b) \\ &= \frac{1}{1 + e^{-(w \cdot x + b)}} \end{aligned}$$

# Logistic Regression

Logistic Regression: a text example  
on sentiment classification

Learning: Cross-Entropy Loss

Logistic  
Regression

# Wait, where did the $W$ 's come from?

Supervised classification:

- We know the correct label  $y$  (either 0 or 1) for each  $x$ .
- But what the system produces is an estimate,  $\hat{y}$

We want to set  $w$  and  $b$  to minimize the **distance** between our estimate  $\hat{y}^{(i)}$  and the true  $y^{(i)}$ .

- We need a distance estimator: a **loss function** or a **cost function**
- We need an optimization algorithm to update  $w$  and  $b$  to minimize the loss.

# Learning components

A loss function:

- **cross-entropy loss**

An optimization algorithm:

- **stochastic gradient descent**

# The distance between $\hat{y}$ and $y$

We want to know how far the classifier output is:

$$\hat{y} = \sigma(w \cdot x + b)$$

from the true output:

$$y \quad [= \text{either } 0 \text{ or } 1]$$

We'll call this difference:

$$L(\hat{y}, y) = \text{how much } \hat{y} \text{ differs from the true } y$$

# Intuition of negative log likelihood loss = cross-entropy loss

A case of conditional maximum likelihood estimation

We choose the parameters  $w, b$  that maximize the log probability of the true  $y$  labels in the training data given the observations  $x$

# What is Entropy?

"Entropy" is defined as **the level of uncertainty in an outcome.**

Practically speaking, given a set of data or a function, what is the likelihood of an expected outcome? How certain are you that the outcome will be what is expected?

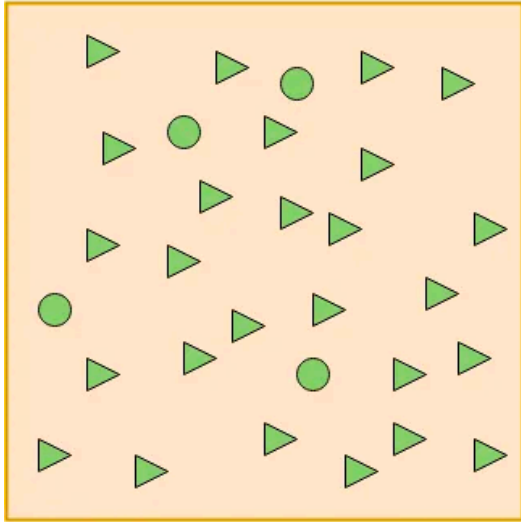
The Shannon formula for calculating entropy on discrete values is:

$$-\sum_x p(x) \log p(x)$$



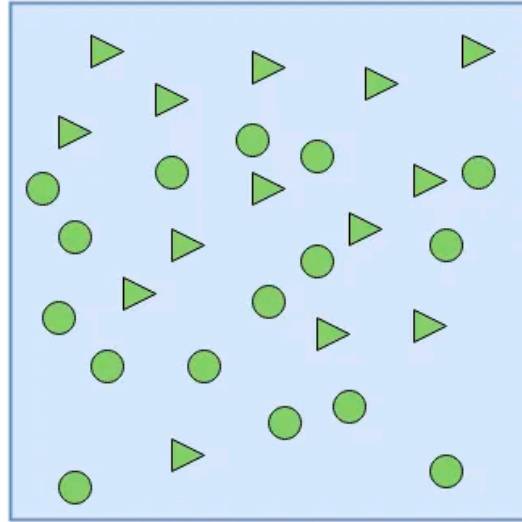
# Entropy Example

Container 1



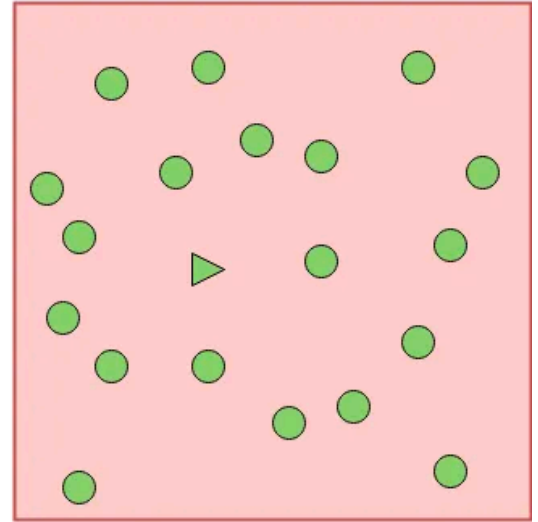
#▶ = 26, #● = 4

Container 2



#▶ = 14, #● = 16

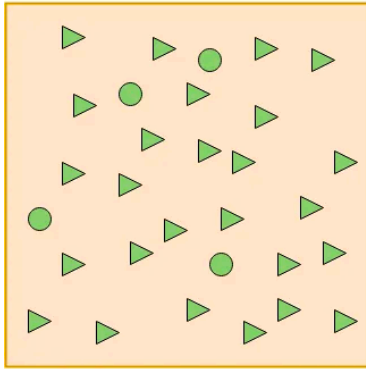
Container 3



#▶ = 1 #● = 29

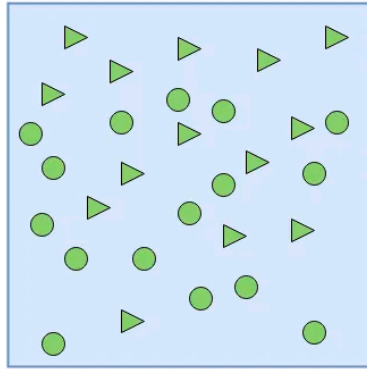
# Entropy Example

Container 1



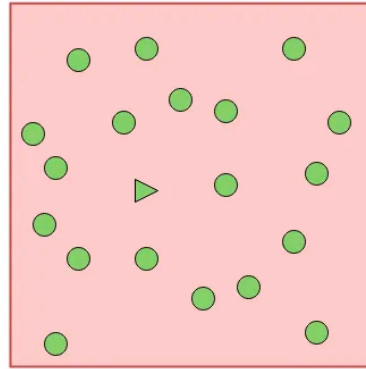
#▶ = 26, #● = 4

Container 2



#▶ = 14, #● = 16

Container 3

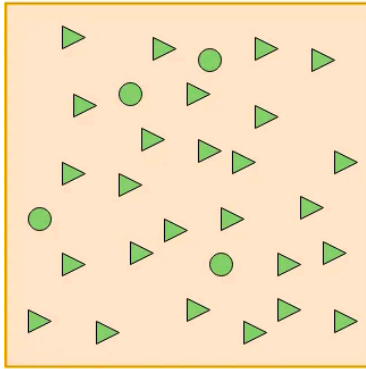


#▶ = 1 #● = 29

Container 1 Entropy: 
$$-\left[ \frac{26}{30} \log_2 \left( \frac{26}{30} \right) + \frac{4}{30} \log_2 \left( \frac{4}{30} \right) \right] = 0.5665$$

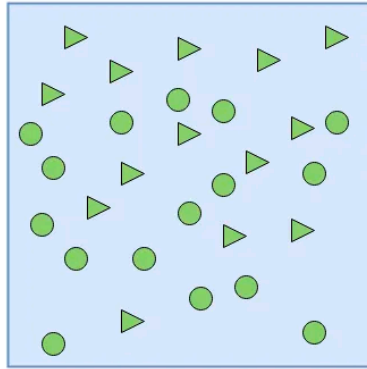
# Entropy Example

Container 1



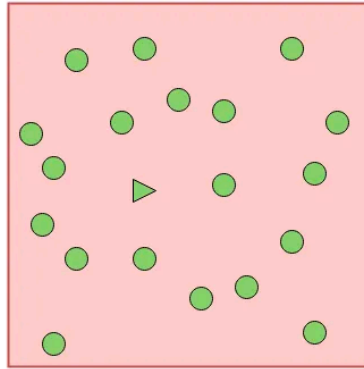
#▶ = 26, #● = 4

Container 2



#▶ = 14, #● = 16

Container 3

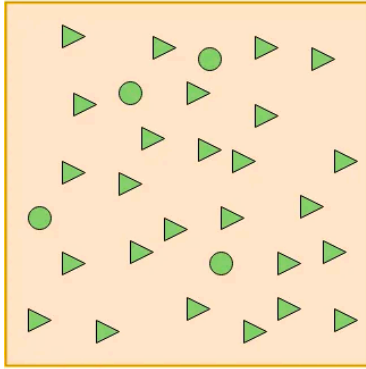


#▶ = 1 #● = 29

Container 2 Entropy: 
$$-\left[ \frac{14}{30} \log_2 \left( \frac{14}{30} \right) + \frac{16}{30} \log_2 \left( \frac{16}{30} \right) \right] = 0.9968$$

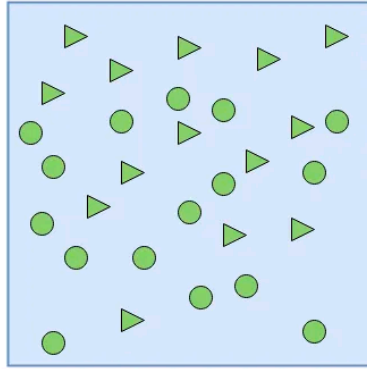
# Entropy Example

Container 1



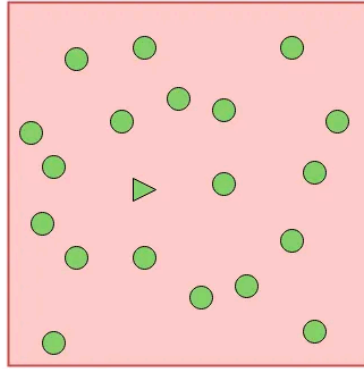
#▶ = 26, #● = 4

Container 2



#▶ = 14, #● = 16

Container 3



#▶ = 1 #● = 29

Container 3 Entropy: 
$$-\left[ \frac{1}{30} \log_2 \left( \frac{1}{30} \right) + \frac{29}{30} \log_2 \left( \frac{29}{30} \right) \right] = 0.2108$$

# Deriving cross-entropy loss for a single observation $x$

**Goal:** maximize probability of the correct label  $p(y|x)$

Since there are only 2 discrete outcomes (0 or 1) we can express the probability  $p(y|x)$  from our classifier (the thing we want to maximize) as

$$p(y|x) = \hat{y}^y (1 - \hat{y})^{(1-y)}$$

noting:

if  $y=1$ , this simplifies to  $\hat{y}$

if  $y=0$ , this simplifies to  $1 - \hat{y}$

# Deriving cross-entropy loss for a single observation $x$

**Goal:** maximize probability of the correct label  $p(y|x)$

Maximize:  $p(y | x) = \hat{y}^y(1 - \hat{y})^{(1-y)}$

Now take the log of both sides (mathematically handy)

Maximize:  $\log(p(y | x)) = \log [\hat{y}^y(1 - \hat{y})^{(1-y)}] =$   
 $y \log \hat{y} + (1 - y) \log(1 - \hat{y})$

Whatever values maximize  $\log p(y|x)$  will also maximize  $p(y|x)$

# Deriving cross-entropy loss for a single observation $x$

**Goal:** maximize probability of the correct label  $p(y|x)$

**Maximize:**  $\log(p(y|x)) = \log [\hat{y}^y(1 - \hat{y})^{(1-y)}] =$   
 $y \log \hat{y} + (1 - y) \log(1 - \hat{y})$

Now flip sign to turn this into a loss: something to minimize

**Cross-entropy loss** (because is formula for cross-entropy( $y, \hat{y}$ ))

**Minimize:**  $L_{CE}(\hat{y}, y) = -\log(p(y|x)) = [-y \log \hat{y} + (1 - y) \log(1 - \hat{y})]$

Or, plugging in definition of  $\hat{y}$ :

$$L_{CE}(\hat{y}, y) = -y \log \sigma(w \cdot x + b) + (1 - y) \log(1 - \sigma(w \cdot x + b))$$

# Let's see if this works for our sentiment example

We want loss to be:

- smaller if the model estimate is close to correct
- bigger if model is confused

Let's first suppose the true label of this is  $y=1$  (positive)

It's hokey . There are virtually no surprises , and the writing is second-rate . So why was it so enjoyable ? For one thing , the cast is great . Another nice touch is the music . I was overcome with the urge to get off the couch and start dancing . It sucked me in , and it'll do the same to you .



# Let's see if this works for our sentiment example

True value is  $y=1$ . How well is our model doing?

$$\begin{aligned} p(+|x) &= P(Y=1|x) = s(w \cdot x + b) \\ &= s([2.5, -5.0, -1.2, 0.5, 2.0, 0.7] \cdot [3, 2, 1, 3, 0, 4.19] + 0.1) \\ &= s(.833) \\ &= 0.70 \end{aligned} \tag{5.6}$$

Pretty well! What's the loss?

$$\begin{aligned} L_{\text{CE}}(\hat{y}, y) &= -[y \log \sigma(w \cdot x + b) + (1 - y) \log (1 - \sigma(w \cdot x + b))] \\ &= -[\log \sigma(w \cdot x + b)] \\ &= -\log(.70) \\ &= .36 \end{aligned}$$

# Let's see if this works for our sentiment example

Suppose true value instead was  $y=0$ .

$$\begin{aligned} p(-|x) &= P(Y = 0|x) = 1 - s(w \cdot x + b) \\ &= 0.30 \end{aligned}$$

What's the loss?

$$\begin{aligned} L_{\text{CE}}(\hat{y}, y) &= -[y \log \sigma(w \cdot x + b) + (1 - y) \log (1 - \sigma(w \cdot x + b))] \\ &= -[\log (1 - \sigma(w \cdot x + b))] \\ &= -\log (.30) \\ &= 1.2 \end{aligned}$$

# Let's see if this works for our sentiment example

The loss when model was right (if true  $y=1$ )

$$\begin{aligned} L_{\text{CE}}(\hat{y}, y) &= -[y \log \sigma(w \cdot x + b) + (1 - y) \log (1 - \sigma(w \cdot x + b))] \\ &= -[\log \sigma(w \cdot x + b)] \\ &= -\log(.70) \\ &= .36 \end{aligned}$$

Is lower than the loss when model was wrong (if true  $y=0$ ):

$$\begin{aligned} L_{\text{CE}}(\hat{y}, y) &= -[y \log \sigma(w \cdot x + b) + (1 - y) \log (1 - \sigma(w \cdot x + b))] \\ &= -[\log (1 - \sigma(w \cdot x + b))] \\ &= -\log (.30) \\ &= 1.2 \end{aligned}$$

Sure enough, loss was bigger when model was wrong!

# Cross-Entropy Loss

## Logistic Regression

# Stochastic Gradient Descent

## Logistic Regression

# Our goal: minimize the loss

Let's make explicit that the loss function is parameterized by weights  $\Theta=(w,b)$

- And we'll represent  $\hat{y}$  as  $f(x; \theta)$  to make the dependence on  $\theta$  more obvious

We want the weights that minimize the loss, averaged over all examples:

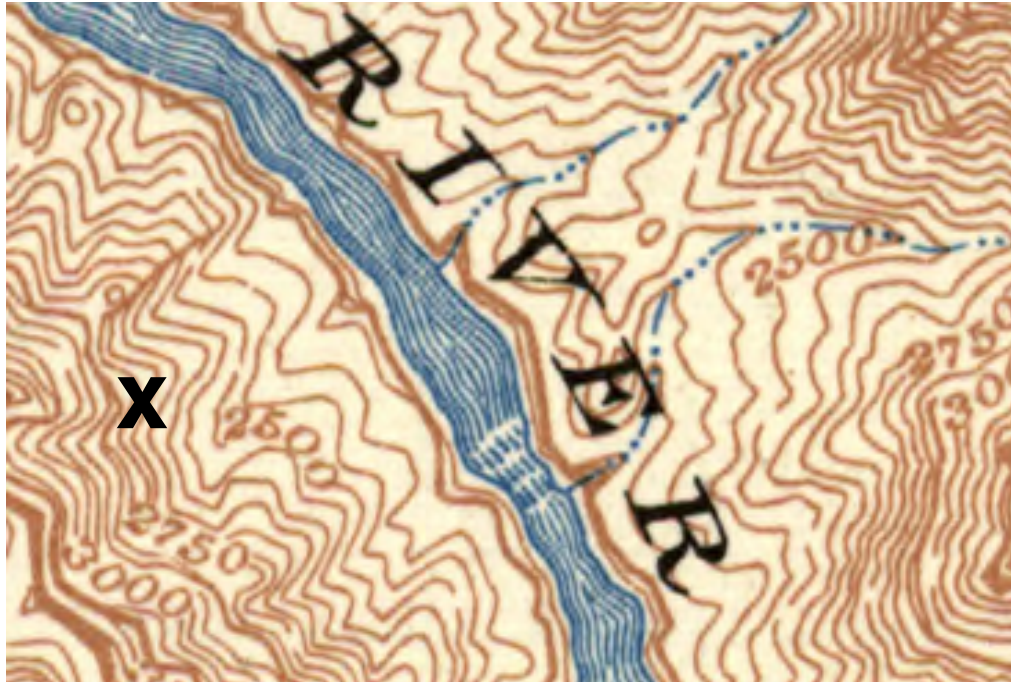
$$\hat{\theta} = \underset{\theta}{\operatorname{argmin}} \frac{1}{m} \sum_{i=1}^m L_{CE}(f(x^{(i)}; \theta), y^{(i)})$$

$$\hat{\theta} = \underset{\theta}{\operatorname{argmin}} \frac{1}{m} \sum_{i=1}^m L_{CE}(f(x^{(i)}; \theta), y^{(i)})?$$

- $\Theta$  = minimal loss function
- $m$  = number of examples
- $L_{CE}$  = Cross-entropy loss function  $(-\log(w \bullet x + b))$

# Intuition of gradient descent

How do I get to the bottom of this river canyon?



Look around me 360°  
Find the direction of  
steepest slope down  
Go that way



# Our goal: minimize the loss

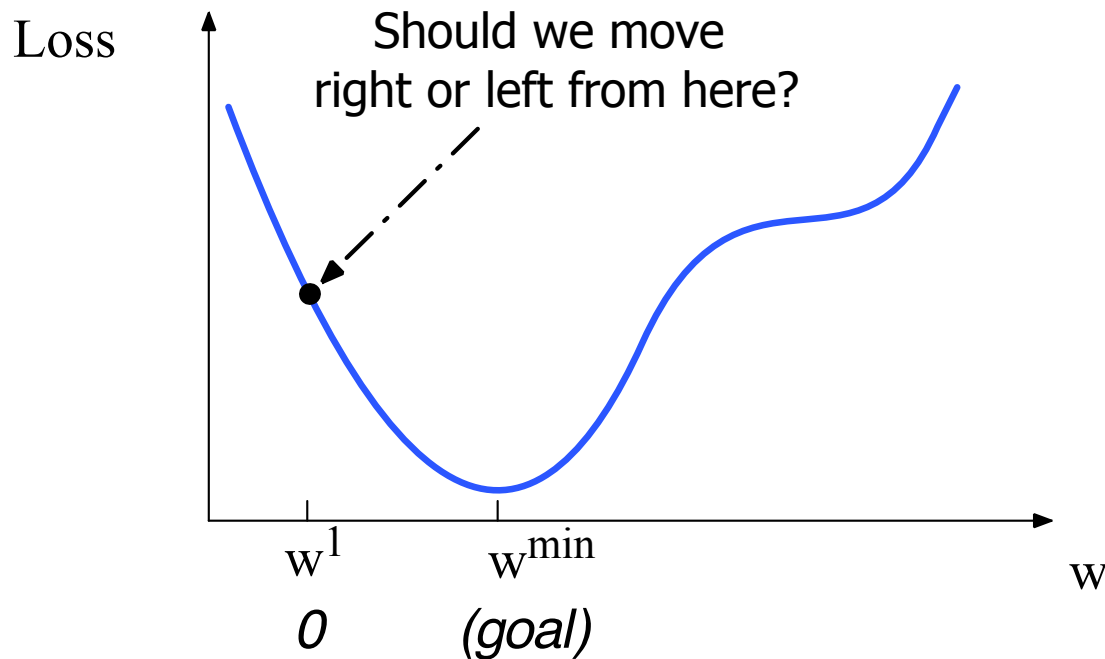
For logistic regression, loss function is **convex**

- A convex function has just one minimum
- Gradient descent starting from any point is guaranteed to find the minimum
  - (Loss for neural networks is non-convex)

# Let's first visualize for a single scalar $w$

Q: Given current  $w$ , should we make it bigger or smaller?

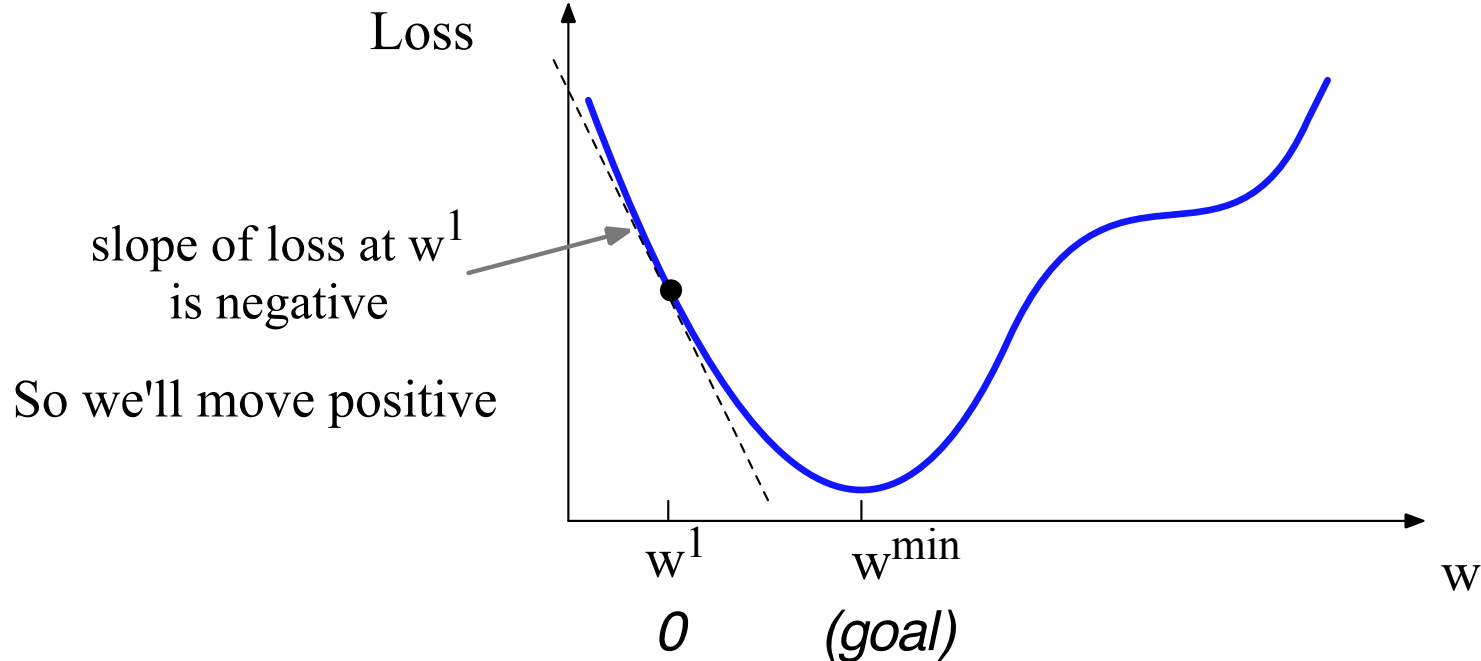
A: Move  $w$  in the reverse direction from the slope of the function



# Let's first visualize for a single scalar $w$

Q: Given current  $w$ , should we make it bigger or smaller?

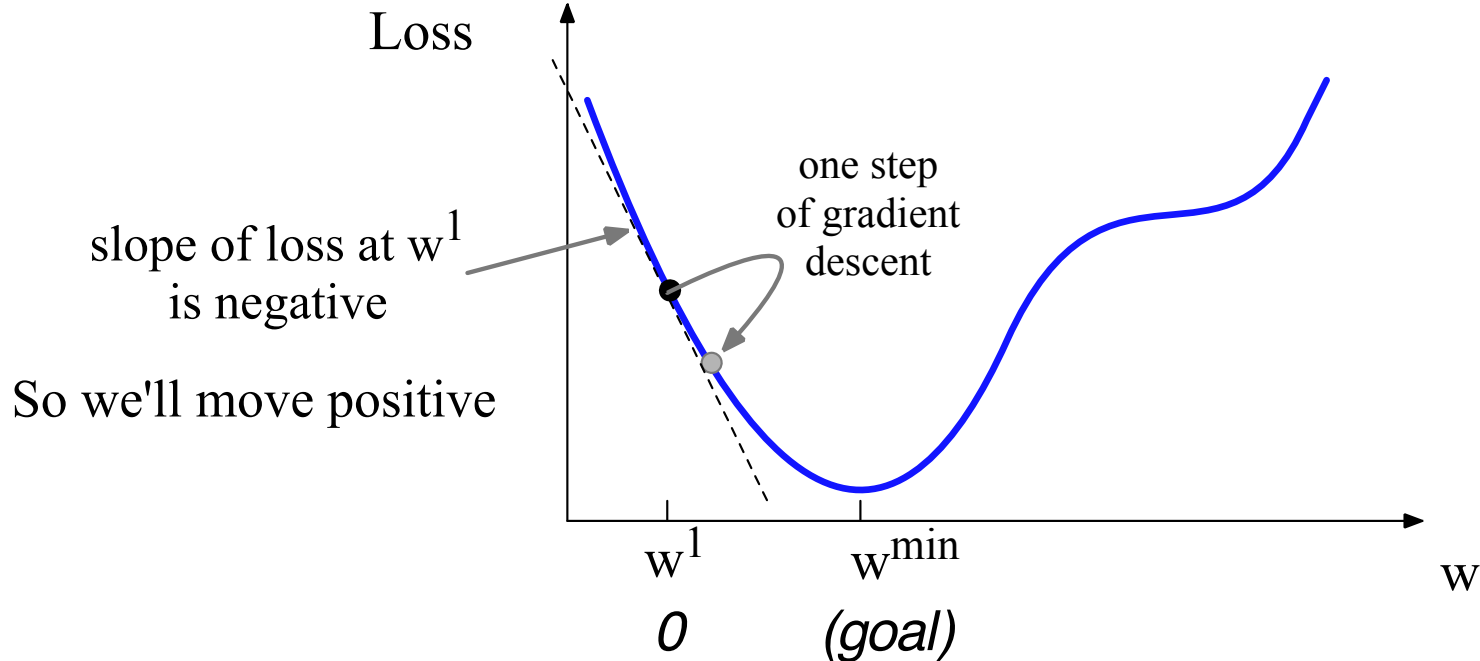
A: Move  $w$  in the reverse direction from the slope of the function



# Let's first visualize for a single scalar $w$

Q: Given current  $w$ , should we make it bigger or smaller?

A: Move  $w$  in the reverse direction from the slope of the function



# Gradients

The **gradient** of a function of many variables is a vector pointing in the direction of the greatest increase in a function.

**Gradient Descent:** Find the gradient of the loss function at the current point and move in the **opposite** direction.

# How much do we move in that direction ?

- The value of the gradient (slope in our example)  $\frac{d}{dw}L(f(x; w), y)$  weighted by a **learning rate**  $\eta$
- Higher learning rate means move  $w$  faster

$$w^{t+1} = w^t - \eta \frac{d}{dw}L(f(x; w), y)$$

# Now let's consider $N$ dimensions

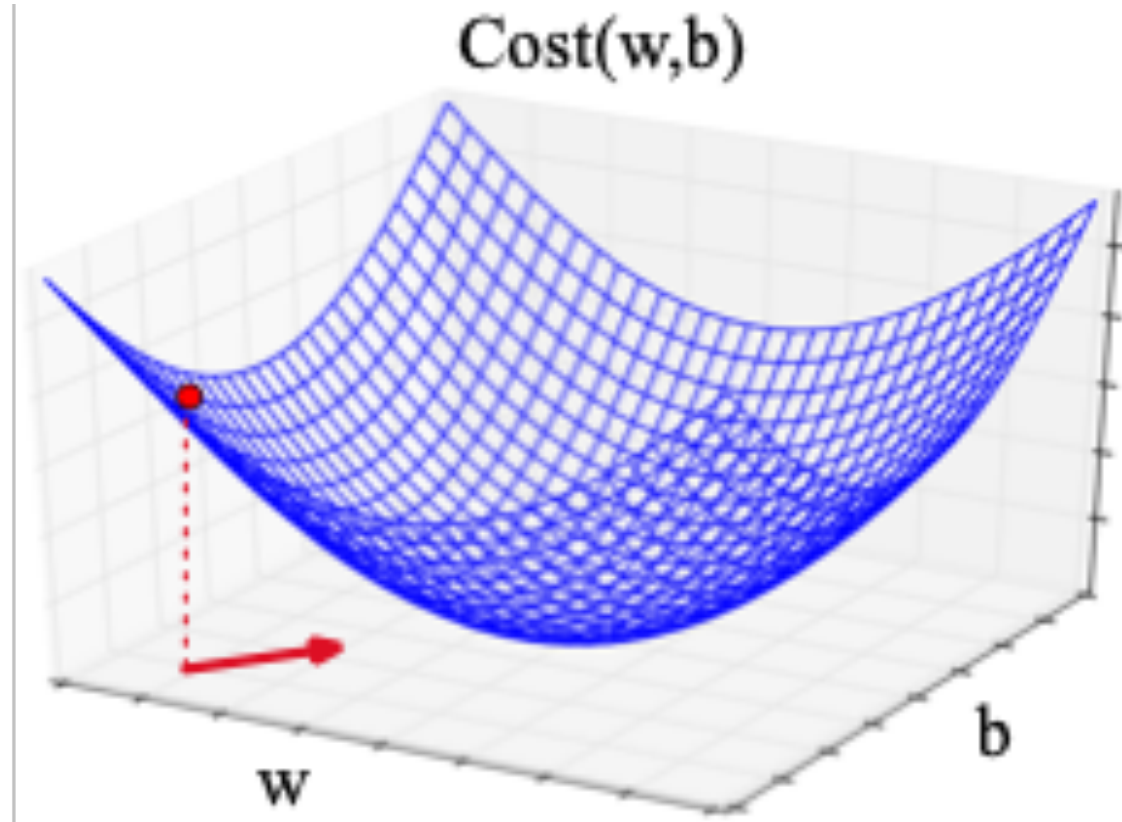
We want to know where in the  $N$ -dimensional space (of the  $N$  parameters that make up  $\theta$ ) we should move.

The gradient is just such a vector; it expresses the directional components of the sharpest slope along each of the  $N$  dimensions.

# Imagine 2 dimensions, $w$ and $b$

Visualizing the  
gradient vector at  
the red point

It has two  
dimensions shown  
in the  $x$ - $y$  plane





# Real gradients

Are much longer; lots and lots of weights

For each dimension  $w_i$ , the gradient component  $i$  tells us the slope with respect to that variable.

- “How much would a small change in  $w_i$  influence the total loss function  $L$ ?”
- We express the slope as a partial derivative  $\partial$  of the loss  $\partial w_i$

The gradient is then defined as a vector of these partials.

# The gradient

We'll represent  $\hat{y}$  as  $f(x; \theta)$  to make the dependence on  $\theta$  more obvious:

$$\nabla L(f(x; \theta), y) = \begin{bmatrix} \frac{\partial}{\partial w_1} L(f(x; \theta), y) \\ \frac{\partial}{\partial w_2} L(f(x; \theta), y) \\ \vdots \\ \frac{\partial}{\partial w_n} L(f(x; \theta), y) \\ \frac{\partial}{\partial b} L(f(x; \theta), y) \end{bmatrix}$$

The final equation for updating  $\theta$  based on the gradient is thus

$$\theta_{t+1} = \theta_t - \eta \nabla L(f(x; \theta), y)$$

# What are these partial derivatives for logistic regression?

The loss function

$$L_{\text{CE}}(\hat{y}, y) = -[y \log \sigma(w \cdot x + b) + (1 - y) \log (1 - \sigma(w \cdot x + b))]$$

The elegant derivative of this function (see textbook 5.8 for derivation)

$$\frac{\partial L_{\text{CE}}(\hat{y}, y)}{\partial w_j} = [\sigma(w \cdot x + b) - y]x_j$$

**function** STOCHASTIC GRADIENT DESCENT( $L()$ ,  $f()$ ,  $x$ ,  $y$ ) **returns**  $\theta$

# where:  $L$  is the loss function

#  $f$  is a function parameterized by  $\theta$

#  $x$  is the set of training inputs  $x^{(1)}, x^{(2)}, \dots, x^{(m)}$

#  $y$  is the set of training outputs (labels)  $y^{(1)}, y^{(2)}, \dots, y^{(m)}$

$\theta \leftarrow 0$

**repeat** til done

For each training tuple  $(x^{(i)}, y^{(i)})$  (in random order)

1. Optional (for reporting): # How are we doing on this tuple?

    Compute  $\hat{y}^{(i)} = f(x^{(i)}; \theta)$  # What is our estimated output  $\hat{y}$ ?

    Compute the loss  $L(\hat{y}^{(i)}, y^{(i)})$  # How far off is  $\hat{y}^{(i)}$  from the true output  $y^{(i)}$ ?

2.  $g \leftarrow \nabla_{\theta} L(f(x^{(i)}; \theta), y^{(i)})$  # How should we move  $\theta$  to maximize loss?

3.  $\theta \leftarrow \theta - \eta g$  # Go the other way instead

**return**  $\theta$

# Hyperparameters

The learning rate  $\eta$  is a **hyperparameter**

- too high: the learner will take big steps and overshoot
- too low: the learner will take too long

Hyperparameters:

- Briefly, a special kind of parameter for an ML model
- Instead of being learned by algorithm from supervision (like regular parameters), they are chosen by algorithm designer.

# Stochastic Gradient Descent

## Logistic Regression

# Logistic Regression

Stochastic Gradient Descent:  
An example and more details

# Working through an example

One step of gradient descent

A mini-sentiment example, where the true  $y=1$  (positive)

Two features:

$x_1 = 3$  (count of positive lexicon words)

$x_2 = 2$  (count of negative lexicon words)

Assume 3 parameters (2 weights and 1 bias) in  $\Theta^0$  are zero:

$$w_1 = w_2 = b = 0$$

$$\eta = 0.1$$



# Example of gradient descent

Update step for update  $\theta$  is:

$$w_1 = w_2 = b = 0;$$

$$x_1 = 3; \quad x_2 = 2$$

$$\theta^{t+1} = \theta^t - \eta \nabla_{\theta} L(f(x^{(i)}; \theta), y^{(i)})$$

where 
$$\frac{\partial L_{\text{CE}}(\hat{y}, y)}{\partial w_j} = [\sigma(w \cdot x + b) - y] x_j$$

Gradient vector has 3 dimensions:

$$\nabla_{w,b} = \begin{bmatrix} \frac{\partial L_{\text{CE}}(\hat{y}, y)}{\partial w_1} \\ \frac{\partial L_{\text{CE}}(\hat{y}, y)}{\partial w_2} \\ \frac{\partial L_{\text{CE}}(\hat{y}, y)}{\partial b} \end{bmatrix}$$

# Example of gradient descent

Update step for update  $\theta$  is:

$$w_1 = w_2 = b = 0;$$

$$x_1 = 3; \quad x_2 = 2$$

$$\theta^{t+1} = \theta^t - \eta \nabla_{\theta} L(f(x^{(i)}; \theta), y^{(i)})$$

where 
$$\frac{\partial L_{\text{CE}}(\hat{y}, y)}{\partial w_j} = [\sigma(w \cdot x + b) - y] x_j$$

Gradient vector has 3 dimensions:

$$\nabla_{w,b} = \begin{bmatrix} \frac{\partial L_{\text{CE}}(\hat{y}, y)}{\partial w_1} \\ \frac{\partial L_{\text{CE}}(\hat{y}, y)}{\partial w_2} \\ \frac{\partial L_{\text{CE}}(\hat{y}, y)}{\partial b} \end{bmatrix} = \begin{bmatrix} \phantom{0} \\ \phantom{0} \\ \phantom{0} \end{bmatrix}$$

# Example of gradient descent

Update step for update  $\theta$  is:

$$w_1 = w_2 = b = 0;$$

$$x_1 = 3; \quad x_2 = 2$$

$$\theta^{t+1} = \theta^t - \eta \nabla_{\theta} L(f(x^{(i)}; \theta), y^{(i)})$$

where 
$$\frac{\partial L_{\text{CE}}(\hat{y}, y)}{\partial w_j} = [\sigma(w \cdot x + b) - y]x_j$$

Gradient vector has 3 dimensions:

$$\nabla_{w,b} = \begin{bmatrix} \frac{\partial L_{\text{CE}}(\hat{y}, y)}{\partial w_1} \\ \frac{\partial L_{\text{CE}}(\hat{y}, y)}{\partial w_2} \\ \frac{\partial L_{\text{CE}}(\hat{y}, y)}{\partial b} \end{bmatrix} = \begin{bmatrix} (\sigma(w \cdot x + b) - y)x_1 \\ (\sigma(w \cdot x + b) - y)x_2 \\ \sigma(w \cdot x + b) - y \end{bmatrix}$$

# Example of gradient descent

Update step for update  $\theta$  is:

$$w_1 = w_2 = b = 0;$$

$$x_1 = 3; \quad x_2 = 2$$

$$\theta^{t+1} = \theta^t - \eta \nabla_{\theta} L(f(x^{(i)}; \theta), y^{(i)})$$

where 
$$\frac{\partial L_{\text{CE}}(\hat{y}, y)}{\partial w_j} = [\sigma(w \cdot x + b) - y]x_j$$

Gradient vector has 3 dimensions:

$$\nabla_{w,b} = \begin{bmatrix} \frac{\partial L_{\text{CE}}(\hat{y}, y)}{\partial w_1} \\ \frac{\partial L_{\text{CE}}(\hat{y}, y)}{\partial w_2} \\ \frac{\partial L_{\text{CE}}(\hat{y}, y)}{\partial b} \end{bmatrix} = \begin{bmatrix} (\sigma(w \cdot x + b) - y)x_1 \\ (\sigma(w \cdot x + b) - y)x_2 \\ \sigma(w \cdot x + b) - y \end{bmatrix} = \begin{bmatrix} (\sigma(0) - 1)x_1 \\ (\sigma(0) - 1)x_2 \\ \sigma(0) - 1 \end{bmatrix} ;$$

# Example of gradient descent

Update step for update  $\theta$  is:

$$w_1 = w_2 = b = 0;$$

$$x_1 = 3; \quad x_2 = 2$$

$$\theta^{t+1} = \theta^t - \eta \nabla_{\theta} L(f(x^{(i)}; \theta), y^{(i)})$$

where 
$$\frac{\partial L_{\text{CE}}(\hat{y}, y)}{\partial w_j} = [\sigma(w \cdot x + b) - y]x_j$$

Gradient vector has 3 dimensions:

$$\nabla_{w,b} = \begin{bmatrix} \frac{\partial L_{\text{CE}}(\hat{y}, y)}{\partial w_1} \\ \frac{\partial L_{\text{CE}}(\hat{y}, y)}{\partial w_2} \\ \frac{\partial L_{\text{CE}}(\hat{y}, y)}{\partial b} \end{bmatrix} = \begin{bmatrix} (\sigma(w \cdot x + b) - y)x_1 \\ (\sigma(w \cdot x + b) - y)x_2 \\ \sigma(w \cdot x + b) - y \end{bmatrix} = \begin{bmatrix} (\sigma(0) - 1)x_1 \\ (\sigma(0) - 1)x_2 \\ \sigma(0) - 1 \end{bmatrix} = \begin{bmatrix} -0.5x_1 \\ -0.5x_2 \\ -0.5 \end{bmatrix} = \begin{bmatrix} -1.5 \\ -1.0 \\ -0.5 \end{bmatrix}$$

# Example of gradient descent

$$\nabla_{w,b} = \begin{bmatrix} \frac{\partial L_{CE}(\hat{y}, y)}{\partial w_1} \\ \frac{\partial L_{CE}(\hat{y}, y)}{\partial w_2} \\ \frac{\partial L_{CE}(\hat{y}, y)}{\partial b} \end{bmatrix} = \begin{bmatrix} (\sigma(w \cdot x + b) - y)x_1 \\ (\sigma(w \cdot x + b) - y)x_2 \\ \sigma(w \cdot x + b) - y \end{bmatrix} = \begin{bmatrix} (\sigma(0) - 1)x_1 \\ (\sigma(0) - 1)x_2 \\ \sigma(0) - 1 \end{bmatrix} = \begin{bmatrix} -0.5x_1 \\ -0.5x_2 \\ -0.5 \end{bmatrix} = \begin{bmatrix} -1.5 \\ -1.0 \\ -0.5 \end{bmatrix}$$

Now that we have a gradient, we compute the new parameter vector  $\theta^1$  by moving  $\theta^0$  in the opposite direction from the gradient:

$$\theta^{t+1} = \theta^t - \eta \nabla_{\theta} L(f(x^{(i)}; \theta), y^{(i)}) \quad \eta = 0.1;$$

$$\theta^1 =$$

# Example of gradient descent

$$\nabla_{w,b} = \begin{bmatrix} \frac{\partial L_{CE}(\hat{y}, y)}{\partial w_1} \\ \frac{\partial L_{CE}(\hat{y}, y)}{\partial w_2} \\ \frac{\partial L_{CE}(\hat{y}, y)}{\partial b} \end{bmatrix} = \begin{bmatrix} (\sigma(w \cdot x + b) - y)x_1 \\ (\sigma(w \cdot x + b) - y)x_2 \\ \sigma(w \cdot x + b) - y \end{bmatrix} = \begin{bmatrix} (\sigma(0) - 1)x_1 \\ (\sigma(0) - 1)x_2 \\ \sigma(0) - 1 \end{bmatrix} = \begin{bmatrix} -0.5x_1 \\ -0.5x_2 \\ -0.5 \end{bmatrix} = \begin{bmatrix} -1.5 \\ -1.0 \\ -0.5 \end{bmatrix}$$

Now that we have a gradient, we compute the new parameter vector  $\theta^1$  by moving  $\theta^0$  in the opposite direction from the gradient:

$$\theta^{t+1} = \theta^t - \eta \nabla_{\theta} L(f(x^{(i)}; \theta), y^{(i)}) \quad \eta = 0.1;$$

$$\theta^1 = \begin{bmatrix} w_1 \\ w_2 \\ b \end{bmatrix} - \eta \begin{bmatrix} -1.5 \\ -1.0 \\ -0.5 \end{bmatrix}$$

# Example of gradient descent

$$\nabla_{w,b} = \begin{bmatrix} \frac{\partial L_{CE}(\hat{y}, y)}{\partial w_1} \\ \frac{\partial L_{CE}(\hat{y}, y)}{\partial w_2} \\ \frac{\partial L_{CE}(\hat{y}, y)}{\partial b} \end{bmatrix} = \begin{bmatrix} (\sigma(w \cdot x + b) - y)x_1 \\ (\sigma(w \cdot x + b) - y)x_2 \\ \sigma(w \cdot x + b) - y \end{bmatrix} = \begin{bmatrix} (\sigma(0) - 1)x_1 \\ (\sigma(0) - 1)x_2 \\ \sigma(0) - 1 \end{bmatrix} = \begin{bmatrix} -0.5x_1 \\ -0.5x_2 \\ -0.5 \end{bmatrix} = \begin{bmatrix} -1.5 \\ -1.0 \\ -0.5 \end{bmatrix}$$

Now that we have a gradient, we compute the new parameter vector  $\theta^1$  by moving  $\theta^0$  in the opposite direction from the gradient:

$$\theta^{t+1} = \theta^t - \eta \nabla_{\theta} L(f(x^{(i)}; \theta), y^{(i)}) \quad \eta = 0.1;$$

$$\theta^1 = \begin{bmatrix} w_1 \\ w_2 \\ b \end{bmatrix} - \eta \begin{bmatrix} -1.5 \\ -1.0 \\ -0.5 \end{bmatrix} = \begin{bmatrix} .15 \\ .1 \\ .05 \end{bmatrix}$$



# Example of gradient descent

$$\nabla_{w,b} = \begin{bmatrix} \frac{\partial L_{CE}(\hat{y}, y)}{\partial w_1} \\ \frac{\partial L_{CE}(\hat{y}, y)}{\partial w_2} \\ \frac{\partial L_{CE}(\hat{y}, y)}{\partial b} \end{bmatrix} = \begin{bmatrix} (\sigma(w \cdot x + b) - y)x_1 \\ (\sigma(w \cdot x + b) - y)x_2 \\ \sigma(w \cdot x + b) - y \end{bmatrix} = \begin{bmatrix} (\sigma(0) - 1)x_1 \\ (\sigma(0) - 1)x_2 \\ \sigma(0) - 1 \end{bmatrix} = \begin{bmatrix} -0.5x_1 \\ -0.5x_2 \\ -0.5 \end{bmatrix} = \begin{bmatrix} -1.5 \\ -1.0 \\ -0.5 \end{bmatrix}$$

Now that we have a gradient, we compute the new parameter vector  $\theta^1$  by moving  $\theta^0$  in the opposite direction from the gradient:

$$\theta^{t+1} = \theta^t - \eta \nabla_{\theta} L(f(x^{(i)}; \theta), y^{(i)}) \quad \eta = 0.1;$$

$$\theta^1 = \begin{bmatrix} w_1 \\ w_2 \\ b \end{bmatrix} - \eta \begin{bmatrix} -1.5 \\ -1.0 \\ -0.5 \end{bmatrix} = \begin{bmatrix} .15 \\ .1 \\ .05 \end{bmatrix}$$

Note that enough negative examples would eventually make  $w_2$  negative

# Mini-batch training

Stochastic gradient descent chooses a single random example at a time.

That can result in choppy movements

More common to compute gradient over batches of training instances.

**Batch training:** entire dataset

**Mini-batch training:**  $m$  examples (512, or 1024)

# Logistic Regression

Stochastic Gradient Descent:  
An example and more details

Regularization

Logistic  
Regression

# Overfitting

A model that perfectly match the training data has a problem.

It will also **overfit** to the data, modeling noise

- A random word that perfectly predicts  $y$  (it happens to only occur in one class) will get a very high weight.
- Failing to generalize to a test set without this word.

A good model should be able to **generalize**

# Overfitting

+

This movie drew me in, and it'll  
do the same to you.

-

I can't tell you how much I hated  
this movie. It sucked.

Useful or harmless features

X1 = "this"

X2 = "movie"

X3 = "hated"

X4 = "drew me in"

4gram features that just  
"memorize" training set and  
might cause problems

X5 = "the same to you"

X7 = "tell you how much"

# Overfitting

4-gram model on tiny data will just memorize the data

- 100% accuracy on the training set

But it will be surprised by the novel 4-grams in the test data

- Low accuracy on test set

Models that are too powerful can **overfit** the data

- Fitting the details of the training data so exactly that the model doesn't generalize well to the test set
- How to avoid overfitting?
  - Regularization in logistic regression
  - Dropout in neural networks

# Regularization

A solution for overfitting

Add a regularization term  $R(\theta)$  to the loss function  
(for now written as maximizing logprob rather than minimizing loss)

$$\hat{\theta} = \underset{\theta}{\operatorname{argmax}} \sum_{i=1}^m \log P(y^{(i)} | x^{(i)}) - \alpha R(\theta)$$

Idea: choose an  $R(\theta)$  that penalizes large weights

- fitting the data well with lots of big weights not as good as fitting the data a little less well, with small weights



# L2 Regularization (= ridge regression)

The sum of the squares of the weights

The name is because this is the (square of the)

**L2 norm**  $\|\theta\|_2$ , = **Euclidean distance** of  $\theta$  to the origin.

$$R(\theta) = \|\theta\|_2^2 = \sum_{j=1}^n \theta_j^2$$

L2 regularized objective function:

$$\hat{\theta} = \operatorname{argmax}_{\theta} \left[ \sum_{i=1}^m \log P(y^{(i)} | x^{(i)}) \right] - \alpha \sum_{j=1}^n \theta_j^2$$

# L1 Regularization (= lasso regression)

The sum of the (absolute value of the) weights

Named after the **L1 norm**  $\|W\|_1$ , = sum of the absolute values of the weights, = **Manhattan distance**

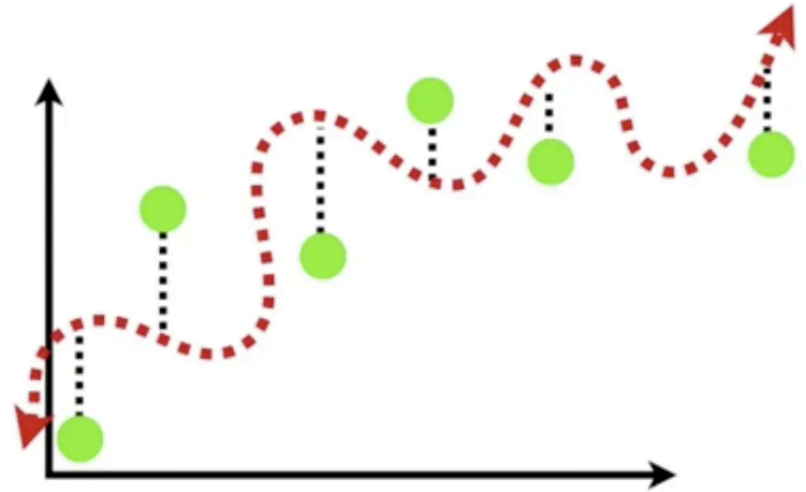
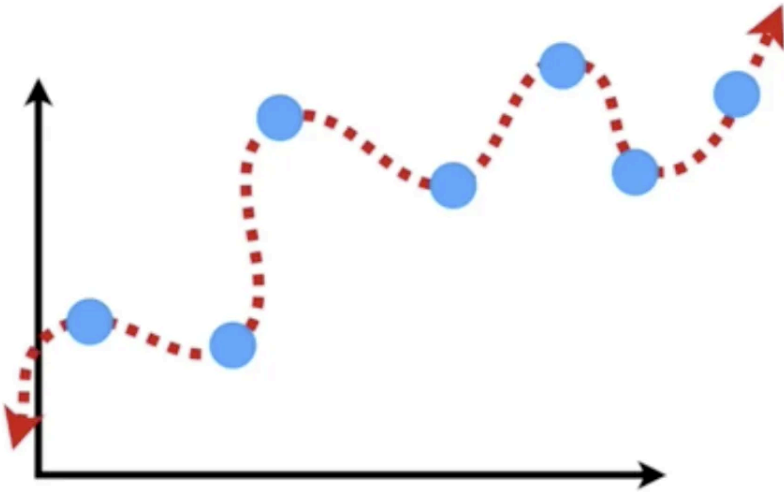
$$R(\theta) = \|\theta\|_1 = \sum_{i=1}^n |\theta_i|$$

L1 regularized objective function:

$$\hat{\theta} = \underset{\theta}{\operatorname{argmax}} \left[ \sum_{i=1}^m \log P(y^{(i)} | x^{(i)}) \right] - \alpha \sum_{j=1}^n |\theta_j|$$

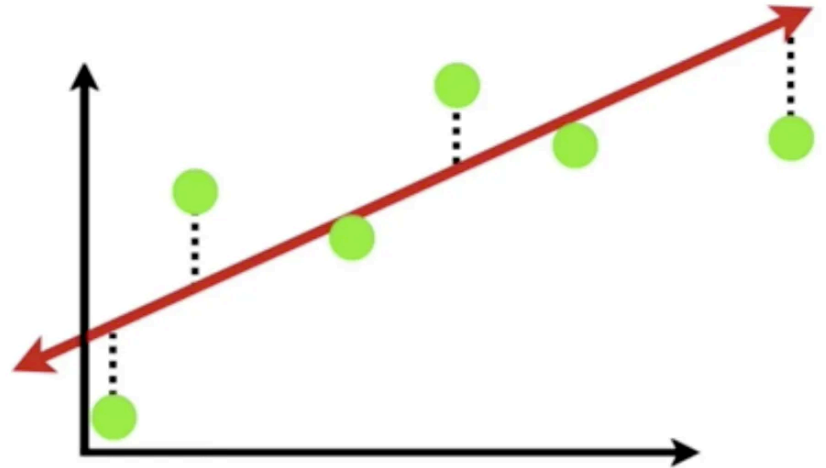
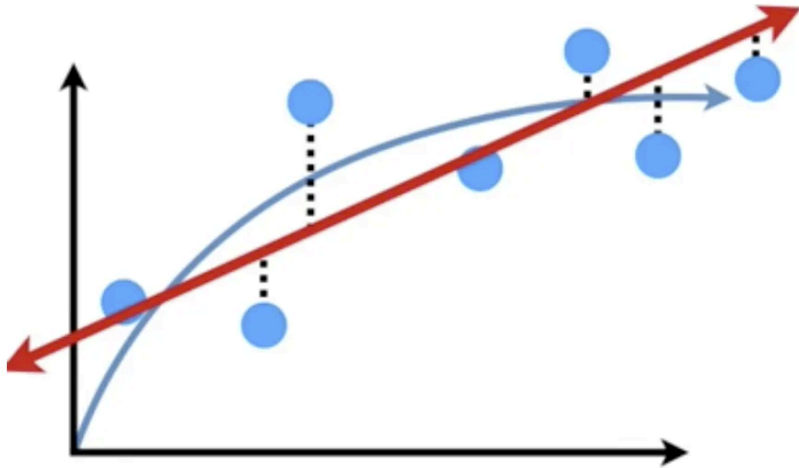
# Variance

In Machine Learning lingo, the difference in fits between data sets is called **Variance**.



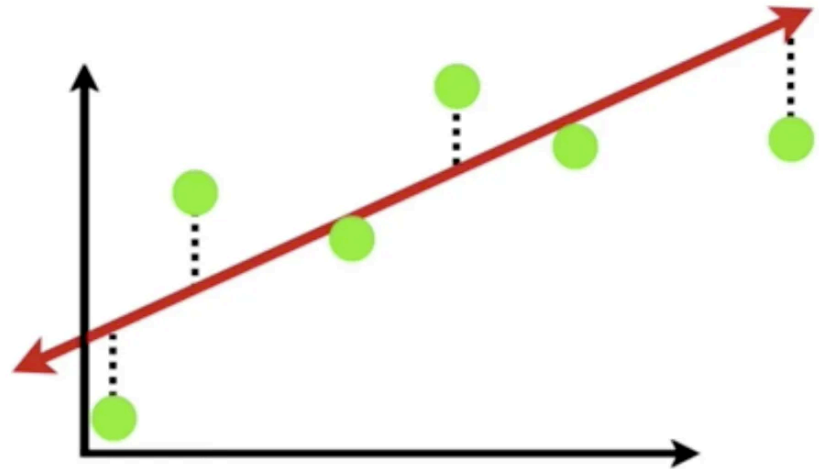
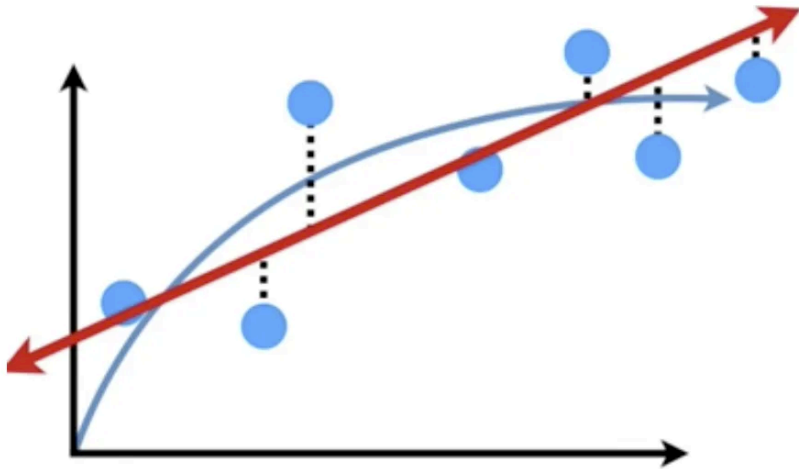
# Linear Regression

Linear regression uses the sum of squares to find a line that best fits all of the data points



# Ridge Regression

Ridge regression is Linear Regression with an added bias intended to exaggerate the regression towards the linear approximation (to reduce the pull of outlier values on the mean (the best fit))



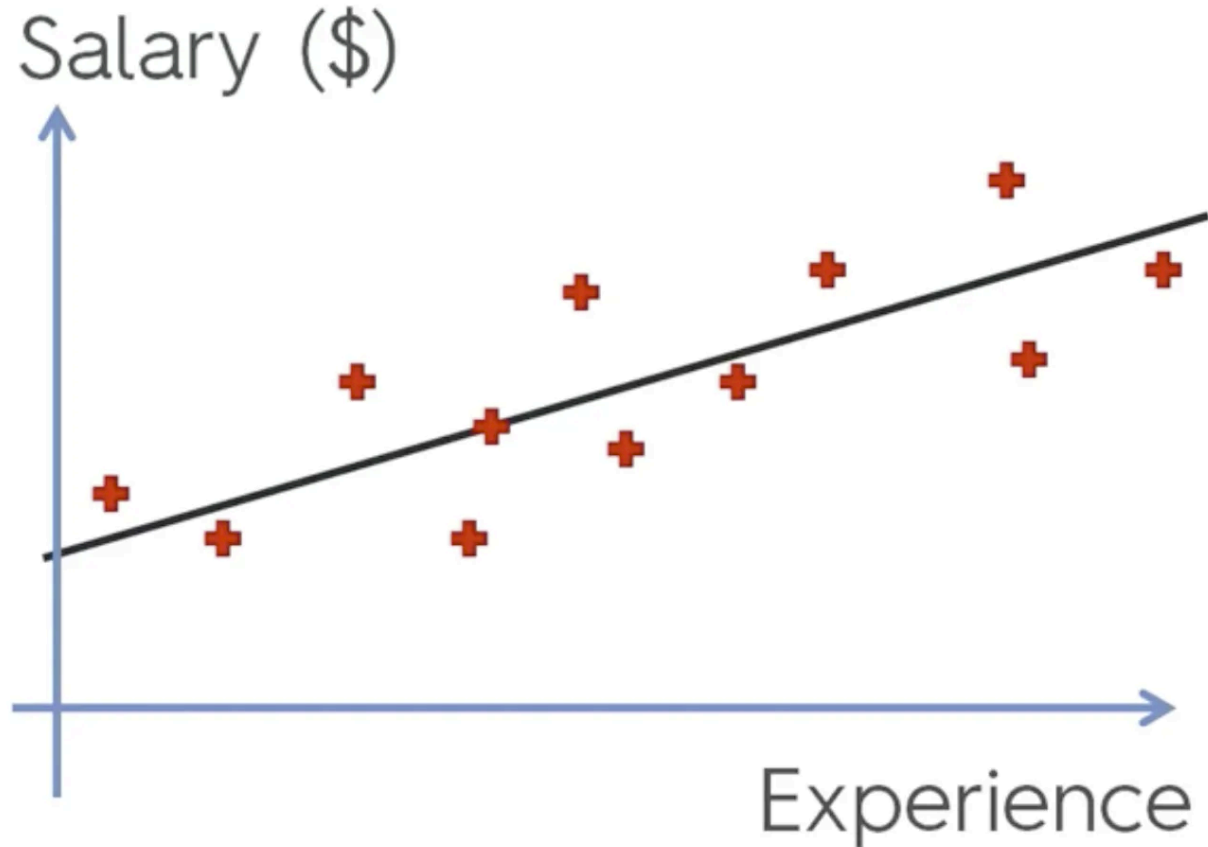
# Ridge Regression Penalty

The penalty is calculated by taking the sum of the squares of the weights (the total distance from the mean), multiplying it by a coefficient  $\lambda$ , and subtracting it from the loss function

$$\theta^1 = \begin{bmatrix} w_1 \\ w_2 \\ b \end{bmatrix} - \eta \begin{bmatrix} -1.5 \\ -1.0 \\ -0.5 \end{bmatrix} = \begin{bmatrix} .15 \\ .1 \\ .05 \end{bmatrix} \rightarrow \lambda(0.15^2 + 0.1^2) = 0.0325\lambda$$

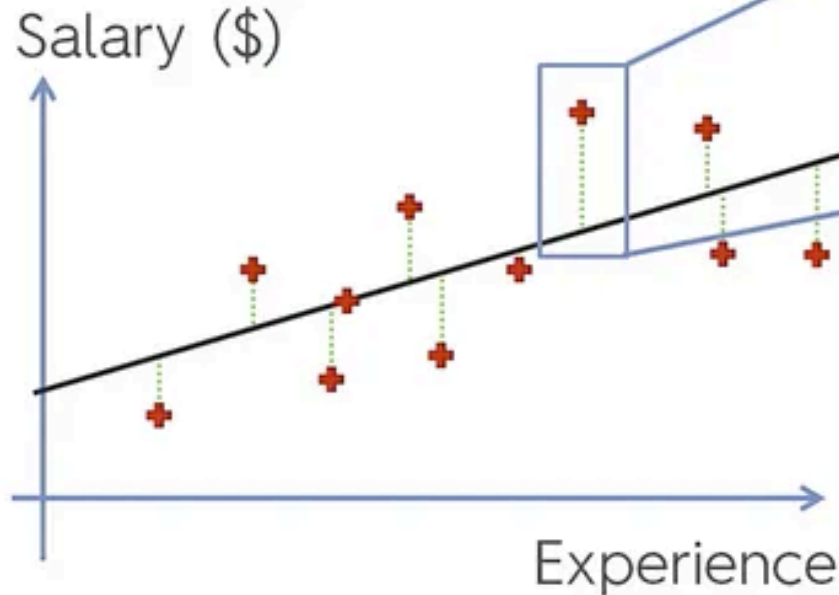
Choosing  $\lambda$  is often a matter of using **cross-validation** on a variety of  $\lambda$  values and find one with the least **variance**

# Ridge Regression Example



# Ridge Regression Example

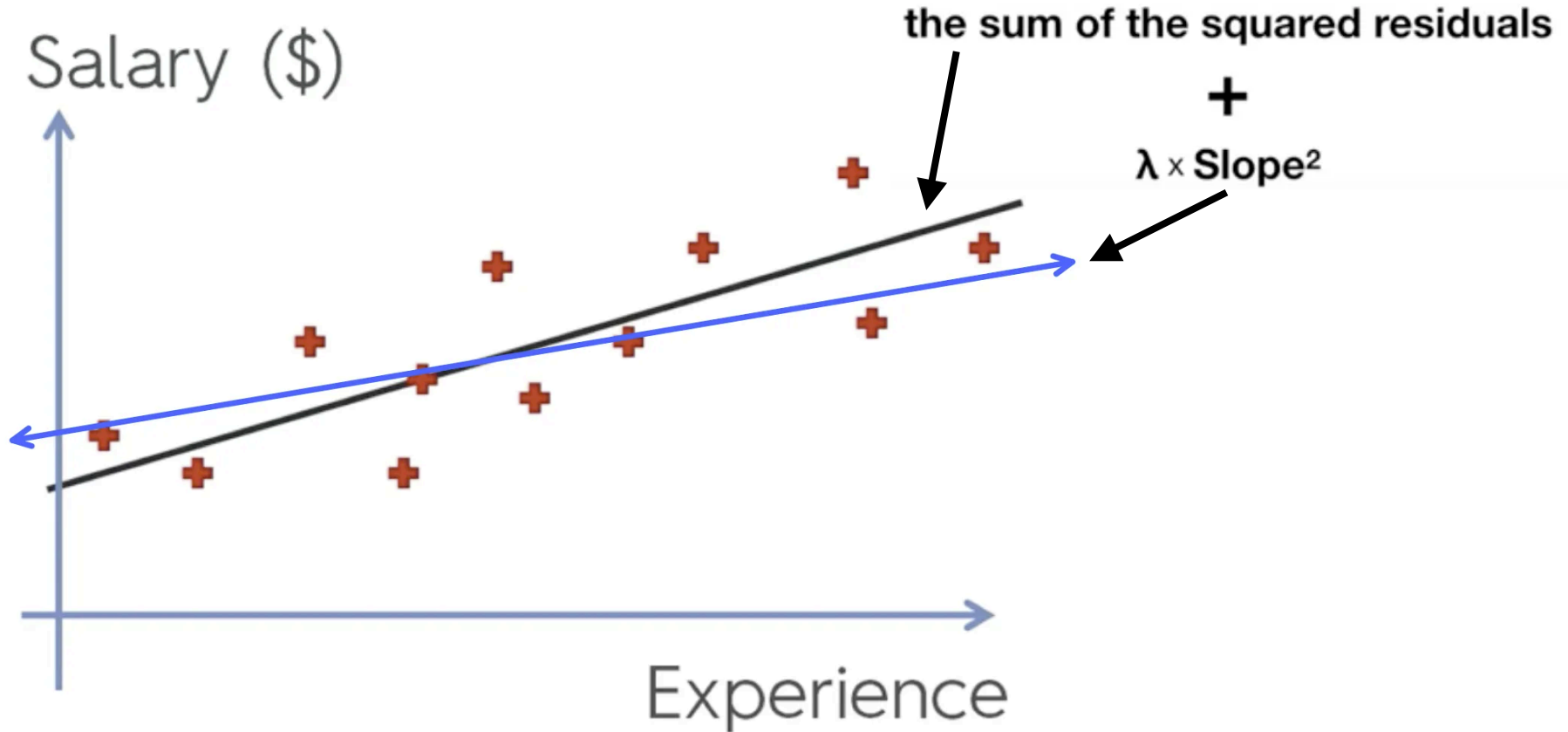
Simple Linear Regression:



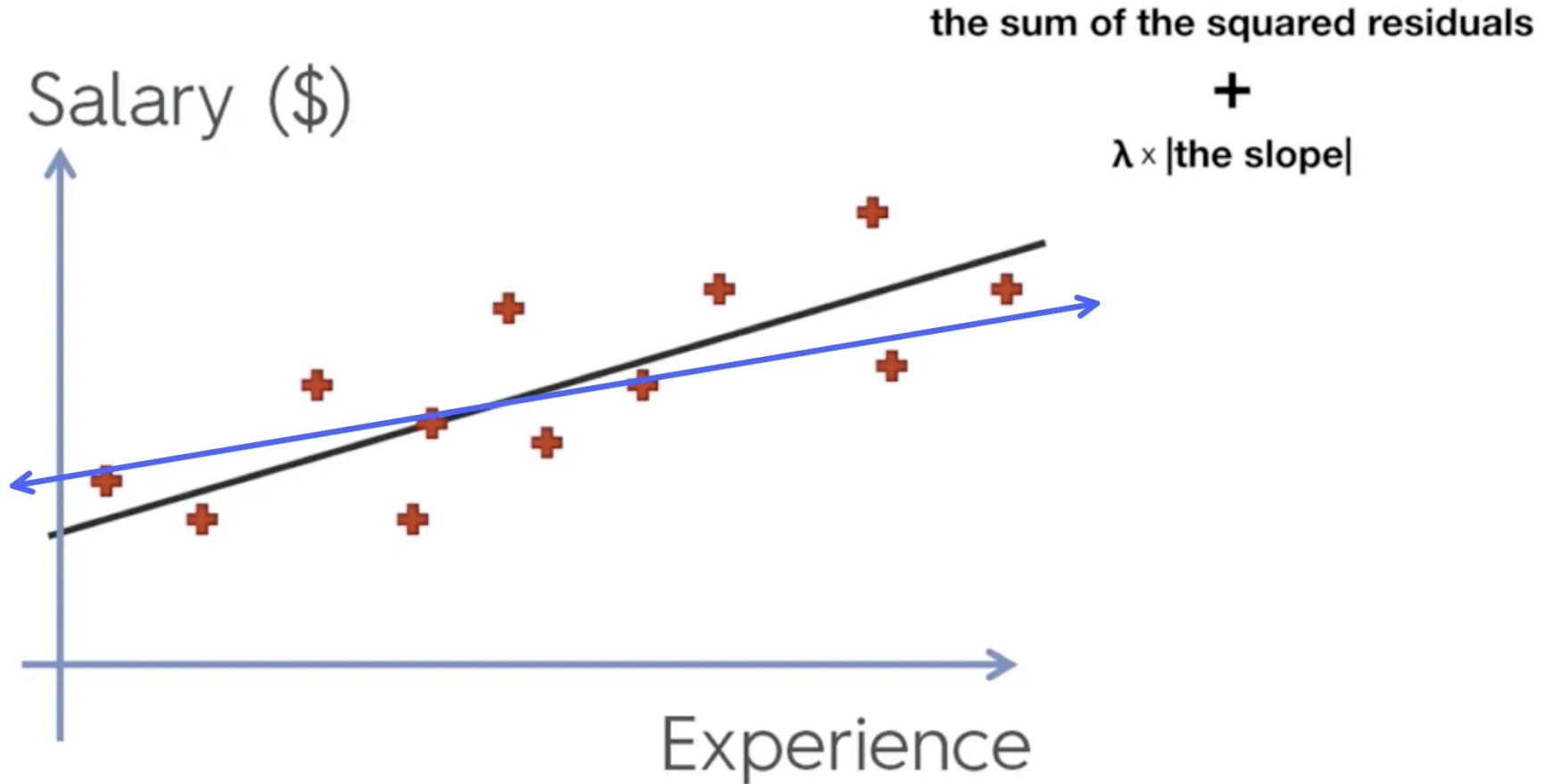
$$\text{SUM } (y - \hat{y})^2 \rightarrow \min$$



# Ridge Regression Example



# Lasso Regression



# Regularization

- Lasso Regression
  - Can shrink slope to 0
  - Useful for filtering irrelevant features
  - Will ignore some low weighted features
- Ridge Regression
  - Can shrink slope asymptotically close to 0
  - High **multicollinearity**
    - Highly correlated features
  - Will use all features, regardless of importance

# Regularization

## Logistic Regression

# Logistic Regression

## Multinomial Logistic Regression

# Multinomial Logistic Regression

Often we need more than 2 classes

- Positive/negative/neutral
- Parts of speech (noun, verb, adjective, adverb, preposition, etc.)
- Classify emergency SMSs into different actionable classes

If >2 classes we use **multinomial logistic regression**

= Softmax regression

= Multinomial logit

= (defunct names : Maximum entropy modeling or MaxEnt)

So "logistic regression" will just mean binary (2 output classes)

# Multinomial Logistic Regression

The probability of everything must still sum to 1

$$P(\text{positive}|\text{doc}) + P(\text{negative}|\text{doc}) + P(\text{neutral}|\text{doc}) = 1$$

Need a generalization of the sigmoid called the **softmax**

- Takes a vector  $z = [z_1, z_2, \dots, z_k]$  of  $k$  arbitrary values
- Outputs a probability distribution
  - each value in the range  $[0,1]$
  - all the values summing to 1

# The softmax function

Turns a vector  $z = [z_1, z_2, \dots, z_k]$  of  $k$  arbitrary values into probabilities

$$\text{softmax}(z_i) = \frac{\exp(z_i)}{\sum_{j=1}^k \exp(z_j)} \quad 1 \leq i \leq k$$

The denominator  $\sum_{i=1}^k$  is used to normalize all the values into probabilities

$$\text{softmax}(z_i) = \left[ \frac{\exp(z_1)}{\sum_{j=1}^k \exp(z_j)}, \frac{\exp(z_2)}{\sum_{j=1}^k \exp(z_j)}, \dots, \frac{\exp(z_k)}{\sum_{j=1}^k \exp(z_j)} \right]$$



# The softmax function

- Turns a vector  $z = [z_1, z_2, \dots, z_k]$  of  $k$  arbitrary values into probabilities

$$z = [0.6, 1.1, -1.5, 1.2, 3.2, -1.1]$$

$$\text{softmax}(z_i) = \left[ \frac{\exp(z_1)}{\sum_{j=1}^k \exp(z_j)}, \frac{\exp(z_2)}{\sum_{j=1}^k \exp(z_j)}, \dots, \frac{\exp(z_k)}{\sum_{j=1}^k \exp(z_j)} \right]$$

$$[0.055, 0.090, 0.0067, 0.10, 0.74, 0.010]$$

# Softmax in multinomial logistic regression

$$p(y = c | x) = \frac{\exp(w_c \cdot x + b_c)}{\sum_{j=1}^k \exp(w_j \cdot x + b_j)}$$

Input is still the dot product between weight vector  $w$  and input vector  $x$

But now we'll need separate weight vectors for each of the  $K$  classes.

# Features in binary versus multinomial logistic regression

Binary: positive weight  $\rightarrow y=1$  neg weight  $\rightarrow y=0$

$$x_5 = \begin{cases} 1 & \text{if "!"} \in \text{doc} \\ 0 & \text{otherwise} \end{cases} \quad w_5 = 3.0$$

Multinomial: separate weights for each class:

| Feature  | Definition                                                                           | $w_{5,+}$ | $w_{5,-}$ | $w_{5,0}$ |
|----------|--------------------------------------------------------------------------------------|-----------|-----------|-----------|
| $f_5(x)$ | $\begin{cases} 1 & \text{if "!"} \in \text{doc} \\ 0 & \text{otherwise} \end{cases}$ | 3.5       | 3.1       | -5.3      |

# Logistic Regression

## Multinomial Logistic Regression

# Vector Semantics & Embeddings

## Word Meaning

# Lemmas and senses

lemma

mouse (N)

sense

1. any of numerous small rodents...

2. a hand-operated device that controls  
a cursor...

Modified from the online thesaurus WordNet

A **sense** or “**concept**” is the meaning component of a word  
Lemmas can be **polysemous** (have multiple senses)

# Relations between senses: Synonymy

Synonyms have the same meaning in some or all contexts.

- filbert / hazelnut
- couch / sofa
- big / large
- automobile / car
- vomit / throw up
- water / H<sub>2</sub>O

# Relations between senses: Synonymy

Note that there are probably no examples of perfect synonymy.

- Even if many aspects of meaning are identical
- Still may differ based on politeness, slang, register, genre, etc.



# Relation: Synonymy?

water/H<sub>2</sub>O

"H<sub>2</sub>O" in a surfing guide?

big/large

my big sister != my large sister

# The Linguistic Principle of Contrast

Difference in form → difference in meaning

# Relation: Similarity

Words with similar meanings. Not synonyms, but sharing some element of meaning

car, bicycle

cow, horse

# Ask humans how similar 2 words are

| word1  | word2      | similarity |
|--------|------------|------------|
| vanish | disappear  | 9.8        |
| behave | obey       | 7.3        |
| belief | impression | 5.95       |
| muscle | bone       | 3.65       |
| modest | flexible   | 0.98       |
| hole   | agreement  | 0.3        |

# Relation: Word relatedness

Also called "word association"

Words can be related in any way, perhaps via a semantic frame or field

- coffee, tea: **similar**
- coffee, cup: **related**, not similar

# Semantic field

Words that

- cover a particular semantic domain
- bear structured relations with each other.

## **hospitals**

*surgeon, scalpel, nurse, anaesthetic, hospital*

## **restaurants**

*waiter, menu, plate, food, menu, chef*

## **houses**

*door, roof, kitchen, family, bed*

# Relation: Antonymy

Senses that are opposites with respect to only one feature of meaning

Otherwise, they are very similar!

|            |            |           |           |
|------------|------------|-----------|-----------|
| dark/light | short/long | fast/slow | rise/fall |
| hot/cold   | up/down    | in/out    |           |

More formally: antonyms can

- define a binary opposition or be at opposite ends of a scale
  - long/short, fast/slow
- Be *reversives*:
  - rise/fall, up/down

# Connotation (sentiment)

- Words have **affective** meanings
  - Positive connotations (*happy*)
  - Negative connotations (*sad*)
- Connotations can be subtle:
  - Positive connotation: *copy, replica, reproduction*
  - Negative connotation: *fake, knockoff, forgery*
- Evaluation (sentiment!)
  - Positive evaluation (*great, love*)
  - Negative evaluation (*terrible, hate*)



# So far

## Concepts or word senses

- Have a complex many-to-many association with **words** (homonymy, multiple senses)

## Have relations with each other

- Synonymy
- Antonymy
- Similarity
- Relatedness
- Connotation

# Vector Semantics & Embeddings

## Word Meaning

# Vector Semantics & Embeddings

## Vector Semantics

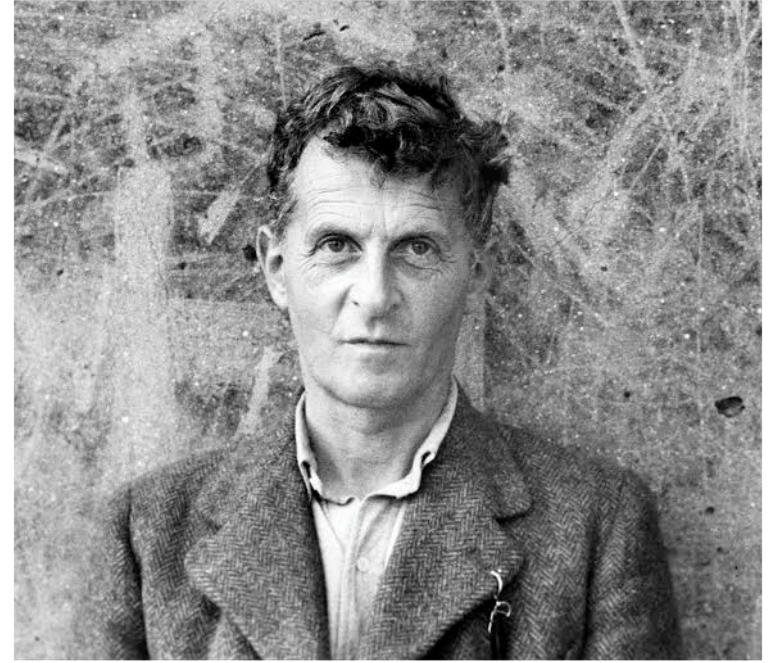
# Computational models of word meaning

Can we build a theory of how to represent word meaning, that accounts for at least some of the desiderata?

We'll introduce **vector semantics**

The standard model in modern language processing

# Ludwig Wittgenstein



PI #43:

"The meaning of a word is its use in the language"

# Let's define words by their usages

One way to define "usage":

words are defined by their environments (the words around them)

Zellig Harris (1954):

**If A and B have almost identical environments we say that they are synonyms.**

# What does recent English borrowing *ong choi* mean?

Suppose you see these sentences:

- Ong choi is delicious **sautéed with garlic**.
- Ong choi is superb **over rice**
- Ong choi **leaves** with salty sauces

And you've also seen these:

- ...spinach **sautéed with garlic over rice**
- Chard stems and **leaves** are **delicious**
- Collard greens and other **salty** leafy greens

Conclusion:

- Ongchoi is a leafy green like spinach, chard, or collard greens
- We could conclude this based on words like "leaves" and "delicious" and "sauteed"

# Ongchoi: *Ipomoea aquatica* "Water Spinach"

空心菜

kangkong

rau muống

...





## Idea 1: Defining meaning by linguistic distribution

Let's define the meaning of a word by its distribution in language use, meaning its neighboring words or grammatical environments.

## Idea 2: Meaning as a point in space (Osgood et al. 1957)

### 3 affective dimensions for a word

- **valence**: pleasantness
- **arousal**: intensity of emotion
- **dominance**: the degree of control exerted

|                  | Word       | Score |  | Word      | Score |
|------------------|------------|-------|--|-----------|-------|
| <b>Valence</b>   | love       | 1.000 |  | toxic     | 0.008 |
|                  | happy      | 1.000 |  | nightmare | 0.005 |
| <b>Arousal</b>   | elated     | 0.960 |  | mellow    | 0.069 |
|                  | frenzy     | 0.965 |  | napping   | 0.046 |
| <b>Dominance</b> | powerful   | 0.991 |  | weak      | 0.045 |
|                  | leadership | 0.983 |  | empty     | 0.081 |

NRC VAD Lexicon  
(Mohammad 2018)

Hence the connotation of a word is a vector in 3-space

Idea 1: Defining meaning by linguistic distribution

Idea 2: Meaning as a point in multidimensional space

# Defining meaning as a point in space based on distribution

Each word = a vector (not just "good" or " $w_{45}$ ")

Similar words are "**nearby in semantic space**"

We build this space automatically by seeing which words are **nearby in text**



# We define meaning of a word as a vector

Called an "embedding" because it's embedded into a space (see textbook)

The standard way to represent meaning in NLP

**Every modern NLP algorithm uses embeddings as the representation of word meaning**

Fine-grained model of meaning for similarity

# Intuition: why vectors?

Consider sentiment analysis:

- With **words**, a feature is a word identity
  - Feature 5: 'The previous word was "terrible"'
  - requires **exact same word** to be in training and test
- With **embeddings**:
  - Feature is a word vector
  - 'The previous word was vector [35,22,17...]
  - Now in the test set we might see a similar vector [34,21,14]
  - We can generalize to **similar but unseen** words!!!

# We'll discuss 2 kinds of embeddings

## tf-idf

- Information Retrieval workhorse!
- A common baseline model
- **Sparse** vectors
- Words are represented by (a simple function of) the **counts** of nearby words

## Word2vec

- **Dense** vectors
- Representation is created by training a classifier to **predict** whether a word is likely to appear nearby
- Later we'll discuss extensions called **contextual embeddings**

# Vector Semantics & Embeddings

## Vector Semantics



# Vector Semantics & Embeddings

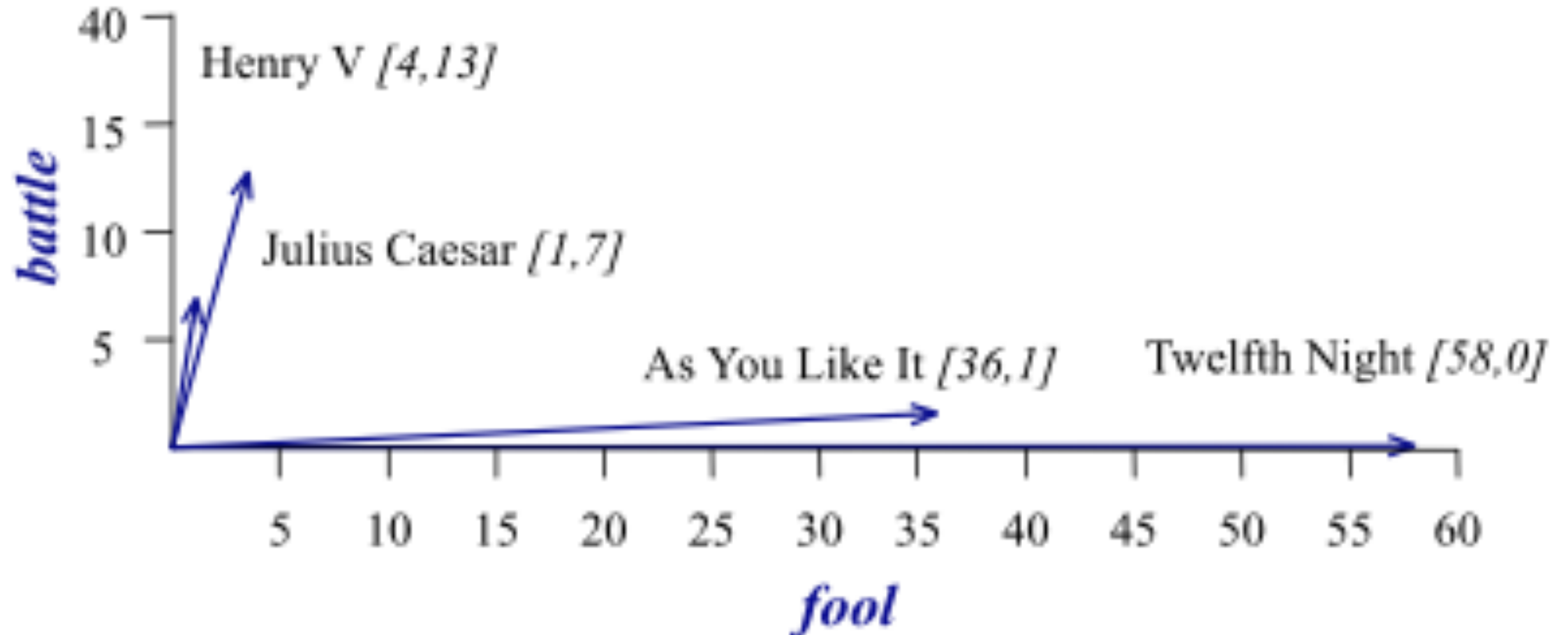
## Words and Vectors

# Term-document matrix

Each document is represented by a vector of words

|        | As You Like It | Twelfth Night | Julius Caesar | Henry V |
|--------|----------------|---------------|---------------|---------|
| battle | 1              | 0             | 7             | 13      |
| good   | 114            | 80            | 62            | 89      |
| fool   | 36             | 58            | 1             | 4       |
| wit    | 20             | 15            | 2             | 3       |

# Visualizing document vectors



# Vectors are the basis of information retrieval

|        | As You Like It | Twelfth Night | Julius Caesar | Henry V |
|--------|----------------|---------------|---------------|---------|
| battle | 1              | 0             | 7             | 13      |
| good   | 114            | 80            | 62            | 89      |
| fool   | 36             | 58            | 1             | 4       |
| wit    | 20             | 15            | 2             | 3       |

Vectors are similar for the two comedies

But comedies are different than the other two

Comedies have more *fools* and *wit* and fewer *battles*.

# Idea for word meaning: Words can be vectors too!!!

|               | As You Like It | Twelfth Night | Julius Caesar | Henry V |
|---------------|----------------|---------------|---------------|---------|
| <b>battle</b> | 1              | 0             | 7             | 13      |
| <b>good</b>   | 114            | 80            | 62            | 89      |
| <b>fool</b>   | 36             | 58            | 1             | 4       |
| <b>wit</b>    | 20             | 15            | 2             | 3       |

*battle* is "the kind of word that occurs in Julius Caesar and Henry V"

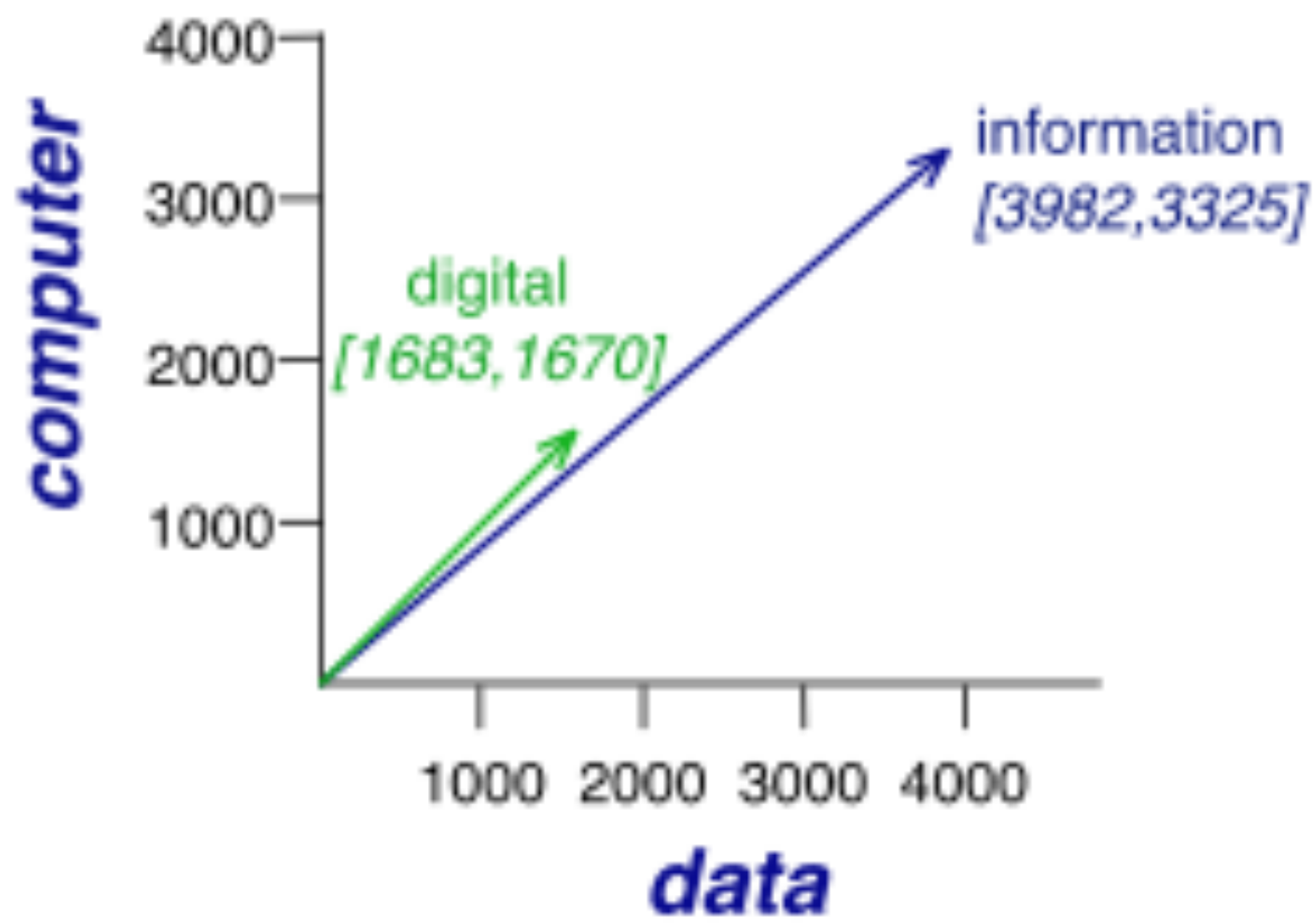
*fool* is "the kind of word that occurs in comedies, especially Twelfth Night"

# More common: word-word matrix (or "term-context matrix")

Two **words** are similar in meaning if their context vectors are similar

is traditionally followed by **cherry** pie, a traditional dessert  
often mixed, such as **strawberry** rhubarb pie. Apple pie  
computer peripherals and personal **digital** assistants. These devices usually  
a computer. This includes **information** available on the internet

|             | aardvark | ... | computer | data | result | pie | sugar | ... |
|-------------|----------|-----|----------|------|--------|-----|-------|-----|
| cherry      | 0        | ... | 2        | 8    | 9      | 442 | 25    | ... |
| strawberry  | 0        | ... | 0        | 0    | 1      | 60  | 19    | ... |
| digital     | 0        | ... | 1670     | 1683 | 85     | 5   | 4     | ... |
| information | 0        | ... | 3325     | 3982 | 378    | 5   | 13    | ... |



# Vector Semantics & Embeddings

## Words and Vectors



# Cosine for computing word similarity

Vector  
Semantics &  
Embeddings

# Computing word similarity: Dot product and cosine

The dot product between two vectors is a scalar:

$$\text{dot product}(\mathbf{v}, \mathbf{w}) = \mathbf{v} \cdot \mathbf{w} = \sum_{i=1}^N v_i w_i = v_1 w_1 + v_2 w_2 + \dots + v_N w_N$$

The dot product tends to be high when the two vectors have large values in the same dimensions

Dot product can thus be a useful similarity metric between vectors

# Problem with raw dot-product

Dot product favors long vectors

Dot product is higher if a vector is longer (has higher values in many dimension)

Vector length:

$$|\mathbf{v}| = \sqrt{\sum_{i=1}^N v_i^2}$$

Frequent words (of, the, you) have long vectors (since they occur many times with other words).

So dot product overly favors frequent words

# Alternative: cosine for computing word similarity

$$\text{cosine}(\vec{v}, \vec{w}) = \frac{\mathbf{v} \cdot \mathbf{w}}{|\mathbf{v}| |\mathbf{w}|} = \frac{\sum_{i=1}^N v_i w_i}{\sqrt{\sum_{i=1}^N v_i^2} \sqrt{\sum_{i=1}^N w_i^2}}$$

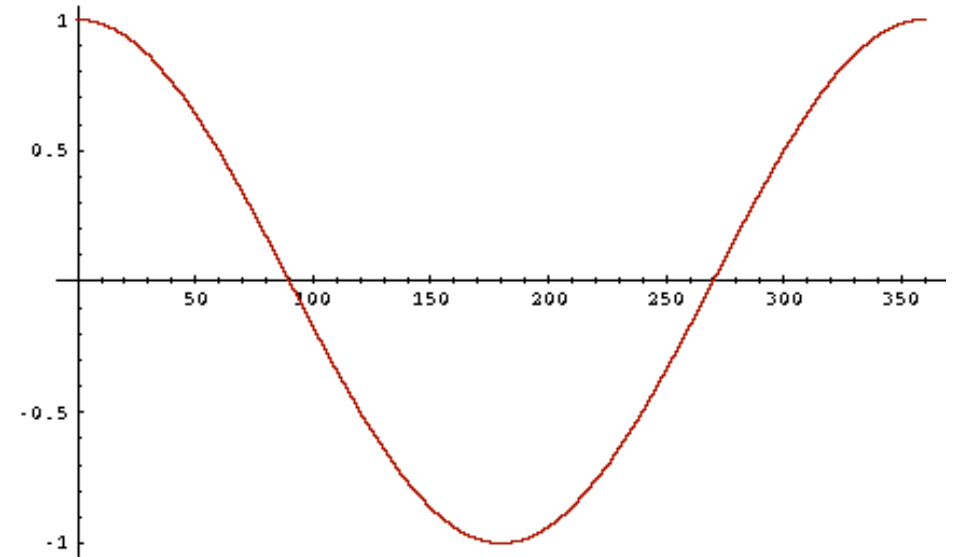
Based on the definition of the dot product between two vectors  $\mathbf{a}$  and  $\mathbf{b}$

$$\mathbf{a} \cdot \mathbf{b} = |\mathbf{a}| |\mathbf{b}| \cos\theta$$

$$\frac{\mathbf{a} \cdot \mathbf{b}}{|\mathbf{a}| |\mathbf{b}|} = \cos\theta$$

# Cosine as a similarity metric

- 1: vectors point in opposite directions
- +1: vectors point in same directions
- 0: vectors are orthogonal



But since raw frequency values are non-negative, the cosine for term-term matrix vectors ranges from 0–1

# Cosine examples

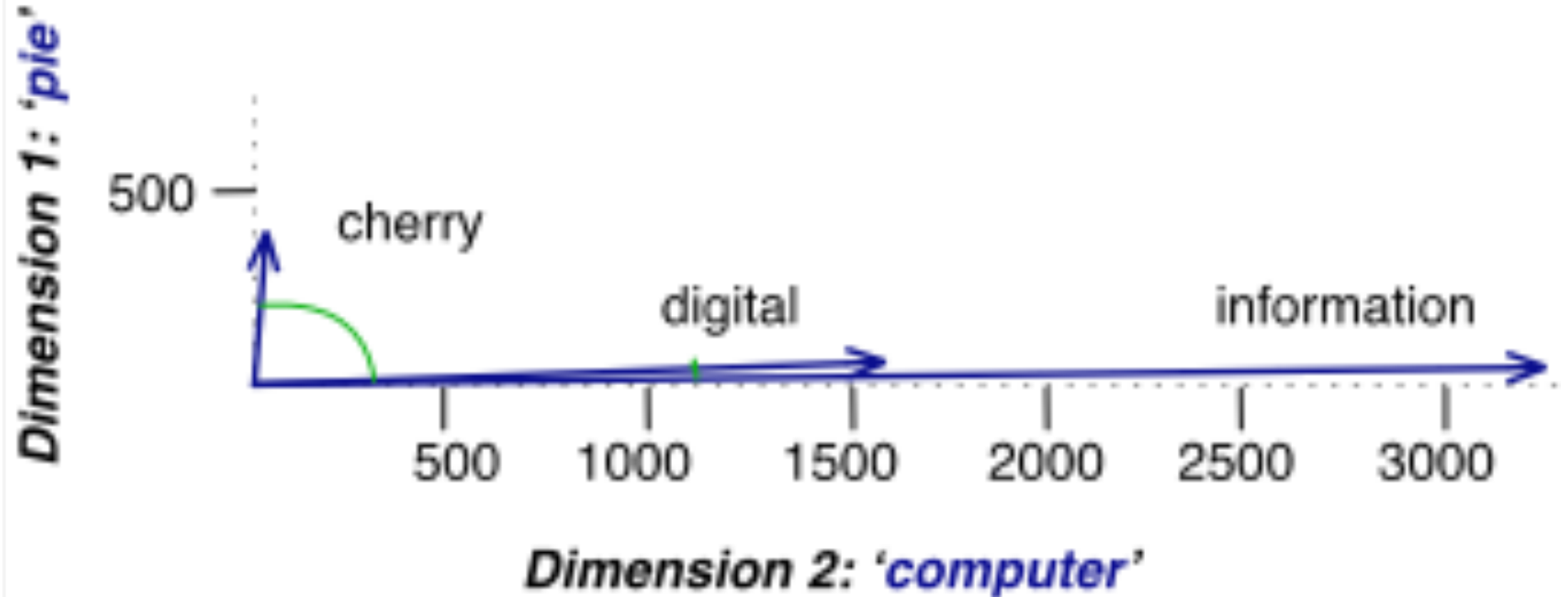
$$\text{cosine}(v, w) = \frac{v \cdot w}{|v||w|} = \frac{\sum_{i=1}^N v_i w_i}{\sqrt{\sum_{i=1}^N v_i^2} \sqrt{\sum_{i=1}^N w_i^2}}$$

|             | pie | data | computer |
|-------------|-----|------|----------|
| cherry      | 442 | 8    | 2        |
| digital     | 5   | 1683 | 1670     |
| information | 5   | 3982 | 3325     |

$$\text{cos}(\text{cherry}, \text{information}) = \frac{442 * 5 + 8 * 3982 + 2 * 3325}{\sqrt{442^2 + 8^2 + 2^2} \sqrt{5^2 + 3982^2 + 3325^2}} = 0.018$$

$$\text{cos}(\text{digital}, \text{information}) = \frac{5 * 5 + 1683 * 3982 + 1670 * 3325}{\sqrt{5^2 + 1683^2 + 1670^2} \sqrt{5^2 + 3982^2 + 3325^2}} = 0.996$$

## Visualizing cosines (well, angles)



# Vector Semantics & Embeddings

Cosine for computing word  
similarity



# Vector Semantics & Embeddings

TF-IDF

# But raw frequency is a bad representation

- The co-occurrence matrices we have seen represent each cell by word frequencies.
- Frequency is clearly useful; if *sugar* appears a lot near *apricot*, that's useful information.
- But overly frequent words like *the*, *it*, or *they* are not very informative about the context
- It's a paradox! How can we balance these two conflicting constraints?

# Two common solutions for word weighting

**tf-idf:** tf-idf value for word  $t$  in document  $d$ :

$$w_{t,d} = tf_{t,d} \times idf_t$$

Words like "the" or "it" have very low idf

**PMI:** (Pointwise mutual information)

- $PMI(w, c) = \log_2 \frac{P(w, c)}{P(w)P(c)}$

See if words like "good" appear more often with "great" than we would expect by chance

# Term frequency (tf)

$$\text{tf}_{t,d} = \text{count}(t,d)$$

Instead of using raw count, we squash a bit:

$$\text{tf}_{t,d} = \log_{10}(\text{count}(t,d)+1)$$

# Document frequency (df)

$df_t$  is the number of documents  $t$  occurs in.

(note this is not collection frequency: total count across all documents)

"*Romeo*" is very distinctive for one Shakespeare play:

|        | Collection Frequency | Document Frequency |
|--------|----------------------|--------------------|
| Romeo  | 113                  | 1                  |
| action | 113                  | 31                 |

# Inverse document frequency (idf)

$$idf_t = \log_{10} \left( \frac{n}{df_t} \right)$$

N is the total number of documents in the collection

| Word     | df | idf   |
|----------|----|-------|
| Romeo    | 1  | 1.57  |
| salad    | 2  | 1.27  |
| Falstaff | 4  | 0.967 |
| forest   | 12 | 0.489 |
| battle   | 21 | 0.246 |
| wit      | 34 | 0.037 |
| fool     | 36 | 0.012 |
| good     | 37 | 0     |
| sweet    | 37 | 0     |

# What is a document?

Could be a play or a Wikipedia article

But for the purposes of tf-idf, documents can be **anything**; we often call each paragraph a document!

# Final tf-idf weighted value for a word

Raw counts:

$$w_{t,d} = tf_{t,d} \times idf_t = (\log_{10} count(t, d) + 1) \times \log_{10} \left( \frac{N}{df_t} \right)$$

|               | As You Like It | Twelfth Night | Julius Caesar | Henry V |
|---------------|----------------|---------------|---------------|---------|
| <b>battle</b> | 1              | 0             | 7             | 13      |
| <b>good</b>   | 114            | 80            | 62            | 89      |
| <b>fool</b>   | 36             | 58            | 1             | 4       |
| <b>wit</b>    | 20             | 15            | 2             | 3       |

tf-idf:

|               | As You Like It | Twelfth Night | Julius Caesar | Henry V |
|---------------|----------------|---------------|---------------|---------|
| <b>battle</b> | 0.074          | 0             | 0.22          | 0.28    |
| <b>good</b>   | 0              | 0             | 0             | 0       |
| <b>fool</b>   | 0.019          | 0.021         | 0.0036        | 0.0083  |
| <b>wit</b>    | 0.049          | 0.044         | 0.018         | 0.022   |



# Vector Semantics & Embeddings

TF-IDF

# Vector Semantics & Embeddings

PPMI - Positive Pointwise  
Mutual Information

# Pointwise Mutual Information

## Pointwise mutual information:

Do events  $x$  and  $y$  co-occur more than if they were independent?

$$PMI(w, c) = \log_2 \frac{P(w, c)}{P(w)P(c)}$$

## PMI between two words: (Church & Hanks 1989)

Do words  $x$  and  $y$  co-occur more than if they were independent?

$$PMI(word_1, word_2) = \log_2 \frac{P(word_1, word_2)}{P(word_1)P(word_2)}$$

# Positive Pointwise Mutual Information

- PMI ranges from  $-\infty$  to  $+\infty$
- But the negative values are problematic
  - Things are co-occurring **less than** we expect by chance
  - Unreliable without enormous corpora
    - Imagine  $w_1$  and  $w_2$  whose probability is each  $10^{-6}$
    - Hard to be sure  $p(w_1, w_2)$  is significantly different than  $10^{-12}$
  - Plus it's not clear people are good at “unrelatedness”
- So we just replace negative PMI values by 0
- Positive PMI (**PPMI**) between word1 and word2:

$$\text{PPMI}(word_1, word_2) = \max\left(\log_2 \frac{P(word_1, word_2)}{P(word_1)P(word_2)}, 0\right)$$

# Computing PPMI on a term-context matrix

Matrix  $F$  with  $W$  rows (words) and  $C$  columns (contexts)

$f_{ij}$  is # of times  $w_i$  occurs in context  $c_j$

$$p_{ij} = \frac{f_{ij}}{\sum_{i=1}^W \sum_{j=1}^C f_{ij}}$$
$$p_{*j} = \frac{\sum_{i=1}^W f_{ij}}{\sum_{i=1}^W \sum_{j=1}^C f_{ij}}$$
$$p_{i*} = \frac{\sum_{j=1}^C f_{ij}}{\sum_{i=1}^W \sum_{j=1}^C f_{ij}}$$

|                | computer | data | result | pie | sugar | count(w) |
|----------------|----------|------|--------|-----|-------|----------|
| cherry         | 2        | 8    | 9      | 442 | 25    | 486      |
| strawberry     | 0        | 0    | 1      | 60  | 19    | 80       |
| digital        | 1670     | 1683 | 85     | 5   | 4     | 3447     |
| information    | 3325     | 3982 | 378    | 5   | 13    | 7703     |
| count(context) | 4997     | 5673 | 473    | 512 | 61    | 11716    |

$$pmi_{ij} = \log_2 \frac{p_{ij}}{p_{i*} p_{*j}}$$

$$ppmi_{ij} = \max(\log_2 \frac{p_{ij}}{p_{i*} p_{*j}}, 0)$$

$$p_{ij} = \frac{f_{ij}}{\sum_{i=1}^W \sum_{j=1}^C f_{ij}}$$

|                | computer | data | result | pie | sugar | count(w) |
|----------------|----------|------|--------|-----|-------|----------|
| cherry         | 2        | 8    | 9      | 442 | 25    | 486      |
| strawberry     | 0        | 0    | 1      | 60  | 19    | 80       |
| digital        | 1670     | 1683 | 85     | 5   | 4     | 3447     |
| information    | 3325     | 3982 | 378    | 5   | 13    | 7703     |
| count(context) | 4997     | 5673 | 473    | 512 | 61    | 11716    |

$$P(\text{information}, \text{data}) = 3982/11716 = .3399$$

$$P(\text{information}) = 7703/11716 = .6575$$

$$P(\text{data}) = 5673/11716 = .4842$$

$$p(w_i) = \frac{\sum_{j=1}^C f_{ij}}{N} \quad p(c_j) = \frac{\sum_{i=1}^W f_{ij}}{N}$$

|             | p(w,context) |        |        |        |        | p(w)   |
|-------------|--------------|--------|--------|--------|--------|--------|
|             | computer     | data   | result | pie    | sugar  | p(w)   |
| cherry      | 0.0002       | 0.0007 | 0.0008 | 0.0377 | 0.0021 | 0.0415 |
| strawberry  | 0.0000       | 0.0000 | 0.0001 | 0.0051 | 0.0016 | 0.0068 |
| digital     | 0.1425       | 0.1436 | 0.0073 | 0.0004 | 0.0003 | 0.2942 |
| information | 0.2838       | 0.3399 | 0.0323 | 0.0004 | 0.0011 | 0.6575 |
| p(context)  | 0.4265       | 0.4842 | 0.0404 | 0.0437 | 0.0052 |        |

$$pmi_{ij} = \log_2 \frac{p_{ij}}{p_i * p_j}$$

|             | p(w,context) |        |        |        |        | p(w)   |
|-------------|--------------|--------|--------|--------|--------|--------|
|             | computer     | data   | result | pie    | sugar  | p(w)   |
| cherry      | 0.0002       | 0.0007 | 0.0008 | 0.0377 | 0.0021 | 0.0415 |
| strawberry  | 0.0000       | 0.0000 | 0.0001 | 0.0051 | 0.0016 | 0.0068 |
| digital     | 0.1425       | 0.1436 | 0.0073 | 0.0004 | 0.0003 | 0.2942 |
| information | 0.2838       | 0.3399 | 0.0323 | 0.0004 | 0.0011 | 0.6575 |
| p(context)  | 0.4265       | 0.4842 | 0.0404 | 0.0437 | 0.0052 |        |

$$pmi(\text{information}, \text{data}) = \log_2 \left( \frac{0.3399}{0.6575 \times 0.4842} \right) = 0.0944$$

Resulting PPMI matrix (negatives replaced by 0)

|             | computer | data | result | pie  | sugar |
|-------------|----------|------|--------|------|-------|
| cherry      | 0        | 0    | 0      | 4.38 | 3.30  |
| strawberry  | 0        | 0    | 0      | 4.10 | 5.51  |
| digital     | 0.18     | 0.01 | 0      | 0    | 0     |
| information | 0.02     | 0.09 | 0.28   | 0    | 0     |

# Weighting PMI

PMI is biased toward infrequent events

- Very rare words have very high PMI values

Two solutions:

- Give rare words slightly higher probabilities
- Use add-one smoothing (which has a similar effect)



## Weighting PMI: Giving rare context words slightly higher probability

Raise the context probabilities to  $\alpha = 0.75$ :

$$PPMI_{\alpha}(w, c) = \max(\log_2 \frac{P(w, c)}{P(w)P_{\alpha}(c)}, 0)$$

$$P_{\alpha}(c) = \frac{count(c)^{\alpha}}{\sum_c count(c)^{\alpha}}$$

This helps because  $P_{\alpha}(c) > P(c)$  for rare  $c$

Consider two events,  $P(a) = .99$  and  $P(b) = .01$

$$P_{\alpha}(a) = \frac{.99^{.75}}{.99^{.75} + .01^{.75}} = .97 \quad P_{\alpha}(b) = \frac{.01^{.75}}{.01^{.75} + .01^{.75}} = .03$$

# Vector Semantics & Embeddings

Word2vec

# Sparse versus dense vectors

tf-idf (or PMI) vectors are

- **long** (length  $|V| = 20,000$  to  $50,000$ )
- **sparse** (most elements are zero)

Alternative: learn vectors which are

- **short** (length 50-1000)
- **dense** (most elements are non-zero)

# Sparse versus dense vectors

## Why dense vectors?

- Short vectors may be easier to use as **features** in machine learning (fewer weights to tune)
- Dense vectors may **generalize** better than explicit counts
- Dense vectors may do better at capturing synonymy:
  - *car* and *automobile* are synonyms; but are distinct dimensions
  - a word with *car* as a neighbor and a word with *automobile* as a neighbor should be similar, but aren't
- **In practice, they work better**

# Common methods for getting short dense vectors

## “Neural Language Model”-inspired models

- Word2vec (skipgram, CBOW), GloVe

## Singular Value Decomposition (SVD)

- A special case of this is called LSA – Latent Semantic Analysis

## Alternative to these "static embeddings":

- Contextual Embeddings (ELMo, BERT)
- Compute distinct embeddings for a word in its context
- Separate embeddings for each token of a word

# Simple static embeddings you can download!

Word2vec (Mikolov et al)

<https://code.google.com/archive/p/word2vec/>

GloVe (Pennington, Socher, Manning)

<http://nlp.stanford.edu/projects/glove/>

# Word2vec

Popular embedding method

Very fast to train

Code available on the web

Idea: **predict** rather than **count**

Word2vec provides various options. We'll do:

**skip-gram with negative sampling (SGNS)**

# Word2vec

Instead of **counting** how often each word  $w$  occurs near "*apricot*"

- Train a classifier on a binary **prediction** task:
  - Is  $w$  likely to show up near "*apricot*"?

We don't actually care about this task

- But we'll take the learned classifier weights as the word embeddings

Big idea: **self-supervision**:

- A word  $c$  that occurs near *apricot* in the corpus acts as the gold "correct answer" for supervised learning
- No need for human labels
- Bengio et al. (2003); Collobert et al. (2011)



# Approach: predict if candidate word $c$ is a "neighbor"

1. Treat the target word  $t$  and a neighboring context word  $c$  as **positive examples**.
2. Randomly sample other words in the lexicon to get negative examples
3. Use logistic regression to train a classifier to distinguish those two cases
4. Use the learned weights as the embeddings

# Skip-Gram Training Data

Assume a  $\pm 2$  word window, given training sentence:

...lemon, a [tablespoon of apricot jam, a] pinch...

c1                      c2 [target]    c3    c4

# Skip-Gram Classifier

(assuming a +/- 2 word window)

...lemon, a [tablespoon of apricot jam, a] pinch...

c1 c2 [target] c3 c4

Goal: train a classifier that is given a candidate (**w**ord, **c**ontext) pair

(apricot, jam)

(apricot, aardvark)

...

And assigns each pair a probability:

$$P(+ | w, c)$$

$$P(- | w, c) = 1 - P(+ | w, c)$$

# Similarity is computed from dot product

Remember: two vectors are similar if they have a high dot product

- Cosine is just a normalized dot product

So:

- $\text{Similarity}(w, c) \propto w \cdot c$

We'll need to normalize to get a probability

- (cosine isn't a probability either)

# Turning dot products into probabilities

$$\text{Sim}(\mathbf{w}, \mathbf{c}) \approx \mathbf{w} \cdot \mathbf{c}$$

To turn this into a probability

We'll use the sigmoid from logistic regression:

$$P(+ | \mathbf{w}, \mathbf{c}) = \sigma(\mathbf{c} \cdot \mathbf{w}) = \frac{1}{1 + \exp(-\mathbf{c} \cdot \mathbf{w})}$$

$$\begin{aligned} P(- | \mathbf{w}, \mathbf{c}) &= 1 - P(+ | \mathbf{w}, \mathbf{c}) \\ &= \sigma(-\mathbf{c} \cdot \mathbf{w}) = \frac{1}{1 + \exp(\mathbf{c} \cdot \mathbf{w})} \end{aligned}$$

# How Skip-Gram Classifier computes $P(+ | w, c)$

$$P(+ | w, c) = \sigma(c \cdot w) = \frac{1}{1 + \exp(-c \cdot w)}$$

This is for one context word, but we have lots of context words.  
We'll assume independence and just multiply them:

$$P(+ | w, c_{1:L}) = \prod_{i=1}^L \sigma(c_i \cdot w)$$

$$\log P(+ | w, c_{1:L}) = \sum_{i=1}^L \log \sigma(c_i \cdot w)$$

# Skip-gram classifier: summary

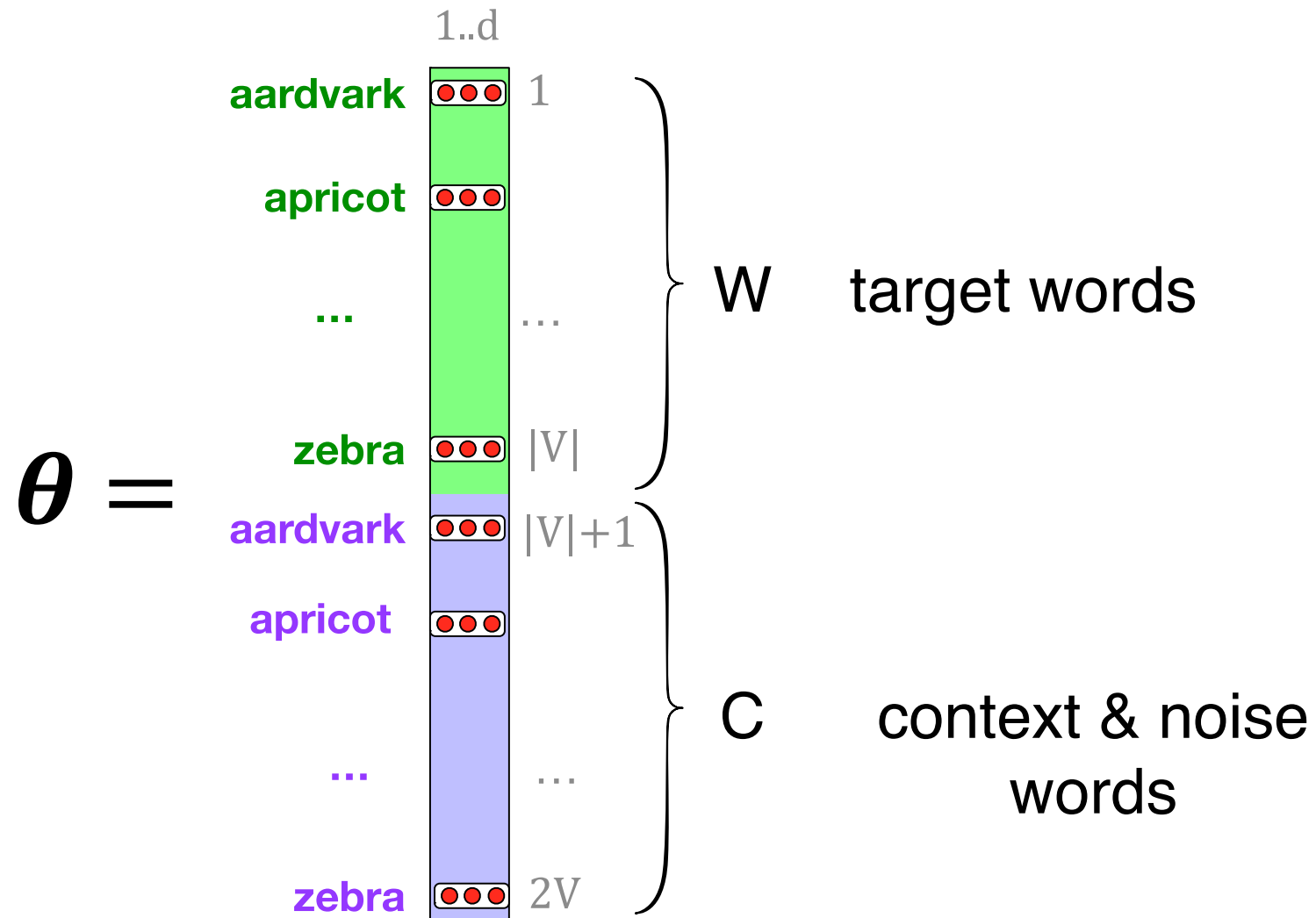
A probabilistic classifier, given

- a test target word  $w$
- its context window of  $L$  words  $c_{1:L}$

Estimates probability that  $w$  occurs in this window based on similarity of  $w$  (embeddings) to  $c_{1:L}$  (embeddings).

To compute this, we just need embeddings for all the words.

These embeddings will need: a set for  $w$ , a set for  $c$





# Starting Embeddings

- The embeddings themselves start out as random values
  - For example, for  $V=10,000$  words and vector length of 100, a  $10,000 \times 100$  matrix would be created with random values
- Then, the training begins on the first target word and context window
  - Using Logistic Regression and Gradient Descent, all the random weights are altered to become more similar
- This is repeated through all the training data to shape the embeddings

# Updating Embeddings

- For each new input, the system predicts what context words are likely to appear nearby, randomly grabs some words in  $V$  that would not appear with the target word (very dissimilar vectors). Then:
  - **Forward Pass:** Uses current embeddings to predict the probabilities of context words appearing nearby.
    - This prediction is typically done using **softmax**
  - **Loss Calculation:** Calculates loss function based on predicted probabilities and the actual context words
  - **Backpropagation:** Stochastic gradient descent to minimize loss
    - **Embedding Update:** Embeddings are updated in the direction that reduces the loss.
      - Embeddings for words predicted poorly adjusted more
      - Embeddings for words predicted well adjusted less.

# Vector Semantics & Embeddings

Word2vec

# Vector Semantics & Embeddings

Word2vec: Learning the  
embeddings

# Skip-Gram Training data

...lemon, a [tablespoon of apricot jam, a] pinch...

c1 c2 [target] c3 c4



**positive examples +**

| t       | c          |
|---------|------------|
| apricot | tablespoon |
| apricot | of         |
| apricot | jam        |
| apricot | a          |

# Skip-Gram Training data

...lemon, a [tablespoon of apricot jam, a] pinch...

c1 c2 [target] c3 c4



**positive examples +**

| t       | c          |
|---------|------------|
| apricot | tablespoon |
| apricot | of         |
| apricot | jam        |
| apricot | a          |

For each positive example we'll grab k negative examples, sampling by frequency

# Skip-Gram Training data

...lemon, a [tablespoon of apricot jam, a] pinch...

c1                      c2 [target]    c3    c4

↑

## positive examples +

| t       | c          |
|---------|------------|
| apricot | tablespoon |
| apricot | of         |
| apricot | jam        |
| apricot | a          |

## negative examples -

| t       | c        | t       | c       |
|---------|----------|---------|---------|
| apricot | aardvark | apricot | seven   |
| apricot | my       | apricot | forever |
| apricot | where    | apricot | dear    |
| apricot | coaxial  | apricot | if      |

# Word2vec: how to learn vectors

Given the set of positive and negative training instances, and an initial set of embedding vectors

The goal of learning is to adjust those word vectors such that we:

- **Maximize** the similarity of the **target word**, **context word** pairs  $(w, c_{\text{pos}})$  drawn from the positive data
- **Minimize** the similarity of the  $(w, c_{\text{neg}})$  pairs drawn from the negative data.



# Loss function for one $w$ with $c_{pos}$ , $c_{neg1} \dots c_{negk}$

Maximize the similarity of the target with the actual context words, and minimize the similarity of the target with the  $k$  negative sampled non-neighbor words.

$$\begin{aligned} L_{CE} &= -\log \left[ P(+ | w, c_{pos}) \prod_{i=1}^k P(- | w, c_{neg_i}) \right] \\ &= - \left[ \log P(+ | w, c_{pos}) + \sum_{i=1}^k \log P(- | w, c_{neg_i}) \right] \\ &= - \left[ \log P(+ | w, c_{pos}) + \sum_{i=1}^k \log(1 - P(w, c_{neg_i})) \right] \\ &= - \left[ \log \sigma(c_{pos} \cdot w) + \sum_{i=1}^k \log \sigma(-c_{neg_i} \cdot w) \right] \end{aligned}$$

# Learning the classifier

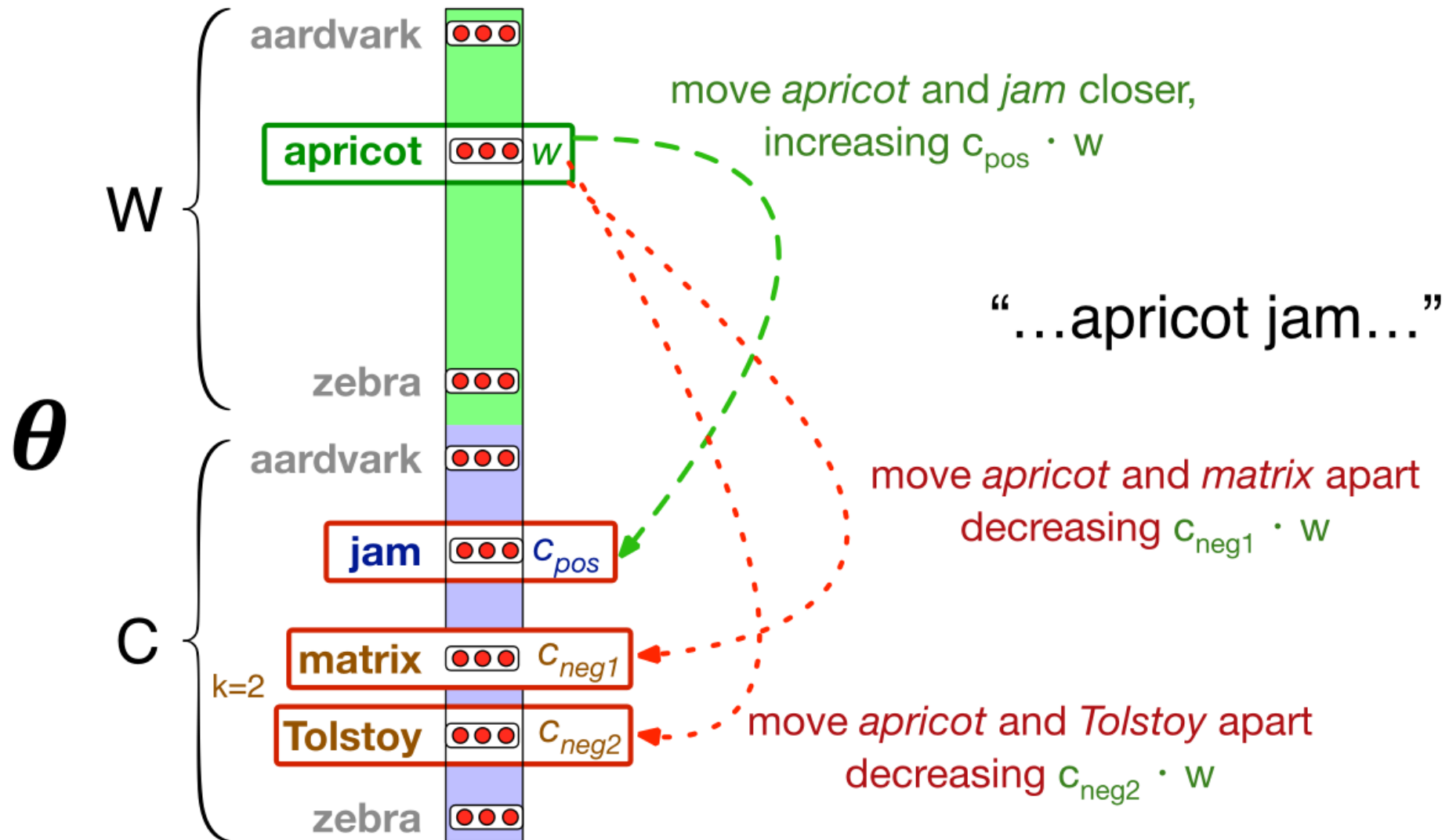
How to learn?

- Stochastic gradient descent!

We'll adjust the word weights to

- make the positive pairs more likely
- and the negative pairs less likely,
- over the entire training set.

# Intuition of one step of gradient descent



# Two sets of embeddings

SGNS learns two sets of embeddings

Target embeddings matrix  $W$

Context embedding matrix  $C$

It's common to just add them together, representing word  $i$  as the vector  $w_i + c_i$

Summary: How to learn word2vec (skip-gram) embeddings

Start with  $V$  random  $d$ -dimensional vectors as initial embeddings

Train a classifier based on embedding similarity

- Take a corpus and take pairs of words that co-occur as positive examples
- Take pairs of words that don't co-occur as negative examples
- Train the classifier to distinguish these by slowly adjusting all the embeddings to improve the classifier performance
- Throw away the classifier code and keep the embeddings.

# Vector Semantics & Embeddings

Word2vec: Learning the  
embeddings

# Vector Semantics & Embeddings

## Properties of Embeddings

# The kinds of neighbors depend on window size

**Small windows** ( $C = +/- 2$ ) : nearest words are syntactically similar words in same taxonomy

- *Hogwarts* nearest neighbors are other fictional schools
- *Sunnydale, Evernight, Blandings*

**Large windows** ( $C = +/- 5$ ) : nearest words are related words in same semantic field

- *Hogwarts* nearest neighbors are Harry Potter world:
- *Dumbledore, half-blood, Malfoy*

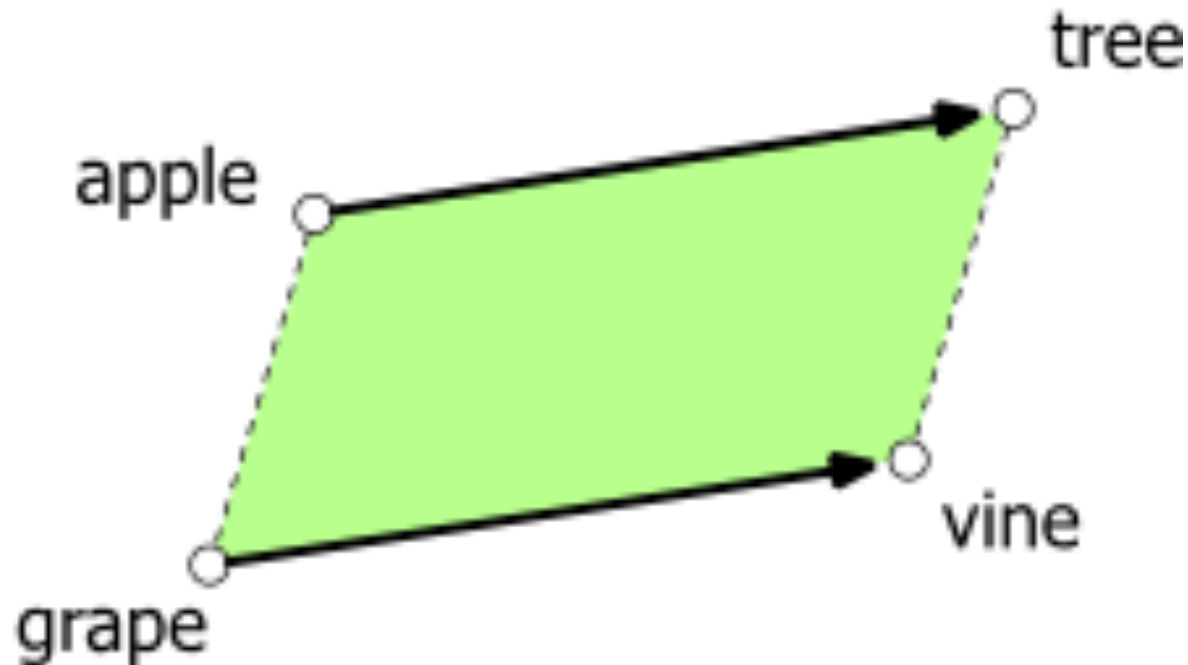


# Analogical relations

The classic parallelogram model of analogical reasoning  
(Rumelhart and Abrahamson 1973)

To solve: *"apple is to tree as grape is to \_\_\_\_\_"*

Add  $\overrightarrow{\text{tree} - \text{apple}}$  to  $\overrightarrow{\text{grape}}$  to get *vine*



# Analogical relations via parallelogram

The parallelogram method can solve analogies with both sparse and dense embeddings (Turney and Littman 2005, Mikolov et al. 2013b)

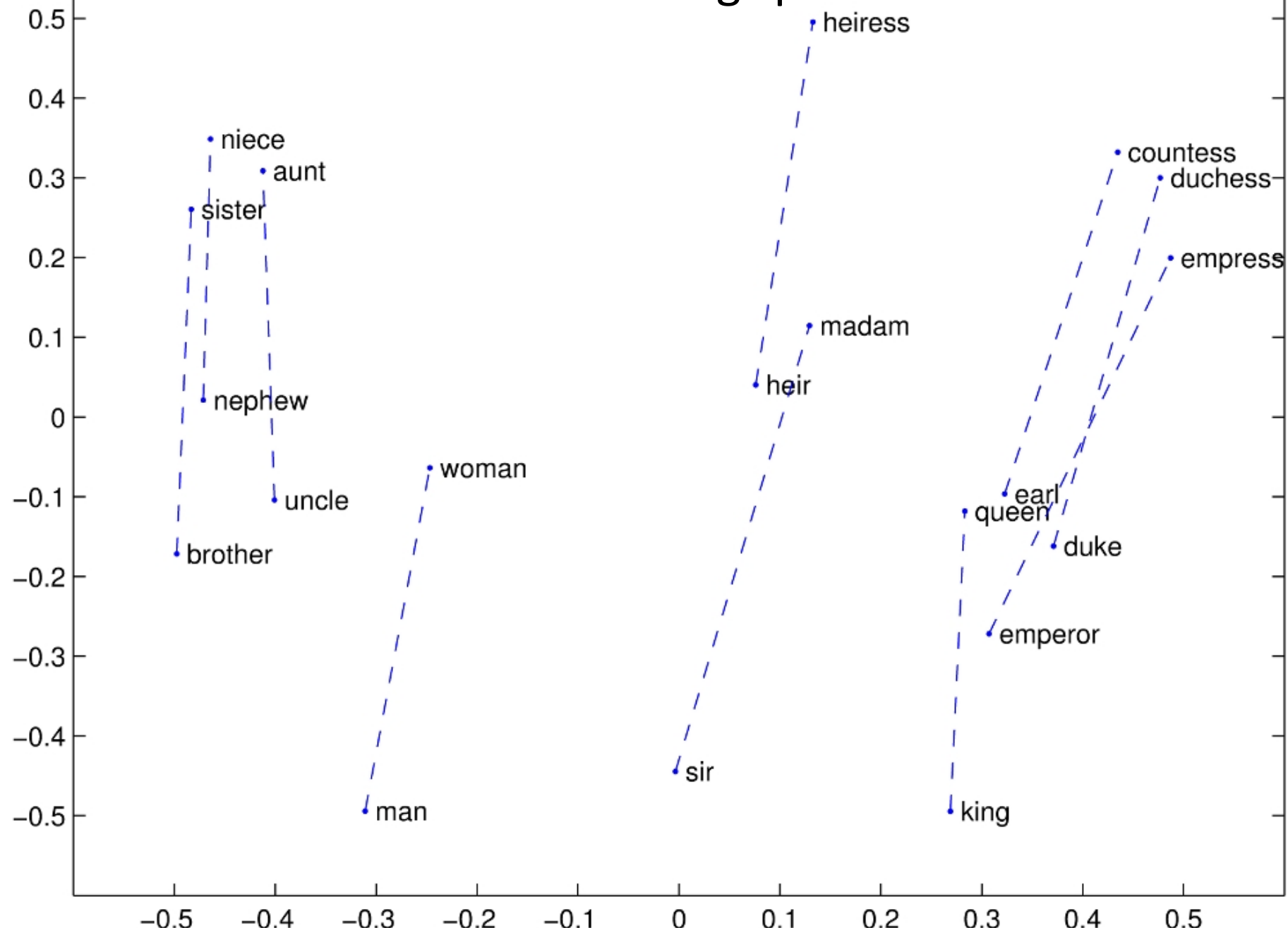
$\overrightarrow{\text{king}} - \overrightarrow{\text{man}} + \overrightarrow{\text{woman}}$  is close to  $\overrightarrow{\text{queen}}$

$\overrightarrow{\text{Paris}} - \overrightarrow{\text{France}} + \overrightarrow{\text{Italy}}$  is close to  $\overrightarrow{\text{Rome}}$

For a problem  $a:a^*:b:b^*$ , the parallelogram method is:

$$\hat{b}^* = \underset{x}{\operatorname{argmin}} \operatorname{distance}(\mathbf{x}, \mathbf{b} - \mathbf{a} + \mathbf{a}^*)$$

# Structure in GloVe Embedding space



# Caveats with the parallelogram method

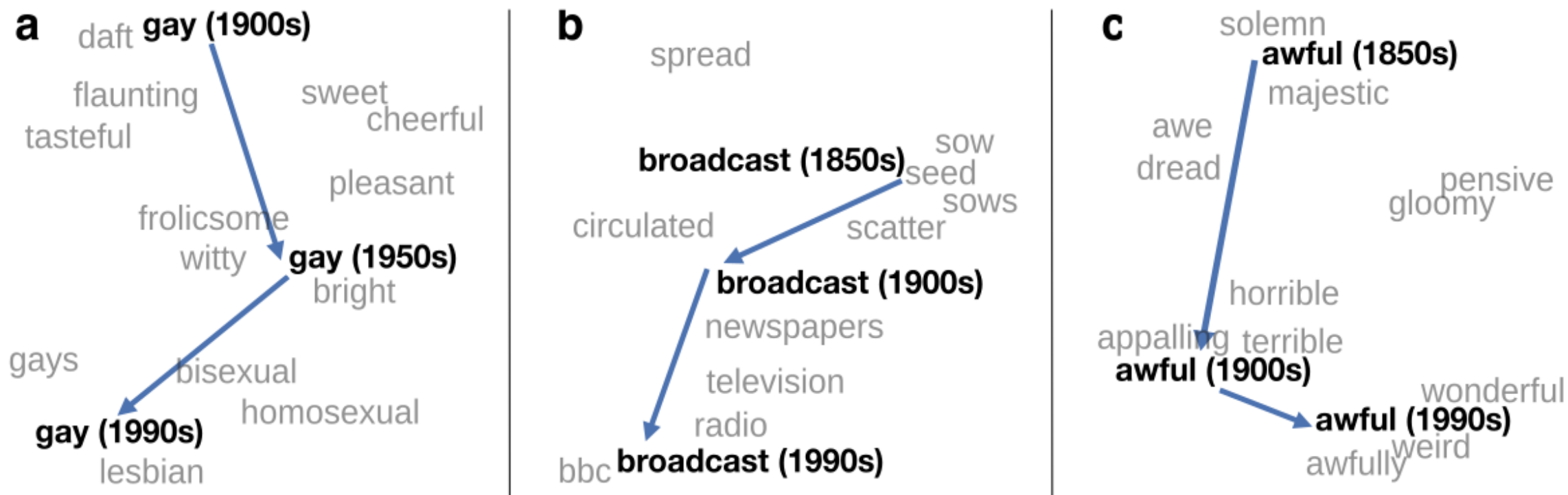
It only seems to work for frequent words, small distances and certain relations (relating countries to capitals, or parts of speech), but not others. (Linzen 2016, Gladkova et al. 2016, Ethayarajh et al. 2019a)

Understanding analogy is an open area of research (Peterson et al. 2020)

# Embeddings as a window onto historical semantics

Train embeddings on different decades of historical text to see meanings shift

~30 million books, 1850-1990, Google Books data



William L. Hamilton, Jure Leskovec, and Dan Jurafsky. 2016. Diachronic Word Embeddings Reveal Statistical Laws of Semantic Change. Proceedings of ACL.

# Embeddings reflect cultural bias!

Bolukbasi, Tolga, Kai-Wei Chang, James Y. Zou, Venkatesh Saligrama, and Adam T. Kalai. "Man is to computer programmer as woman is to homemaker? debiasing word embeddings." In *NeurIPS*, pp. 4349-4357. 2016.

Ask “Paris : France :: Tokyo : x”

- x = Japan

Ask “father : doctor :: mother : x”

- x = nurse

Ask “man : computer programmer :: woman : x”

- x = homemaker

Algorithms that use embeddings as part of e.g., hiring searches for programmers, might lead to bias in hiring

# Historical embedding as a tool to study cultural biases

Garg, N., Schiebinger, L., Jurafsky, D., and Zou, J. (2018). Word embeddings quantify 100 years of gender and ethnic stereotypes. *Proceedings of the National Academy of Sciences* 115(16), E3635–E3644.

- Compute a **gender or ethnic bias** for each adjective: e.g., how much closer the adjective is to "woman" synonyms than "man" synonyms, or names of particular ethnicities
  - Embeddings for **competence** adjective (*smart, wise, brilliant, resourceful, thoughtful, logical*) are biased toward men, a bias slowly decreasing 1960-1990
  - Embeddings for **dehumanizing** adjectives (barbaric, monstrous, bizarre) were biased toward Asians in the 1930s, bias decreasing over the 20<sup>th</sup> century.
- These match the results of old surveys done in the 1930s

# Vector Semantics & Embeddings

## Properties of Embeddings